



`alittlebyte.swe@gmail.com`

Specifica Tecnica

Gruppo	aLittleByte (gruppo 19)
Versione	1.0.0
Redattori	A. Maule, A. Oloni, E. Bellet, V. Y. K. Fernandes, A. Gastaldello, L. Soligo, D. Belluz
Verificatori	V. Y. K. Fernandes, E. Bellet, D. Belluz, A. Oloni, A. Maule, L. Soligo, Angelica Gastaldello
Destinatari	Prof. Tullio Vardanega, Prof. Riccardo Cardin
Data	03/09/2026

Registro delle Modifiche

Versione	Data	Descrizione	Redattore	Verificatore	Approvatore
1.0.0	03/09/2026	Approvazione specifica tecnica	-	-	V. Y. K. Fernandes
0.18.1	20/08/2026	Fix	A. Oloni	Diego Belluz	-
0.18.0	19/08/2026	Aggiunta diagrammi	Angela Maule	Angelica Gastaldello	-
0.17.0	18/08/2026	Database (schema ER, schema relazionale e descrizione tabelle)	V. Y. K. Fernandes	V. Y. K. Fernandes	-
0.16.0	17/08/2026	Design Pattern (backend e frontend)	Alex Oloni	Diego Belluz	-
0.15.0	16/08/2026	Architettura Frontend (modelli di dati, servizi e componenti UI)	Angelica Gastaldello	Eleonora Bellet	-
0.14.0	15/08/2026	Architettura Frontend (principi di progettazione e pattern MV-VM)	D. Belluz	Angelica Gastaldello	-
0.13.0	14/08/2026	Descrizione architettura Backend (Modulo Communications e classi condivise)	Eleonora Bellet	A. Oloni	-
0.12.0	13/08/2026	Descrizione architettura Backend (Modulo Documents)	A. Oloni	Angelica Gastaldello	-
0.11.0	12/08/2026	Architettura del Sistema (deployment, containerizzazione e stile architetturale)	Eleonora Bellet	Leonardo Soligo	-
0.10.0	11/08/2026	Architettura del Sistema (architettura logica interna e componenti)	V. Y. K. Fernandes	Diego Belluz	-
0.9.0	10/08/2026	Architettura del Sistema (contesto e architettura applicativa)	Angela Maule	A. Oloni	-
0.8.0	09/08/2026	API (State API, Communications API, Documents API)	Diego Belluz	Leonardo Soligo	-
0.7.0	08/08/2026	Tecnologie (osservabilità, CI e quality gate)	Angela Maule	Eleonora Bellet	-
0.6.0	07/08/2026	Tecnologie (edge e web server, servizi cloud AWS, persistenza dei dati)	L. Soligo	Diego Belluz	-
0.5.0	06/08/2026	Tecnologie (contratto API, ambiente locale e provisioning)	Eleonora Bellet	Leonardo Soligo	-
0.4.0	05/08/2026	Tecnologie (linguaggi di programmazione, framework e librerie)	Leonardo Soligo	A. Maule	-
0.3.0	04/08/2026	Introduzione (glossario e riferimenti)	Eleonora Bellet	Angelica Gastaldello	-

Versione	Data	Descrizione	Redattore	Verificatore	Approvatore
0.2.0	03/08/2026	Introduzione (contenuto, scopo e struttura del documento)	Angelica Gastaldello	A. Maule	-
0.1.0	02/08/2026	Stesura prima versione	Angela Maule	Diego Belluz	-
0.0.4	01/08/2026	Bozza preliminare del capitolo Tecnologie, con il primo elenco dello stack proposto	Angela Maule	V. Y. K. Fernandes	-
0.0.3	26/07/2026	Prima bozza della sezione Introduzione (scopo del prodotto e struttura del documento)	Eleonora Bellet	A. Oloni	-
0.0.2	21/07/2026	Redazione dell'indice delle sezioni e definizione della struttura complessiva del documento	Leonardo Soligo	Angelica Gastaldello	-
0.0.1	16/07/2026	Creazione del template del documento: frontespizio, stile delle tabelle e registro delle modifiche	Diego Belluz	Angela Maule	-

Indice

1	Introduzione	7
1.1	Contenuto del documento	7
1.2	Scopo del prodotto	7
1.3	Struttura del documento	7
1.4	Glossario	8
1.5	Riferimenti	8
1.5.1	Riferimenti normativi	8
1.5.2	Riferimenti informativi	9
2	Tecnologie	9
2.1	Linguaggi di programmazione	9
2.2	Framework	10
2.3	Librerie	10
2.4	Contratto API	11
2.5	Ambiente locale e provisioning	12
2.6	Edge e web server	13
2.7	Servizi cloud AWS	14
2.8	Tecnologie per la persistenza dei dati	15
2.9	Osservabilità	16
2.10	CI e quality gate	17
3	API	18
3.1	State API	19
3.2	Communications API	19
3.3	Documents API	21
4	Architettura del Sistema	22
4.1	Contesto di Sistema	22
4.2	Architettura Applicativa	23
4.3	Architettura Logica Interna	24
4.3.1	Frontend: Interfaccia Utente	24
4.3.2	Backend: Architettura Esagonale	24
4.3.2.1	Adapter primari	25
4.3.2.2	Dominio e applicazione	25
4.3.2.3	Adapter secondari	25
4.3.2.4	Esempio: il dominio Documents	25
4.4	Dettaglio dei Componenti	26
4.4.1	Modulo Documents	26
4.4.2	Modulo Communications	27
4.5	Architettura di Deployment	27
4.6	Containerizzazione con Docker	28
4.7	Stile architetturale	28

5	Descrizione architettura (Backend)	29
5.1	Modulo Documents (AI Co-Pilot)	29
5.1.1	Diagrammi delle classi parziali del modulo Documents	31
5.1.2	Diagrammi di sequenza: caricamento ed elaborazione di un documento	33
5.1.3	Tutte le classi del modulo Documents	36
5.1.3.1	DocumentController	36
5.1.3.2	DocumentPreviewController, SendMessageController	37
5.1.3.3	DocumentWorkflowTaskHandler	38
5.1.3.4	OriginalDocument	39
5.1.3.5	SubDocument	41
5.1.3.6	UploadDocumentService	42
5.1.3.7	StartDocumentWorkflowService	42
5.1.3.8	RunOcrService	44
5.1.3.9	ProcessDocumentService	44
5.1.3.10	ExtractSubDocumentFieldsService	45
5.1.3.11	FinalizeDocumentWorkflowService	46
5.1.3.12	PollDocumentProgressService	47
5.1.3.13	DocumentReviewController	47
5.1.3.14	ReviewDocumentService	48
5.1.3.15	PreviewDocumentService	49
5.1.3.16	SendMessageService	49
5.1.3.17	ListDocumentsService, DeleteDocumentService	50
5.1.3.18	EloquentDocumentRepository	51
5.1.3.19	Adapter AI e OCR: TextractOcrAdapter, BedrockDocu- mentAiAdapter	53
5.1.3.20	FlysystemDocumentStorageAdapter, DompdfSendMessa- geRenderer	54
5.1.3.21	LaravelDocumentEventDispatcher	55
5.2	Modulo Communications (AI Assistant Generativo)	56
5.2.1	Diagrammi delle classi parziali del modulo Communications	57
5.2.2	Diagramma di sequenza: generazione di una comunicazione	61
5.2.3	Tutte le classi del modulo Communications	62
5.2.3.1	CommunicationController	62
5.2.3.2	CommunicationCoverController	63
5.2.3.3	CommunicationExportController	64
5.2.3.4	CommunicationRatingController	64
5.2.3.5	CommunicationStreamController	65
5.2.3.6	CommunicationWorkflowTaskHandler	65
5.2.3.7	Communication	66
5.2.3.8	GenerateCommunicationService	68
5.2.3.9	StartCommunicationWorkflowService	69
5.2.3.10	GenerateCommunicationTextService, GenerateCommuni- cationCoverService	70
5.2.3.11	FinalizeCommunicationService	71
5.2.3.12	PollCommunicationProgressService	71
5.2.3.13	CommunicationDraftService	72

5.2.3.14	RateCommunicationService	73
5.2.3.15	UpdateCommunicationCoverService	73
5.2.3.16	DownloadCommunicationCoverService, ExportCommuni- cationService	74
5.2.3.17	ListCommunicationsService, DeleteCommunicationService	75
5.2.3.18	PromptConfigurationController	76
5.2.3.19	PromptConfigurationService	77
5.2.3.20	EloquentCommunicationRepository	77
5.2.3.21	EloquentPromptConfigurationRepository	78
5.2.3.22	BedrockCommunicationAiAdapter	79
5.2.3.23	FlysystemCommunicationCoverAdapter, DompdfCommuni- cationPdfRenderer	80
5.2.3.24	LaravelCommunicationEventDispatcher	81
5.3	Classi condivise	81
5.3.1	Le principali classi condivise	81
5.3.1.1	WorkflowTaskRegistry	81
5.3.1.2	WorkflowTaskRunner	82
5.3.1.3	ConsumeWorkflowTasks	83
5.3.1.4	WorkflowTaskHeartbeat	83
5.3.1.5	SfnWorkflowEngineAdapter	84
5.3.1.6	WorkflowContext	84
5.3.1.7	Actor	85
5.3.1.8	MvpUserProvider	86
5.3.1.9	SystemClock	86
5.3.1.10	RandomUuidGenerator	87
5.3.1.11	LaravelTransactionManager	87
5.3.1.12	AuditLogger	87
6	Architettura Frontend	88
6.1	Principi di progettazione	88
6.2	Il pattern MVVM	88
6.2.1	Esempio di pattern MVVM nel progetto	90
6.2.2	Two-way data binding	91
6.2.3	AssistantPageViewModel	92
6.2.4	CopilotPageViewModel	96
6.2.5	OverviewPageViewModel	98
6.2.6	MvpStateStore (stato condiviso fra i ViewModel)	100
6.3	Definizione dei modelli di dati	102
6.3.1	assistant.model.ts	102
6.3.1.1	CommunicationDraftForm	102
6.3.1.2	CommunicationGenerationPhase	102
6.3.1.3	CommunicationGenerationProgress	102
6.3.1.4	GeneratedDraft	103
6.3.1.5	RateDraftPayload	103
6.3.1.6	communicationTones e communicationStyles	103
6.3.2	Tipi del modulo Co-Pilot	104

6.3.2.1	DocumentUploadPhase	104
6.3.2.2	DocumentFilters	105
6.3.2.3	ConfidenceCriterion	105
6.3.2.4	DocumentUploadMetadata	105
6.3.2.5	DocumentUploadProgress	106
6.3.2.6	DocumentPage	106
6.3.2.7	DocumentPreviewStatus	106
6.3.2.8	DocumentFilterFormValue	107
6.3.2.9	createDocumentFilterForm e toDocumentFilters	107
6.4	Servizi di comunicazione con il backend	107
6.4.1	AssistantService	107
6.4.2	DocumentWorkflowService	109
6.4.3	SseClient	111
6.5	Interceptor HTTP e servizi trasversali	111
6.6	Serializzazione e deserializzazione dei dati	112
6.7	Micro componenti UI	113
6.7.1	ButtonComponent	113
6.7.2	AlertComponent, ErrorStateComponent	114
6.7.3	EmptyStateComponent, LoadingStateComponent, StatusBadgeComponent, StatusDotComponent	115
6.7.4	AttentionNoteComponent	115
6.7.5	MetricCardComponent, MetricsPanelComponent	116
6.7.6	Le forme di scheda del pannello metriche	117
6.7.7	MetricCompositionComponent	119
6.7.8	StageProgressComponent	120
6.7.9	StarRating	120
6.7.10	SelectFieldComponent, TextAreaFieldComponent	121
6.7.11	FileDropzoneComponent	122
6.8	Componenti di feature	122
6.8.1	Componenti di feature dell'AI Assistant Generativo	122
6.8.2	Componenti di feature dell'AI Co-Pilot	125
6.8.3	SectionComponent	127
6.9	Macro componenti UI (pagine)	128
6.9.1	AssistantPage	128
6.9.2	CopilotPage	131
6.9.3	OverviewPage	133
7	Design Pattern	135
7.1	Design Pattern Backend	136
7.1.1	Adapter	136
7.1.2	Strategy	137
7.1.3	Facade	138
7.1.4	Factory Method	140
7.1.5	Singleton	141
7.1.6	Observer	142
7.1.7	Command	143

7.1.8	Pattern nativi del framework	145
7.2	Design Pattern Frontend	146
7.2.1	Facade	146
7.2.2	Presentation Model	147
8	Database	149
8.1	Schema ER	149
8.2	Schema Relazionale	151
8.3	Schema ERD (Entity-Relationship Diagram)	152
8.4	Descrizione tabelle	154
8.4.1	communications	154
8.4.2	original_documents	155
8.4.3	sub_documents	157
8.4.4	extracted_data	157
8.4.5	workflow_tasks	158
8.4.6	prompt_configurations	159
8.4.7	audit_events	160
9	Requisiti Funzionali Soddisfatti	160
9.1	Completamento requisiti	161
9.2	Tabella di tracciamento dei requisiti	161

Indice delle Figure

1	System Context Diagram	23
2	Container Diagram	24
3	Architettura Esagonale: Esempio Dominio Documents	25
4	Component Diagram: Pipeline centrale del modulo Documents	27
5	Component Diagram: Pipeline centrale del modulo Communications	27
6	Diagramma delle classi: Modulo Documents	30
7	Diagramma classi parziale: DocumentController	31
8	Diagramma classi parziale: Flusso di upload e avvio	32
9	Diagramma classi parziale: Flusso di elaborazione OCR/AI	32
10	Diagramma classi parziale: Flusso di revisione e invio	33
11	Diagramma classi parziale: Flusso di consultazione ed eliminazione	33
12	Diagramma di sequenza: Upload e avvio della pipeline	34
13	Diagramma di sequenza: Elaborazione asincrona e streaming SSE	35
14	Classe: DocumentController	36
15	Classe: DocumentPreviewController	37
16	Classe: SendMessageController	37
17	Classe: DocumentWorkflowTaskHandler	38
18	Classe: OriginalDocument	40
19	Classe: SubDocument	41
20	Classe: UploadDocumentService	42
21	Classe: StartDocumentWorkflowService	43
22	Classe: RunOcrService	44
23	Classe: ProcessDocumentService	45
24	Classe: ExtractSubDocumentFieldsService	46
25	Classe: FinalizeDocumentWorkflowService	47
26	Classe: PollDocumentProgressService	47
27	Classe: DocumentReviewController	48
28	Classe: ReviewDocumentService	48
29	Classe: PreviewDocumentService	49
30	Classe: SendMessageService	50
31	Classe: ListDocumentsService	51
32	Classe: DeleteDocumentService	51
33	Classe: EloquentDocumentRepository	52
34	Classe: TextractOcrAdapter	53
35	Classe: BedrockDocumentAiAdapter	54
36	Classe: FlysystemDocumentStorageAdapter	55
37	Classe: DompdfSendMessageRenderer	55
38	Classe: LaravelDocumentEventDispatcher	55
39	Diagramma delle classi: Modulo Communications	57
40	Diagramma classi parziale: CommunicationController	58
41	Diagramma classi parziale: Flusso di generazione e avvio	58
42	Diagramma classi parziale: Flusso di generazione asincrona	59
43	Diagramma classi parziale: Flusso di copertina, esportazione e valutazione	59

44	Diagramma classi parziale: Flusso di consultazione ed eliminazione	60
45	Diagramma classi parziale: Configurazioni di prompt	60
46	Diagramma di sequenza: Generazione di una comunicazione	61
47	Classe: CommunicationController	62
48	Classe: CommunicationCoverController	63
49	Classe: CommunicationExportController	64
50	Classe: CommunicationRatingController	64
51	Classe: CommunicationStreamController	65
52	Classe: CommunicationWorkflowTaskHandler	66
53	Classe: Communication	67
54	Classe: GenerateCommunicationService	68
55	Classe: StartCommunicationWorkflowService	69
56	Classe: GenerateCommunicationTextService	70
57	Classe: GenerateCommunicationCoverService	70
58	Classe: FinalizeCommunicationService	71
59	Classe: PollCommunicationProgressService	72
60	Classe: CommunicationDraftService	72
61	Classe: RateCommunicationService	73
62	Classe: UpdateCommunicationCoverService	74
63	Classe: DownloadCommunicationCoverService	75
64	Classe: ExportCommunicationService	75
65	Classe: ListCommunicationsService	76
66	Classe: DeleteCommunicationService	76
67	Classe: PromptConfigurationController	76
68	Classe: PromptConfigurationService	77
69	Classe: EloquentCommunicationRepository	78
70	Classe: EloquentPromptConfigurationRepository	79
71	Classe: BedrockCommunicationAiAdapter	79
72	Classe: FlysystemCommunicationCoverAdapter	80
73	Classe: DompdfCommunicationPdfRenderer	80
74	Classe: LaravelCommunicationEventDispatcher	81
75	Classe: WorkflowTaskRegistry	82
76	Classe: WorkflowTaskRunner	82
77	Classe: ConsumeWorkflowTasks	83
78	Classe: WorkflowTaskHeartbeat	83
79	Classe: SfnWorkflowEngineAdapter	84
80	Classe: WorkflowContext	85
81	Classe: Actor	85
82	Classe: MvpUserProvider	86
83	Classe: SystemClock	86
84	Classe: RandomUuidGenerator	87
85	Classe: LaravelTransactionManager	87
86	Classe: AuditLogger	87
87	Il pattern MVVM	88
88	Esempio di pattern MVVM: AssistantPage	91
89	Classe: AssistantPageViewModel	93

90	Classe: CopilotPageViewModel	96
91	Classe: OverviewPageViewModel	99
92	Classe: MvpStateStore	100
93	Tipo: CommunicationDraftForm	102
94	Tipo: CommunicationGenerationPhase	102
95	Tipo: CommunicationGenerationProgress	103
96	Tipo: GeneratedDraft	103
97	Tipo: RateDraftPayload	103
98	Tipo: communicationTones	104
99	Tipo: communicationStyles	104
100	Tipo: DocumentUploadPhase	105
101	Tipo: DocumentFilters	105
102	Tipo: ConfidenceCriterion	105
103	Tipo: DocumentUploadMetadata	106
104	Tipo: DocumentUploadProgress	106
105	Tipo: DocumentPage	106
106	Tipo: DocumentPreviewStatus	107
107	Tipo: DocumentFilterFormValue	107
108	Classe: AssistantService	108
109	Classe: DocumentWorkflowService	110
110	Classe: SseClient	111
111	Classi: apiBaseUrlInterceptor, correlationInterceptor, errorLoggingInter- ceptor, LoggerService, GlobalErrorHandler, ThemeService	112
112	Classe: ButtonComponent	113
113	Classi: AlertComponent, ErrorStateComponent	114
114	Classi: EmptyStateComponent, LoadingStateComponent, StatusBadge- Component, StatusDotComponent	115
115	Classe: AttentionNoteComponent	116
116	Classi: MetricCardComponent, MetricsPanelComponent	116
117	Classi: le cinque forme di scheda disacciate da MetricsPanelComponent .	118
118	Classe: MetricCompositionComponent	119
119	Classe: StageProgressComponent	120
120	Classe: StarRating	121
121	Classi: SelectFieldComponent, TextAreaFieldComponent	121
122	Classe: FileDropzoneComponent	122
123	Classi: CommunicationGeneratorPanelComponent, GeneratedCommuni- cationPreviewComponent	123
124	Classi: CommunicationHistoryListComponent, PromptConfigurationList- Component, RecentFeedbackListComponent, GenerationProgressCompo- nent	124
125	Classi: DocumentListComponent, SubDocumentListComponent	126
126	Classi: DocumentUploadPanelComponent, DocumentStatusTimelineCom- ponent, UploadProgressComponent, FieldOriginComponent	126
127	Classe: SectionComponent	127
128	Classe: AssistantPage	130
129	Classe: CopilotPage	132

130	Classe: OverviewPage	134
131	Schema ER	150
132	Schema Relazionale	152
133	Schema ERD	153
134	Tabella communications	154
135	Tabella original_documents	156
136	Tabella sub_documents	157
137	Tabella extracted_data	158
138	Tabella workflow_tasks	159
139	Tabella prompt_configurations	159
140	Tabella audit_events	160

Indice delle Tabelle

2	Linguaggi di programmazione	10
3	Framework	10
4	Librerie	11
5	Strumenti per il contratto API	12
6	Strumenti per l'ambiente locale e il provisioning	13
7	Edge e web server	14
8	Servizi cloud AWS	15
9	Tecnologie per la persistenza dei dati	16
10	Strumenti per CI e quality gate	18
11	State API	19
12	Communications API	21
13	Documents API	22
14	Metriche strutturali dei due domini esagonali	26
15	Design pattern individuati nel backend	135
16	Tabella di tracciamento dei requisiti funzionali	167

Indice dei Listing

1	BedrockDocumentAiAdapter: adapter secondario sopra BedrockService . .	136
2	WorkflowTaskRunner: selezione della Strategy concreta e delega del passo di business	137
3	ProcessDocumentService: Facade su repository, storage, gateway AI, eventi ed heartbeat dietro process()	138
4	WorkflowTaskRegistry: registrazione e lookup dell'handler per tipo di task	140
5	AppServiceProvider: binding singleton dei client AWS e delle porte secondarie	141
6	Registrazione del listener in AppServiceProvider e reazione in RecordDocumentProcessingStarted	143
7	DeleteDocumentService: caso d'uso applicativo con una singola porta primaria (delete())	144
8	AssistantService: Facade su client HTTP generato, SseClient e MvpStateStore dietro generate()	146
9	AssistantPageViewModel: classe pura costruita con new(), niente inject() né riferimenti al DOM	148

1 Introduzione

1.1 Contenuto del documento

Il presente documento costituisce la Specifica Tecnica del sistema sviluppato dal gruppo *aLittleByte* per il proponente Eggon S.r.l. nell'ambito del progetto Nexum.

Al suo interno vengono illustrate le tecnologie selezionate, il contratto delle API REST esposte dal backend e le scelte architetturali adottate nella realizzazione del prodotto. Il documento ha lo scopo di fornire una visione organica e dettagliata della struttura del sistema, delle sue componenti e delle interazioni tra esse.

Deve essere inteso come riferimento tecnico per le attività di progettazione e codifica, garantendo la coerenza tra le decisioni implementative adottate e quanto emerso nella fase di Proof of Concept.

1.2 Scopo del prodotto

Nexum è la piattaforma sviluppata per Eggon S.r.l. a supporto degli operatori che gestiscono la documentazione del personale e le comunicazioni interne aziendali: i consulenti del lavoro da un lato, i redattori aziendali dall'altro.

L'AI Co-Pilot assiste i consulenti del lavoro nella gestione della documentazione HR: un file caricato viene automaticamente suddiviso, riconosciuto tramite OCR e associato al destinatario corretto, lasciando all'operatore solo la revisione dei casi a bassa confidenza invece della classificazione manuale di ogni documento.

L'AI Assistant Generativo assiste i redattori aziendali nella produzione di comunicazioni interne, dal titolo al testo fino all'immagine di copertina, a partire da un semplice prompt: riduce il tempo dedicato alla scrittura senza sottrarre all'utente il controllo editoriale, poiché ogni bozza generata resta modificabile e rigenerabile finché non viene scartata, e valutabile una sola volta nell'intero ciclo di vita della bozza.

1.3 Struttura del documento

La struttura del presente documento è articolata nelle seguenti sezioni principali:

- **Sezione 1 - Introduzione:** Presenta il contenuto e gli obiettivi del documento, il glossario dei termini e i riferimenti normativi e informativi adottati.
- **Sezione 2 - Tecnologie:** Descrive lo stack tecnologico adottato, suddiviso per categoria: linguaggi, framework, librerie, ambiente locale, edge, servizi cloud AWS, persistenza dei dati, osservabilità e CI/quality gate.
- **Sezione 3 - API:** Descrive il contratto REST esposto dal backend, organizzato per area funzionale (stato applicativo, AI Assistant Generativo, AI Co-Pilot).
- **Sezione 4 - Architettura del Sistema:** Descrive il contesto di sistema, l'architettura applicativa e logica interna del backend, il dettaglio dei componenti, la distribuzione, la containerizzazione e lo stile architetturale complessivo.

- **Sezione 5 - Descrizione architettura (Backend):** Scende al livello delle singole classi dei due domini di prodotto (Documents, Communications): adapter primari, entità di dominio, casi d'uso applicativi e adapter secondari.
- **Sezione 6 - Architettura Frontend:** Descrive i principi di progettazione della SPA Angular, il pattern MVVM adottato, i modelli di dati, i servizi di comunicazione col backend e i componenti UI, micro e macro.
- **Sezione 7 - Design Pattern:** Documenta i design pattern applicati esplicitamente nel backend e nel frontend, con l'intento di ciascuno, dove si trova nel codice e cosa succederebbe senza.
- **Sezione 8 - Database:** Descrive il modello dei dati del sistema: schema ER, schema relazionale e la descrizione di ciascuna tabella.
- **Sezione 9 - Requisiti Funzionali Soddisfatti:** Riporta lo stato di implementazione di ciascun requisito funzionale identificato nell'Analisi dei Requisiti, verificato sul codice sorgente del sistema.

1.4 Glossario

Al fine di evitare ambiguità e garantire una comprensione uniforme della terminologia utilizzata, i termini tecnici, gli acronimi e le parole con un significato specifico nel contesto del progetto sono contrassegnati nel testo con una “G” ad apice (es. parola^G). La loro definizione completa è consultabile nel documento esterno *Glossario*.

1.5 Riferimenti

1.5.1 Riferimenti normativi

- **Capitolato d'appalto C5 - NEXUM (Eggon S.r.l.):**
<https://www.math.unipd.it/~tullio/IS-1/2025/Progetto/C5.pdf>
Data Ultimo accesso: 04/09/2026
- **Norme di Progetto v2.0.0:**
 Documento interno del gruppo *aLittleByte* che definisce le regole, i ruoli e le procedure operative adottate nel progetto.
<https://alittlebyte-19.github.io/Documentazione/PB/Documenti%20interni/Norme%20di%20Progetto/Norme%20di%20Progetto.html>
Data Ultimo accesso: 05/09/2026
- **Regolamento del Progetto Didattico:**
 Anno accademico 2025/2026, Corso di Ingegneria del Software, Università di Padova.
<https://www.math.unipd.it/~tullio/IS-1/2025/Dispense/PD1.pdf>
Data Ultimo accesso: 06/09/2026

1.5.2 Riferimenti informativi

- **Glossario v3.0.0:**

Documento interno del gruppo *aLittleByte* contenente le definizioni dei termini tecnici e degli acronimi utilizzati nella documentazione.

<https://alittlebyte-19.github.io/Documentazione/glossario.html>

Data Ultimo accesso: 07/09/2026

- **Analisi dei Requisiti v3.0.0:**

Documento interno del gruppo *aLittleByte* contenente la descrizione dei requisiti funzionali e non funzionali del sistema.

<https://alittlebyte-19.github.io/Documentazione/PB/Documenti%20esterni/Analisi%20dei%20Requisiti/Analisi%20dei%20Requisiti.html>

Data Ultimo accesso: 08/09/2026

2 Tecnologie

Il presente capitolo illustra lo stack tecnologico adottato per la progettazione e la realizzazione del sistema Nexum. Le scelte implementative sono state guidate dai requisiti di vincolo definiti nell'Analisi dei Requisiti e dalle indicazioni del proponente Eggon S.r.l. Lo stack è organizzato attorno a cinque macro-aree principali: il runtime applicativo (backend Laravel su PHP e frontend Angular in TypeScript), l'infrastruttura cloud AWS per l'elaborazione intelligente dei documenti e la gestione asincrona dei job, la persistenza dei dati, lo stack di osservabilità e la pipeline di integrazione continua con quality gate automatici. L'ambiente locale riproduce la logica applicativa e l'orchestrazione asincrona di produzione attraverso la containerizzazione con Docker e l'emulazione dei servizi cloud con LocalStack, garantendo riproducibilità senza credenziali reali; fanno eccezione Bedrock e Textract, che parlano sempre con AWS reale, e l'infrastruttura di produzione vera e propria (RDS, ElastiCache, CloudFront), fuori dal perimetro dell'MVP.

2.1 Linguaggi di programmazione

Nome	Versione	Descrizione
PHP	8.4	Linguaggio di scripting server-side utilizzato per la realizzazione del backend tramite il framework Laravel. La versione 8.4 introduce miglioramenti significativi alle performance e nuove funzionalità del linguaggio.
TypeScript	5.9	Superset tipizzato di JavaScript sviluppato da Microsoft, impiegato per la realizzazione del frontend Angular. La tipizzazione statica riduce gli errori a tempo di compilazione e migliora la manutenibilità del codice.
SQL	—	Linguaggio dichiarativo per la definizione e la manipolazione dei dati nel database relazionale PostgreSQL, utilizzato per la gestione della persistenza strutturata del sistema.

Nome	Versione	Descrizione
------	----------	-------------

Tabella 2: Linguaggi di programmazione

2.2 Framework

Nome	Versione	Descrizione
Laravel	12	Framework PHP open source per lo sviluppo di applicazioni web server-side. Fornisce routing, un ORM (Eloquent) e strumenti integrati per la gestione dei job asincroni; il prodotto non ne adotta il tipico MVC applicativo, ma li usa come infrastruttura sotto un'architettura esagonale a porte e adapter (Sezione 4.3). È stato preferito a Symfony (di cui riusa diversi componenti sotto il cofano, come HttpFoundation e Routing) per le convenzioni integrate (ORM, code asincrone, CLI Artisan) che riducono la configurazione esplicita richiesta per un MVP con team ridotto, a fronte della maggiore modularità che Symfony offrirebbe.
Angular	21	Framework open source sviluppato da Google per la creazione di Single Page Application (SPA). Utilizza TypeScript e adotta un'architettura basata su componenti con supporto nativo per dependency injection, routing, reactive forms e signals. È stato preferito per la struttura integrata e opinione che garantisce coerenza e manutenibilità nel tempo.

Tabella 3: Framework

2.3 Librerie

Nome	Versione	Ambito	Descrizione
RxJS	7.8	Frontend	Libreria per la programmazione reattiva basata su Observable. Utilizzata per la gestione dello stato asincrono e degli stream di eventi all'interno della SPA Angular, in combinazione con i signals nativi di Angular.
@lucide/angular	1.28	Frontend	Libreria di icone open source integrata nel frontend Angular. Fornisce un set di icone SVG consistente e accessibile per l'interfaccia utente.

Nome	Versione	Ambito	Descrizione
AWS SDK for PHP	3.380	Backend	SDK ufficiale AWS per PHP. Utilizzata dal backend Laravel per invocare Bedrock, Texttract, S3, SQS, Step Functions, SSM Parameter Store e Secrets Manager tramite un'unica libreria client.
Flysystem S3 (league/flysystem-aws-s3-v3)	3.32	Backend	Adapter S3 per il filesystem astratto di Laravel. Consente di trattare i bucket S3 (o LocalStack) come un disco Laravel standard, cambiando solo la configurazione tra ambienti.
Opis JSON Schema	2.6	Backend	Libreria per la validazione di payload JSON contro uno schema. Utilizzata per validare la struttura dei dati restituiti dai modelli Bedrock, sia nella pipeline di Documents (split e estrazione campi) sia nella generazione testuale di Communications.
FPDI	2.6	Backend	Libreria PHP per l'importazione di pagine da documenti PDF esistenti. Utilizzata nella pipeline documentale per estrarre pagine specifiche da PDF massivi prima dello split.
FPDF	1.8	Backend	Libreria PHP per la generazione di documenti PDF. Utilizzata in combinazione con FPDI per produrre i sotto-documenti PDF per range di pagine a partire dai file originali.
Dompdf	3.1	Backend	Libreria PHP per il rendering di HTML/Blade in PDF. Genera il PDF del messaggio di invio (Co-Pilot) e l'esportazione della comunicazione (Assistant), entrambi con lo stesso timbro di piè di pagina condiviso.
qpdf	—	Backend	Binario di sistema per l'ispezione strutturale dei PDF (non una libreria Composer). Verifica in upload che il file non sia cifrato e che la struttura interna sia valida prima di accettarlo nella pipeline.

Tabella 4: Librerie

2.4 Contratto API

Il frontend non ha un client HTTP scritto a mano: il contratto OpenAPI 3.1.1 costituisce la fonte di verità condivisa da cui viene generato automaticamente il client TypeScript tipizzato, mantenendo frontend e backend allineati durante l'evoluzione del sistema.

Nome	Versione	Descrizione
OpenAPI	3.1.1	Specifica standard per la descrizione delle API REST. Costituisce il contratto condiviso tra frontend e backend e rappresenta la fonte di verità da cui vengono derivati client e documentazione.
Orval	8.23	Strumento di code generation che, a partire dalla specifica OpenAPI 3.1.1, genera automaticamente il client TypeScript tipizzato per il frontend Angular, eliminando la scrittura manuale delle chiamate HTTP.
Redocly CLI	latest	Strumento per la validazione e il linting del contratto OpenAPI, eseguito in pipeline sempre nell'ultima versione disponibile. Verifica la conformità della specifica agli standard e individua inconsistenze prima che vengano propagate al codice generato.

Tabella 5: Strumenti per il contratto API

2.5 Ambiente locale e provisioning

L'ambiente locale è progettato in modo che ogni membro del team lavori sulla stessa identica configurazione, vicina a quella di produzione, senza dover creare account cloud o installare servizi manualmente. I servizi AWS vengono emulati in locale tramite LocalStack, mentre l'infrastruttura è descritta come codice con Terraform, rendendola versionata e ricreabile.

Nome	Versione	Descrizione
Docker	—	Piattaforma di containerizzazione utilizzata per costruire le immagini dell'applicazione e del web server. Garantisce che l'ambiente di esecuzione sia identico per tutti i membri del team e coerente con la produzione.
Docker Compose	—	Strumento per l'orchestrazione dei container locali. Avvia l'intera infrastruttura (applicazione, database, servizi di supporto) su reti separate con un singolo comando.
Make	—	Automazione dei comandi ricorrenti tramite Makefile. Garantisce che le procedure di setup, avvio e build siano identiche per tutti i membri del team.
Composer	—	Gestore delle dipendenze PHP. Gestisce le librerie del backend e le relative versioni, garantendo riproducibilità delle build.
Node.js	22	Runtime JavaScript utilizzato dagli strumenti di build e sviluppo del frontend (Angular CLI, npm, Orval, ESLint, Jest). Eseguito in un container dedicato, non richiede un'installazione locale sulla macchina dello sviluppatore.

Nome	Versione	Descrizione
LocalStack	4.5	Emulatore locale dei servizi AWS. Riproduce in locale S3, SQS, Step Functions, SSM, Secrets Manager, KMS e IAM, consentendo di testare la pipeline asincrona senza credenziali reali. Bedrock e Textract restano eccezione deliberata: parlano sempre con AWS reale (Sezione 2.7).
Terraform	1.10.5	Strumento di Infrastructure as Code (IaC) per il provisioning dell'infrastruttura. Sostituisce gli script manuali con una descrizione dichiarativa e versionata, ricreabile a comando tramite terraform apply .
AWS SSM Parameter Store	—	Servizio AWS per la gestione della configurazione runtime non sensibile. I parametri vengono letti dall'applicazione all'avvio, mantenendo la configurazione fuori dal codice sorgente.
AWS Secrets Manager	—	Servizio AWS per la gestione dei segreti applicativi (credenziali, chiavi API). Mantiene i dati sensibili separati dal codice e dalla configurazione.

Tabella 6: Strumenti per l'ambiente locale e il provisioning

2.6 Edge e web server

Una richiesta HTTPS attraversa diversi livelli prima di raggiungere il codice applicativo. Traefik opera come entry point TLS esterno; in locale un secondo Nginx (*edge-cdn*) emula il ruolo di una CDN, servendo la SPA Angular direttamente da S3/LocalStack e inoltrando le richieste `/api`, `/health` e `/ready` all'Nginx applicativo, che a sua volta fa da proxy verso PHP-FPM. La stessa immagine di Nginx applicativo incorpora anche il bundle statico della SPA, compilato in una fase separata della propria build e servito con fallback su `index.html`. Un deploy di produzione reale, con una CDN gestita al posto del container *edge-cdn*, resta fuori dal perimetro dell'MVP.

Nome	Versione	Descrizione
Traefik	3.4	Reverse proxy posizionato davanti a tutti i servizi. Si occupa della terminazione TLS e dello smistamento delle richieste in base all'host.
Nginx (applicativo)	1.27	Web server che serve il bundle statico della SPA e fa da proxy per le chiamate <code>/api</code> verso PHP-FPM tramite Fast-CGI. Immagine di produzione, buildata e scansionata, che non conosce LocalStack.
Nginx (edge-cdn)	1.29	Secondo Nginx, presente solo in locale, che emula il ruolo di una CDN/edge davanti a S3-LocalStack per servire gli asset statici della SPA Angular.

Nome	Versione	Descrizione
PHP-FPM	8.4	Runtime PHP con processo FastCGI. Esegue il codice dell'applicazione Laravel. È stato preferito a FrankenPHP e Laravel Octane per un comportamento più prevedibile e una gestione dei processi più semplice in fase di PoC.

Tabella 7: Edge e web server

2.7 Servizi cloud AWS

Le funzionalità di intelligenza artificiale, l'elaborazione asincrona dei documenti e la protezione dei dati si basano su servizi gestiti AWS. L'adozione di servizi gestiti elimina la necessità di infrastrutture dedicate, garantendo scalabilità automatica e alta disponibilità.

In locale, S3, SQS, Step Functions, SSM, Secrets Manager, KMS e IAM sono emulati da LocalStack (Sezione 2.5). Bedrock e Textract restano invece un'eccezione deliberata: parlano sempre con AWS reale, anche in locale, perché LocalStack Community non offre un'emulazione dei modelli generativi utile ai fini del progetto, e condividono lo stesso insieme di credenziali AWS reali. Textract ha in più un interruttore dedicato (`TEXTTRACT_ENABLED`, default `false`): quando è disattivato il task di OCR termina come saltato senza persistere alcun testo, e il passo successivo della pipeline fallisce perché non trova il testo su cui lavorare, quindi il modulo Documents richiede comunque un Textract attivo per completare un'elaborazione. Bedrock non ha un interruttore equivalente nel codice.

Nome	Versione	Descrizione
Amazon Bedrock	—	Servizio AWS per l'accesso a modelli linguistici di grandi dimensioni (LLM). Di default utilizza Amazon Nova Lite per la generazione dei testi nell'AI Assistant Generativo, oltre che per la classificazione e lo split automatico dei documenti e per l'estrazione dei campi nell'AI Co-Pilot, e Stability SD3.5 Large per le immagini di copertina nell'AI Assistant Generativo.
Amazon Textract	—	Servizio AWS di riconoscimento ottico dei caratteri (OCR). Estrae testo e dati strutturati da documenti scansionati o immagini, leggendo i file direttamente da S3.
AWS Step Functions	—	Servizio di orchestrazione delle pipeline asincrone. Due state machine distinte coordinano rispettivamente la pipeline documentale (OCR, estrazione AI, persistenza risultati) e la pipeline delle comunicazioni (generazione testo/copertina), gestendo tentativi, timeout e stato tramite task token. È stato preferito a soluzioni self-hosted come Temporal o Camunda.

Nome	Versione	Descrizione
Amazon SQS	—	Servizio di code di messaggi gestite. Sono presenti due code separate, una per la pipeline documentale e una per quella delle comunicazioni, scelte per la loro durabilità e per la gestione automatica dei messaggi falliti. Un Worker dedicato per pipeline consuma i messaggi dalla propria coda e risponde all'orchestratore Step Functions con un token di callback.
Amazon SQS DLQ	—	Dead Letter Queue: una coda secondaria SQS per pipeline che raccoglie automaticamente i messaggi che hanno superato il numero massimo di tentativi (<code>maxReceiveCount</code>), consentendone l'ispezione tramite comandi diagnostici dedicati.
Amazon S3	—	Servizio di object storage. Un bucket dedicato archivia i documenti originali, i sotto-documenti generati dalla pipeline e le copertine delle comunicazioni, con oggetti cifrati at-rest tramite KMS; un secondo bucket ospita gli asset statici della SPA Angular, serviti dal container edge locale. L'accesso a entrambi è regolato da policy IAM.
AWS KMS	—	Servizio di gestione delle chiavi crittografiche. Utilizzato per la cifratura at-rest degli oggetti del bucket documentale, garantendo la protezione dei documenti sensibili.
AWS IAM	—	Servizio di gestione delle identità e degli accessi. Definisce i ruoli di esecuzione per Step Functions e le policy di accesso alle risorse, seguendo il principio del minimo privilegio.

Tabella 8: Servizi cloud AWS

2.8 Tecnologie per la persistenza dei dati

I dati del sistema hanno nature diverse e vengono gestiti con strumenti dedicati: PostgreSQL per la persistenza strutturata e transazionale, Redis per lo stato volatile ad accesso rapido.

Nome	Versione	Descrizione
PostgreSQL	16	Sistema di gestione di database relazionali open source. Conserva i dati strutturati dell'applicazione (documenti, estratti, audit, stato dei task) e, grazie al supporto JSONB, gestisce anche payload semi-strutturati senza cambiare database. È stato preferito a MySQL e a soluzioni documentali come MongoDB per affidabilità e flessibilità. Nel repository gira come container (postgres:16-alpine); un servizio gestito in cloud (RDS) è ugualmente fuori dal perimetro dell'MVP.
Redis	7	Data store in-memory open source. Gestisce la cache applicativa e il rate limiting, sempre con scadenza. Non viene utilizzato come coda della pipeline, ruolo affidato ad Amazon SQS. Nel repository gira come container (redis:7-alpine); un servizio gestito in cloud (ElastiCache) è ugualmente fuori dal perimetro dell'MVP.

Tabella 9: Tecnologie per la persistenza dei dati

2.9 Osservabilità

Il sistema affida l'osservabilità operativa a tre meccanismi applicativi, senza uno stack di monitoraggio dedicato né un collector:

- **Audit trail di compliance:** `AuditLogger` (Sezione 5.3) registra ogni evento rilevante (upload, elaborazione, revisione, generazione, valutazione...) sulla tabella `audit_events`, append-only (Sezione 8). Non è un sistema di monitoraggio operativo ma un registro di conformità: risponde a "chi ha fatto cosa, quando", non a "il sistema sta bene adesso?".
- **Correlation/request ID:** il middleware `CorrelateRequests` lato backend, con l'equivalente lato frontend nella cartella `core` (Sezione 6.1), taggano ogni richiesta HTTP e ogni messaggio SQS con un identificativo di correlazione, propagato nei log strutturati per ricostruire il percorso di una richiesta attraverso i tre processi backend (API, due Worker).
- **Metriche operative verso CloudWatch:** `EmfMetricsRecorder` scrive righe in CloudWatch Embedded Metric Format (EMF) su un canale di log dedicato (**metrics**); CloudWatch le interpreta automaticamente come dato di metrica quando la riga arriva da CloudWatch Logs, senza endpoint di scrape né SDK applicativo verso `PutMetricData`. Il listener `RecordRequestMetrics` lo invoca a ogni richiesta HTTP (evento nativo `RequestHandled`, non un middleware: un'eccezione che risale la pipeline farebbe comunque terminare la richiesta prima che il codice dopo `$next()` di un middleware venga eseguito) per `RequestCount`, `Latency` ed `Errors`; il comando `php artisan mvp:metrics:dlq-depth` lo invoca per `DlqDepth`, una per pipeline.

I log strutturati sul canale `stderr` sono già in formato JSON con i campi di correlazione iniettati da `CorrelateRequests`, senza chiamate SDK applicative dedicate. Il loro

instradamento verso un servizio gestito di aggregazione dei log resta fuori dal perimetro dell'MVP, che non ha un ambiente AWS reale collegato. La diagnosi di un incidente passa da questi log e da due comandi diagnostici manuali: `php artisan mvp:dlq:list`, che interroga direttamente SQS per ispezionare il contenuto di una DLQ, e il già citato `mvp:metrics:dlq-depth` per la sola profondità.

Le metriche mostrate agli utenti nei pannelli Angular (Assistant, Co-Pilot, Overview; Sezione 6.2, 6.6) sono un sistema distinto dai precedenti: `MvpStateService` le calcola sull'intero tenant a ogni `GET /api/v1/state`, e la SPA le legge da lì.

2.10 CI e quality gate

Ogni modifica inviata al repository attiva la pipeline di integrazione continua su GitHub Actions, che ricostruisce le immagini e le pubblica su GHCR. La pubblicazione avviene nel job `stack`, che è indipendente dagli esiti dei job `backend` e `frontend`: è subordinata ai soli controlli interni a quel job (validazione Terraform, scansione delle immagini, verifica HTTPS e accessibilità), non all'esito di stile, analisi statica e test. I quality gate coprono stile del codice, analisi statica, test, sicurezza delle dipendenze e accessibilità.

Nome	Versione	Categoria	Descrizione
GitHub Actions	—	CI/CD	Orchestratore della pipeline CI. Attivato da push e pull request, esegue quattro job: <code>backend</code> , <code>frontend</code> e <code>stack</code> in parallelo, mentre <code>coverage-diff</code> attende il completamento dei primi due.
GHCR	—	CI/CD	GitHub Container Registry: registro in cui vengono pubblicate le immagini Docker solo dopo il superamento di tutti i quality gate.
Docker Hub	—	CI/CD	Utilizzato esclusivamente per il pull autenticato delle immagini base, al fine di evitare i limiti di rate anonimo. Non è il registro di pubblicazione delle immagini prodotte.
Trivy	0.66.0	Sicurezza	Scanner di vulnerabilità per immagini Docker. Analizza le immagini costruite (vulnerabilità, secret, configurazione) prima della pubblicazione su GHCR.
npm audit	—	Sicurezza	Analisi delle vulnerabilità nelle dipendenze npm di produzione del frontend Angular.
Pint	1.29	Qualità backend	Formattatore e linter per il codice PHP, basato sulle convenzioni di Laravel.

Nome	Versione	Categoria	Descrizione
PHPStan 2.2 / Larastan 3.10	—	Qualità backend	Strumento di analisi statica per PHP (PHPStan) con estensione specifica per Laravel (Larastan). Individua errori di tipo e problemi di logica senza eseguire il codice.
Pest	4.7	Qualità backend	Framework di testing per PHP utilizzato per i test unitari e di integrazione del backend Laravel.
ESLint	9.39	Qualità frontend	Linter per il codice TypeScript della SPA Angular. Verifica la conformità alle regole di stile e individua potenziali errori.
Jest	30.4	Qualità frontend	Framework di testing JavaScript utilizzato per i test unitari del frontend Angular, in combinazione con TypeScript.
Redocly + Orval diff	Orval 8.23 / Redocly latest	Contratto API	Redocly (sempre ultima versione) valida il contratto OpenAPI e Orval rigenera il client tipizzato; un diff verifica che il client generato sia allineato con il contratto aggiornato.
terraform validate	1.10.5	IaC	Valida formato e correttezza sintattica/semantica della configurazione Terraform prima di qualsiasi applicazione.
axe	4.13	Accessibilità	Strumento di testing automatico per l'accessibilità (a11y) dell'interfaccia utente, integrato nella pipeline CI.
pally	9.1	Accessibilità	Strumento complementare ad axe per il testing dell'accessibilità. Esegue test a11y sulle pagine dell'applicazione.
Playwright	1.60	Accessibilità	Framework per l'automazione di un browser reale (Chromium). Utilizzato per eseguire gli audit di accessibilità con axe e Pally e per verificare che la SPA funzioni correttamente con la Content Security Policy applicata.

Tabella 10: Strumenti per CI e quality gate

3 API

Il sistema espone un contratto REST versionato (`/api/v1`), definito tramite la specifica OpenAPI 3.1.1 in `openapi/v1/alittlebyte-mvp-api.yaml` e consumato dalla SPA An-

gular tramite il client generato con Orval. L'identità (**MvpIdentity**: attore e tenant) è risolta in due modalità alternative, selezionate da **MVP_IDENTITY_MODE**: in modalità *local* (default, usata in sviluppo e nella MVP) l'identità è iniettata da configurazione e nessun header è richiesto; in modalità *trusted_headers* la richiesta deve portare **X-Mvp-User-Id** insieme a **X-Mvp-Tenant-Id** e agli altri header fidati, iniettati dal confine aziendale a monte dell'applicazione.

Il rate limiting è granulare per gruppo di endpoint. Il limite di default è 60 richieste al minuto; scende a 20/minuto per le operazioni di generazione e scrittura più onerose (creazione bozze, rigenerazione, upload OCR, aggiornamento copertina, salvataggio configurazioni di prompt, salvataggio e scarto della bozza) e a 30/minuto per anteprima, export e valutazione delle comunicazioni e per l'esportazione del report metriche dell'Assistant (le rotte equivalenti sui documenti restano al limite di default). Sale invece a 120/minuto per la sola lettura della copertina già generata. Gli endpoint che restituiscono un PDF (anteprima ed export delle comunicazioni) supportano la cache HTTP tramite **ETag/If-None-Match**, evitando di rigenerare il documento se il client possiede già l'ultima versione.

L'avanzamento delle elaborazioni asincrone (generazione contenuti, pipeline documentale) è notificato alla SPA tramite due endpoint Server-Sent Events (SSE), uno per dominio, senza dover interrogare ripetutamente il server. Le liste di comunicazioni e documenti supportano filtri lato server e paginazione.

Le API sono organizzate in tre aree funzionali, corrispondenti ai tag della specifica OpenAPI: lo stato applicativo, l'AI Assistant Generativo (comunicazioni) e l'AI Co-Pilot (documenti).

3.1 State API

Espone lo stato corrente dei due moduli applicativi, utilizzato dalla SPA per popolare le viste principali senza richieste multiple.

Endpoint	Metodo	Descrizione
/api/v1/state	GET	Restituisce in un'unica risposta le metriche e lo storico delle comunicazioni dell'AI Assistant Generativo, insieme alle metriche e ai sotto-documenti dell'AI Co-Pilot. Accetta quattro parametri di query opzionali (tone , style , dateFrom , dateTo) che filtrano solo la sezione Metriche dell'AI Assistant, senza incidere sui numeri del Co-Pilot.

Tabella 11: State API

3.2 Communications API

Gestione del ciclo di vita completo di una comunicazione generata dall'AI Assistant Generativo: richiesta della bozza, revisione e modifica, rigenerazione, salvataggio o scarto,

valutazione, esportazione e gestione della copertina. Include inoltre la gestione delle configurazioni di prompt salvate per il riuso.

Endpoint	Metodo	Descrizione
/api/v1/communications	GET	Lista delle bozze approvate del tenant (status = approved), con filtri opzionali (parola chiave nel prompt, tono, stile, data) e paginazione; le bozze non ancora salvate e quelle scartate non compaiono.
/api/v1/communications	POST	Richiede una bozza di comunicazione a partire da prompt, tono e stile; avvia in modo asincrono la pipeline delle comunicazioni e restituisce l'identificativo della comunicazione e l'URL di streaming.
/api/v1/communications/{communication}	PUT	Aggiorna titolo e corpo di una bozza di comunicazione.
/api/v1/communications/{communication}	DELETE	Elimina definitivamente una comunicazione dallo storico.
/api/v1/communications/{communication}/regenerate	POST	Rigenera la bozza di comunicazione riutilizzando prompt, tono e stile già persistiti; la richiesta non accetta un body.
/api/v1/communications/{communication}/save	POST	Salva la bozza nello storico, lasciandola comunque modificabile e rigenerabile.
/api/v1/communications/{communication}/discard	POST	Scarta la bozza di comunicazione.
/api/v1/communications/{communication}/favorite	POST	Aggiunge la comunicazione ai preferiti.
/api/v1/communications/{communication}/favorite	DELETE	Rimuove la comunicazione dai preferiti.
/api/v1/communications/{communication}/rating	POST	Assegna una valutazione da 1 a 5 stelle alla bozza; una volta assegnata, per l'intera vita della bozza (comprese eventuali rigenerazioni) non può più essere modificata: ogni tentativo successivo viene rifiutato.
/api/v1/communications/{communication}/stream	GET	Endpoint SSE che notifica alla SPA l'avanzamento della generazione (testo e copertina) per una comunicazione specifica.
/api/v1/communications/{communication}/cover-image	GET	Download dell'immagine di copertina generata per la comunicazione (PNG, JPEG o WebP).

Endpoint	Metodo	Descrizione
/api/v1/communications/{communication}/cover-image	POST	Sostituisce manualmente l'immagine di copertina generata dall'AI, caricando un file.
/api/v1/communications/{communication}/cover-image	DELETE	Rimuove l'immagine di copertina associata alla comunicazione.
/api/v1/communications/{communication}/preview	GET	Anteprima del PDF impaginato della comunicazione, con supporto ETag per evitare rigenerazioni non necessarie.
/api/v1/communications/{communication}/export	GET	Download del PDF finale della comunicazione come allegato, con lo stesso supporto ETag dell'anteprima.
/api/v1/prompt-configurations	POST	Salva la configurazione corrente di prompt (tono, stile, testo) per un riutilizzo successivo.
/api/v1/prompt-configurations/{promptConfiguration}	DELETE	Elimina definitivamente una configurazione di prompt salvata.
/api/v1/assistant/metrics/export	GET	Download del report riepilogativo delle metriche dell'AI Assistant come PDF allegato.

Tabella 12: Communications API

3.3 Documents API

Gestione del ciclo di vita dei documenti caricati dall'AI Co-Pilot, della relativa pipeline di elaborazione asincrona (OCR, estrazione, revisione) e della composizione del messaggio di invio verso il destinatario individuato.

Endpoint	Metodo	Descrizione
/api/v1/documents	GET	Lista dei sotto-documenti del tenant, con filtri opzionali (ricerca su nome/cognome/azienda, stato di invio, soglia e criterio di confidenza, mese/anno) e paginazione.
/api/v1/documents/ocr	POST	Carica un documento e avvia in modo asincrono la pipeline documentale (OCR tramite Textract se abilitato, altrimenti un passo vuoto; estrazione e classificazione tramite Bedrock); restituisce l'URL di streaming per seguirne l'avanzamento.
/api/v1/documents/{originalDocument}/stream	GET	Endpoint SSE che notifica alla SPA l'avanzamento dell'elaborazione per un documento originale specifico.

Endpoint	Metodo	Descrizione
/api/v1/documents/{subDocument}	DELETE	Elimina un sotto-documento già elaborato dalla pipeline.
/api/v1/documents/{subDocument}/extracted-data	PUT	Corregge i dati estratti automaticamente per un sotto-documento (nominativo, azienda, email destinatario, codice fiscale, id dipendente, tipo documento, data, descrizione, punteggio di confidenza), con un flag opzionale per marcarlo contestualmente come validato manualmente.
/api/v1/documents/{subDocument}/review	POST	Segna un sotto-documento come validato manualmente, ad esempio a valle di una revisione human-in-the-loop per i casi a bassa confidenza.
/api/v1/documents/{subDocument}/preview	GET	Restituisce l'anteprima in formato PDF del sotto-documento.
/api/v1/documents/{subDocument}/original-preview	GET	Restituisce l'anteprima in formato PDF del documento originale completo, non splittato, alternativa indipendente quando quella del sotto-documento non è disponibile.
/api/v1/documents/{subDocument}/send-preview	GET	Anteprima PDF del messaggio di invio precompilato per il sotto-documento.
/api/v1/documents/{subDocument}/send-export	GET	Download del messaggio di invio precompilato come PDF allegato.
/api/v1/documents/{subDocument}/send-message	PUT	Corregge i campi del messaggio di invio precompilato per il sotto-documento.

Tabella 13: Documents API

4 Architettura del Sistema

Questo capitolo descrive l'architettura del sistema, dal generale al particolare: contesto, architettura applicativa, architettura logica interna del backend, dettaglio dei componenti, distribuzione, containerizzazione e stile architetturale complessivo.

4.1 Contesto di Sistema

L'utente finale interagisce con la piattaforma tramite browser, via HTTPS. Il sistema delega l'elaborazione intelligente dei contenuti a due servizi AWS esterni: l'AI Co-Pilot invia i documenti caricati ad AWS Textract per il riconoscimento ottico dei caratteri e poi ad AWS Bedrock per lo split e l'estrazione dei campi; l'AI Assistant Generativo invia i prompt ad AWS Bedrock per la generazione di testi e immagini. Le integrazioni con AWS avvengono esclusivamente lato backend; il browser comunica solo con il sistema.

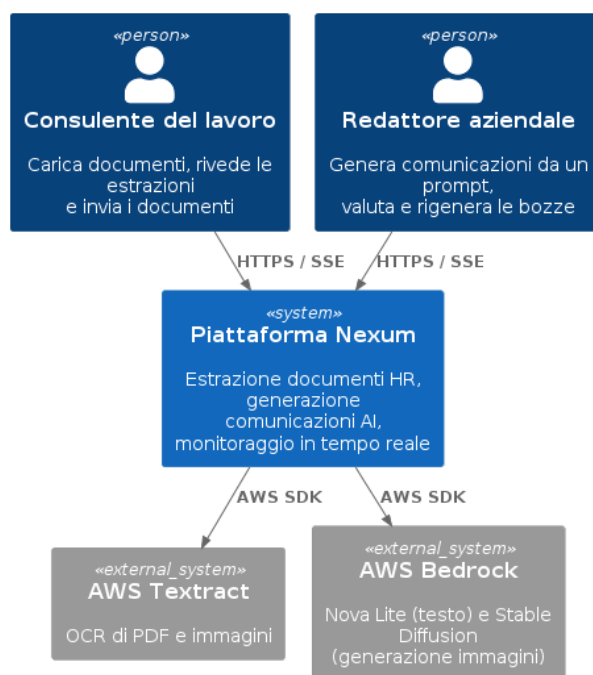


Figura 1: System Context Diagram

4.2 Architettura Applicativa

Il sistema si compone dei seguenti container principali. Il frontend è una Single Page Application in Angular 21. Il backend è un'API in Laravel 12 su PHP 8.4, servita da PHP-FPM e Nginx, che espone un contratto REST le cui risposte sono in formato JSON, con l'eccezione degli endpoint che restituiscono un documento PDF, l'immagine di copertina o un flusso di eventi. Non serve pagine HTML: le uniche viste renderizzate lato server sono i template usati per comporre i PDF. Le operazioni pesanti (OCR e invocazione dei modelli generativi) sono delegate in modo asincrono ad AWS Step Functions e Amazon SQS; due processi Worker separati dall'API, uno per pipeline, consumano i messaggi e coordinano l'esecuzione: la coda documentale è servita da un container dedicato, quella delle comunicazioni da un secondo container, così i due flussi non si contendono gli stessi consumatori. La persistenza è affidata a PostgreSQL per i dati applicativi e a Redis per cache e rate limiting; nessuno dei due è utilizzato come coda, ruolo riservato a SQS.

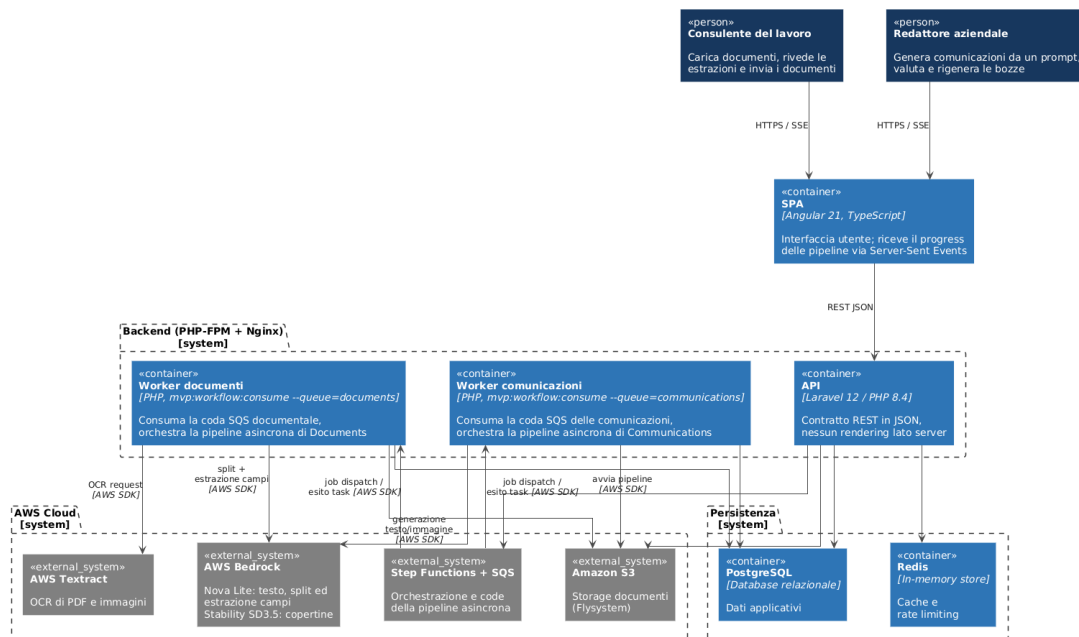


Figura 2: Container Diagram

4.3 Architettura Logica Interna

Il frontend è dedicato all'interfaccia utente. Il backend è organizzato per dominio applicativo secondo i principi dell'architettura esagonale.

4.3.1 Frontend: Interfaccia Utente

Sviluppato in Angular come Single Page Application, gestisce l'interfaccia e l'interazione con l'utente finale.

- Gestisce gli input dell'utente (caricamento documenti, richieste di generazione, revisione dei dati estratti).
- Comunica con il backend tramite chiamate API RESTful, generate a partire dal contratto OpenAPI condiviso.
- Riceve l'avanzamento delle elaborazioni asincrone tramite Server-Sent Events, senza interrogare ripetutamente il server.

4.3.2 Backend: Architettura Esagonale

Il backend adotta un'architettura esagonale, applicata ai due domini di prodotto: **Documents** (AI Co-Pilot) e **Communications** (AI Assistant Generativo). La logica di business è disaccoppiata dai dettagli tecnici (framework, database, SDK esterni) per garantire manutenibilità, testabilità e sostituibilità dei singoli componenti. Il backend si articola in tre parti.

4.3.2.1 Adapter primari

Ricevono uno stimolo esterno e lo traducono in una chiamata a un caso d'uso, senza contenere regole di business. Nella forma più comune sono i Controller HTTP. Il gestore dei task di workflow è un adapter primario equivalente, guidato da un messaggio SQS o da una callback di Step Functions invece che da una richiesta HTTP.

4.3.2.2 Dominio e applicazione

Il dominio contiene le regole di business e le porte che definiscono le capacità del sistema, senza dipendenze da Eloquent, da un client AWS o da HTTP. Le tre aggregazioni principali (**SubDocument**, **OriginalDocument**, **Communication**) sono entità di dominio a sé stanti, distinte dai model Eloquent: incapsulano le proprie transizioni di stato e le relative regole (es. una copertina non può essere generata prima del testo) invece di lasciarle implicite nei singoli servizi applicativi. L'applicazione implementa le porte primarie orchestrando dominio ed entità esclusivamente attraverso le porte secondarie.

4.3.2.3 Adapter secondari

Implementano la comunicazione con i sistemi esterni: il database PostgreSQL tramite repository Eloquent, i modelli AI tramite gli adapter di Bedrock e Textract, lo storage documentale tramite Flysystem e S3.

4.3.2.4 Esempio: il dominio Documents

Un Controller riceve l'upload, lo valida e delega l'esecuzione al caso d'uso corrispondente, che salva il file e avvia la pipeline asincrona senza conoscere i dettagli di Step Functions. Il Worker raggiunge lo stesso dominio applicativo tramite un adapter primario diverso, guidato dal workflow, e delega ai casi d'uso di OCR, estrazione dei campi e finalizzazione della pipeline. La revisione umana delle estrazioni a bassa confidenza avviene invece in modo sincrono, tramite un Controller HTTP dedicato invocato dall'operatore, non dal Worker. Il risultato è persistito tramite il repository del dominio; un evento dedicato notifica l'audit. Il dominio Communications segue lo stesso schema.

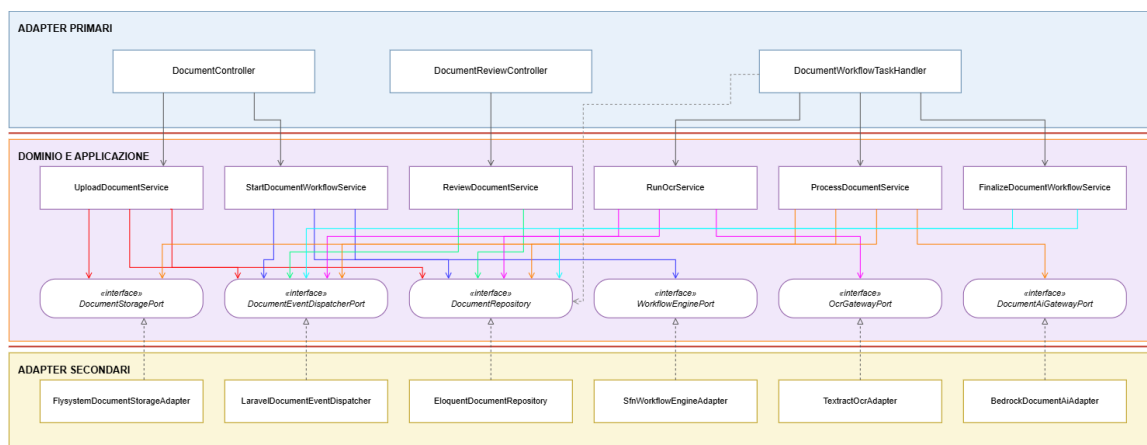


Figura 3: Architettura Esagonale: Esempio Dominio Documents

4.4 Dettaglio dei Componenti

La tabella seguente quantifica la composizione interna dei due domini: casi d'uso, porte verso l'esterno, eventi di dominio e adapter primari.

Dominio	Casi d'uso	Porte secondarie	Eventi	Adapter primario
Documents	12	6	14	4 Controller HTTP + WorkflowTaskHandler
Communicatio	14	6	21	6 Controller HTTP + WorkflowTaskHandler

Tabella 14: Metriche strutturali dei due domini esagonali

I due domini condividono un piccolo numero di porte secondarie puramente infrastrutturali, comuni a entrambi perché non riguardano la logica di business: l'avvio di un'esecuzione Step Functions, la generazione di identificativi univoci e i confini transazionali sulle scritture pure sul database. A queste si aggiunge la segnalazione di attività verso Step Functions durante le chiamate AI più lunghe, dichiarata anch'essa nell'infrastruttura condivisa ma oggi usata dal solo dominio Documents. Ciascuna ha un solo adapter condiviso.

La separazione fra i due domini è verificata automaticamente a ogni build della pipeline CI: uno script analizza il codice e blocca la build se un dominio referencia direttamente il framework, l'SDK AWS o l'altro dominio, così i due moduli restano davvero indipendenti.

4.4.1 Modulo Documents

Copre l'intera pipeline dell'AI Co-Pilot: dall'ingresso del documento caricato dall'utente, attraverso l'orchestrazione asincrona di riconoscimento ottico, classificazione ed estrazione dei campi, fino alla consultazione, alla revisione umana dei dati estratti e alla composizione del documento precompilato per il destinatario, che l'operatore scarica ed invia fuori piattaforma. Le porte secondarie isolano il dominio dalla persistenza, dai gateway AI di riconoscimento ottico e classificazione, dallo storage documentale, dal rendering del documento di invio e dalla pubblicazione degli eventi di dominio.

Il diagramma seguente si concentra sulla pipeline centrale (upload, orchestrazione asincrona, revisione); i controller e i casi d'uso di sola consultazione (`DocumentPreviewController`, `SendMessageController` e i rispettivi casi d'uso di lista, cancellazione e polling) non compaiono qui: il conteggio completo dei 4 Controller HTTP e dei 12 casi d'uso è nella Tabella 14 della Sezione 4.4, il dettaglio di ciascuno nella Sezione 5.1.

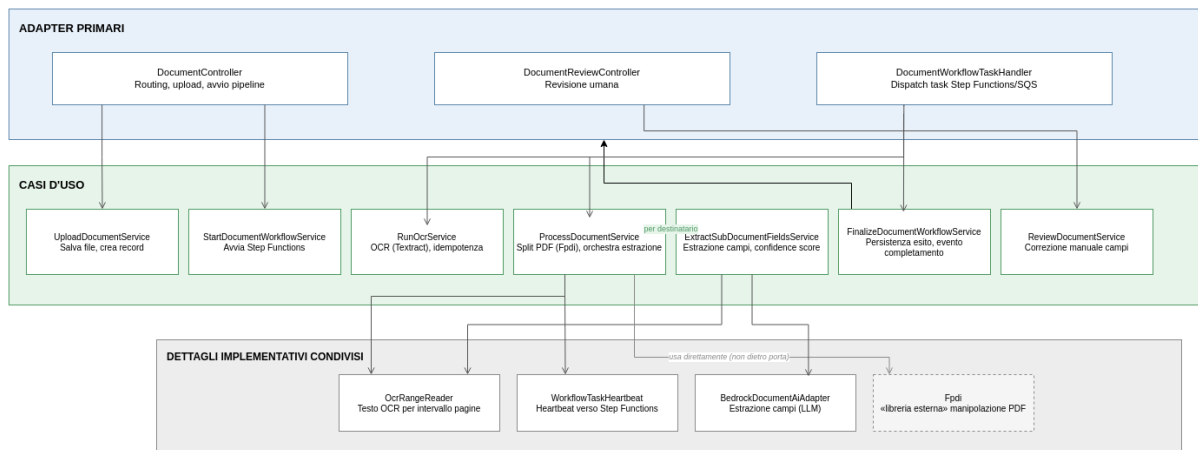


Figura 4: Component Diagram: Pipeline centrale del modulo Documents

4.4.2 Modulo Communications

Copre l'intero ciclo di vita di una bozza generata dall'AI Assistant Generativo: dalla richiesta di generazione (testo e copertina), attraverso l'orchestrazione asincrona della pipeline, fino alla gestione editoriale della bozza (preferiti, modifica, approvazione, scarto), alla sua consultazione, esportazione e valutazione, e alla gestione delle configurazioni di prompt riutilizzabili. Le porte secondarie isolano il dominio dalla persistenza, dal gateway AI generativo, dallo storage delle copertine, dal rendering PDF e dalla pubblicazione degli eventi di dominio.

Il diagramma seguente si concentra sulla pipeline centrale (generazione, orchestrazione asincrona, valutazione); i controller e i casi d'uso di gestione editoriale, consultazione e configurazione (`CommunicationCoverController`, `CommunicationExportController`, `CommunicationStreamController`, `PromptConfigurationController` e i rispettivi casi d'uso) non compaiono qui: il conteggio completo dei 6 Controller HTTP e dei 14 casi d'uso è nella Tabella 14 della Sezione 4.4, il dettaglio di ciascuno nella Sezione 5.2.

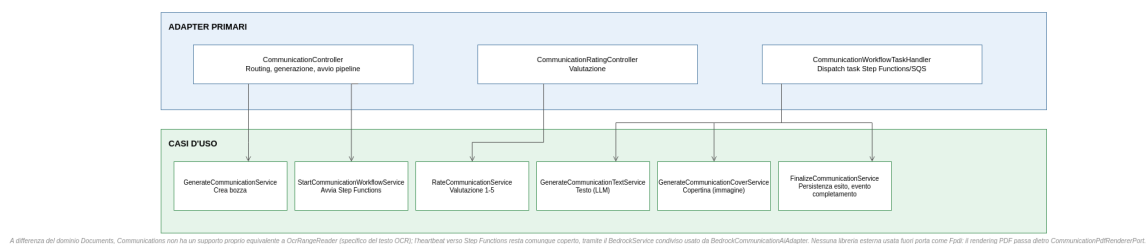


Figura 5: Component Diagram: Pipeline centrale del modulo Communications

4.5 Architettura di Deployment

Il backend e la SPA frontend sono distribuiti come container separati, raggiunti da un unico entry point TLS (la catena completa di proxy è descritta in Sezione 2.6). Il backend gira su tre processi che condividono lo stesso nucleo applicativo: l'API, raggiunta via HTTP, e due Worker, uno per la coda documentale e uno per quella delle comunicazioni,

che eseguono lo stesso dominio tramite i messaggi SQS e le callback di Step Functions: tre adapter primari sullo stesso sistema, non sistemi separati.

In locale l'intera topologia, orchestrazione Step Functions/SQS inclusa, è riprodotta con Docker Compose, LocalStack e Terraform: stessa API applicativa, stessi due Worker, stessa orchestrazione Step Functions/SQS emulata. Bedrock e Textract restano un'eccezione deliberata (Sezione 2.7), e un deploy di produzione reale (RDS, ElastiCache, CloudFront) resta fuori dal perimetro dell'MVP (Sezioni 2.6, 2.8).

4.6 Containerizzazione con Docker

L'intero stack è containerizzato; i dettagli di ciascun servizio sono descritti nella Sezione 2. I container principali sono:

- **Frontend (SPA):** esegue la build statica Angular, servita dall'edge; nessuna logica di business.
- **Backend (API):** esegue PHP-FPM dietro Nginx, gestisce le richieste HTTP verso il dominio applicativo.
- **Worker documenti e Worker comunicazioni:** due container separati che eseguono lo stesso dominio applicativo del backend, in ascolto rispettivamente sulla coda SQS documentale e su quella delle comunicazioni, anziché su HTTP.
- **Database (PostgreSQL) e Cache (Redis):** persistenza applicativa e stato volatile, su volumi Docker dedicati.
- **Emulazione AWS (LocalStack) e provisioning (Terraform):** riproducono in locale, senza credenziali reali, i servizi AWS emulabili (Sezione 2.7); Bedrock e Textract restano un'eccezione e parlano sempre con AWS reale.

Il pool PHP-FPM del container Backend è dimensionato esplicitamente (`docker/php/www-pool.conf`), invece di restare sul default dell'immagine base (`pm.max_children=5`). Il rischio non era il timeout di Traefik o di Nginx verso lo stream SSE (l'heartbeat inviato ogni secondo lo tiene vivo), ma il pool PHP-FPM stesso: ogni connessione SSE occupa un worker per l'intera durata dello stream (fino a 1800 secondi per i documenti), quindi pochi upload concorrenti avrebbero esaurito l'intero pool, bloccando anche l'endpoint `/health` per tutte le richieste in arrivo.

4.7 Stile architetturale

Lo stile architetturale del backend è un monolite modulare organizzato per dominio: un nucleo esagonale limitato ai due domini di prodotto, circondato da un'infrastruttura condivisa che non vi è assoggettata. Tre caratteristiche lo definiscono:

- **Testabilità del dominio senza framework:** i casi d'uso di Documents e Communications sono istanziabili con `new`, senza container, database o LocalStack; la suite di test dedicata usa implementazioni in memoria delle porte secondarie al posto di quelle reali.

- **Orchestrazione asincrona esternalizzata:** le operazioni onerose non transitano dal sistema di code nativo di Laravel, ma da Step Functions e SQS; l'adapter che le riceve è trattato allo stesso livello di un Controller HTTP.
- **Conformità verificata automaticamente:** la Dependency Rule è un controllo eseguito a ogni build, non una convenzione documentata.

Il perimetro dell'esagonale resta limitato per scelta esplicita: Identity, Audit, Observability, Support e l'infrastruttura condivisa di Workflow restano organizzati per servizio, perché nessuno di essi ha oggi, né è previsto abbia nell'arco della MVP, un'implementazione alternativa da rendere sostituibile.

5 Descrizione architettura (Backend)

Questo capitolo scende al livello delle singole classi che compongono il nucleo dei due domini di prodotto, completando la vista d'insieme del capitolo precedente. Per ciascun modulo sono descritti l'adapter primario, le entità di dominio, tutti i casi d'uso applicativi e gli adapter secondari, seguendo lo stesso ordine in cui intervengono nella pipeline descritta dal diagramma di sequenza; la categorizzazione dei casi d'uso per responsabilità resta quella della Sezione 4.4.

5.1 Modulo Documents (AI Co-Pilot)

Il modulo Documents è la componente del sistema dedicata alla gestione e all'elaborazione intelligente della documentazione HR caricata dai consulenti del lavoro. Il suo obiettivo è ridurre il lavoro manuale di classificazione e smistamento dei documenti: un file caricato viene automaticamente suddiviso, riconosciuto tramite OCR e associato al destinatario corretto, lasciando all'operatore solo la revisione dei casi a bassa confidenza. Il modulo si integra con Amazon Textract per il riconoscimento ottico dei caratteri e con Amazon Bedrock per la classificazione e l'estrazione dei dati, orchestrando l'intera pipeline in modo asincrono tramite AWS Step Functions.



5.1.1 Diagrammi delle classi parziali del modulo Documents

I diagrammi seguenti mostrano porzioni del diagramma delle classi complessivo, raggruppate per flusso applicativo, utili a leggere le dipendenze reali senza dover tenere sott'occhio l'intero grafo.

Il primo diagramma mostra `DocumentController`, uno dei quattro adapter primari HTTP del modulo: riceve le richieste di caricamento, elenco, avanzamento e cancellazione dei documenti e le smista verso i casi d'uso corrispondenti. Revisione, anteprima e invio del messaggio precompilato hanno controller HTTP dedicati (`DocumentReviewController`, `DocumentPreviewController`, `SendMessageController`).

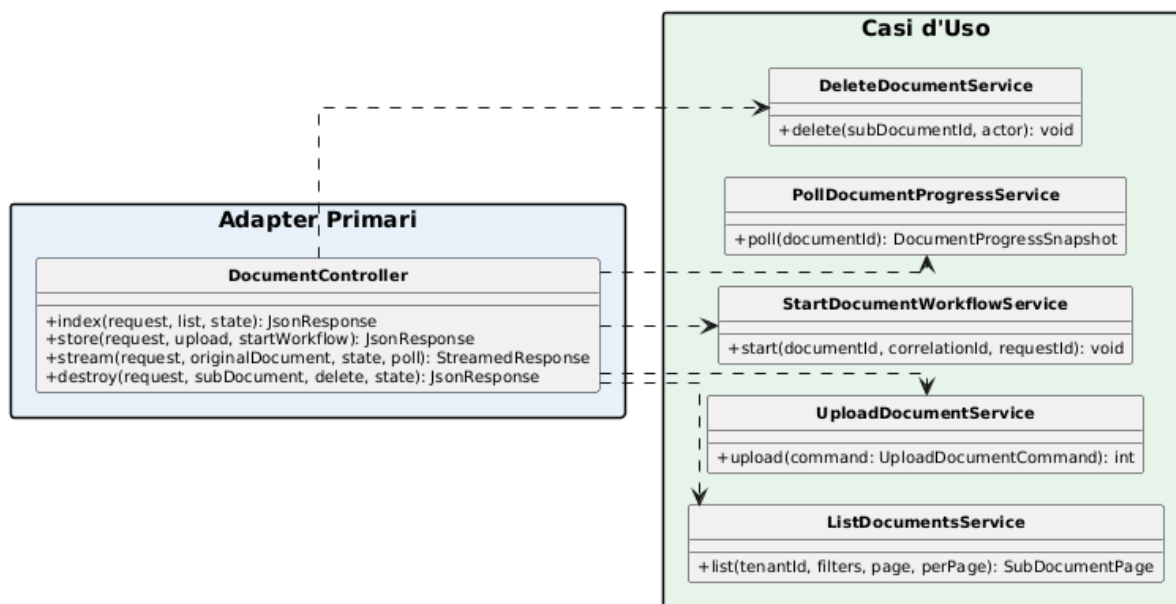


Figura 7: Diagramma classi parziale: DocumentController

Il secondo diagramma rappresenta il flusso di upload e avvio della pipeline: `UploadDocumentService` salva il file e crea il documento originale, mentre `StartDocumentWorkflowService` avvia l'esecuzione Step Functions tramite `SfnWorkflowEngineAdapter` (componente condiviso del modulo Workflow, non specifico di Documents).

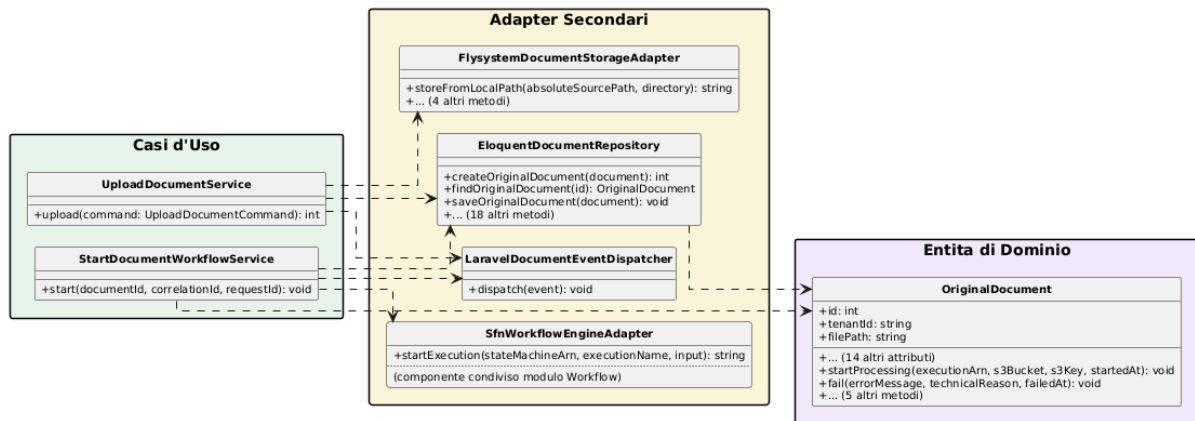


Figura 8: Diagramma classi parziale: Flusso di upload e avvio

Il terzo diagramma evidenzia la catena di elaborazione asincrona orchestrata da `DocumentWorkflowTaskHandler`: riconoscimento OCR, split e classificazione del documento, estrazione dei campi per ogni destinatario ed aggiornamento finale dello stato.

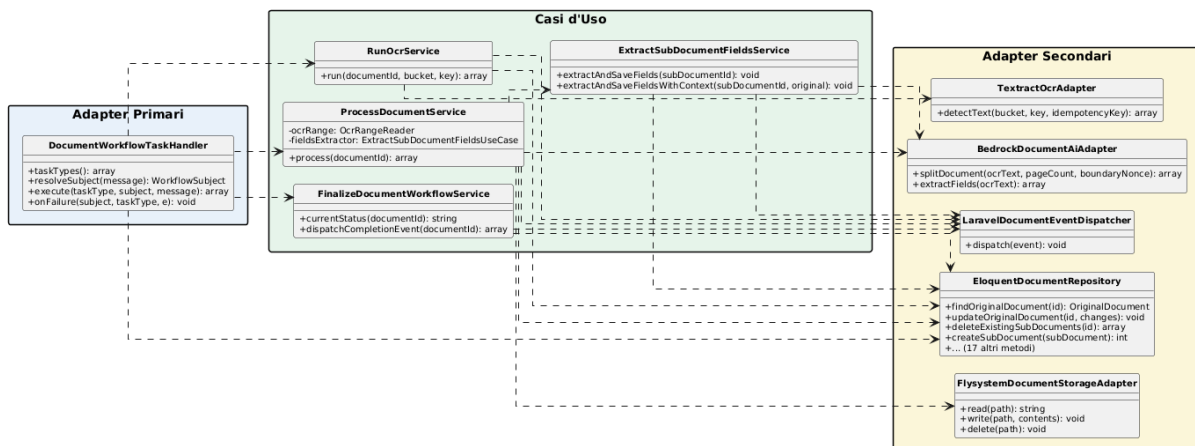


Figura 9: Diagramma classi parziale: Flusso di elaborazione OCR/AI

Il quarto diagramma mostra il percorso di revisione umana e invio: i tre adapter primari dedicati (`DocumentReviewController`, `DocumentPreviewController`, `SendMessageController`) delegano rispettivamente a `ReviewDocumentService`, `PreviewDocumentService` e `SendMessageService`.

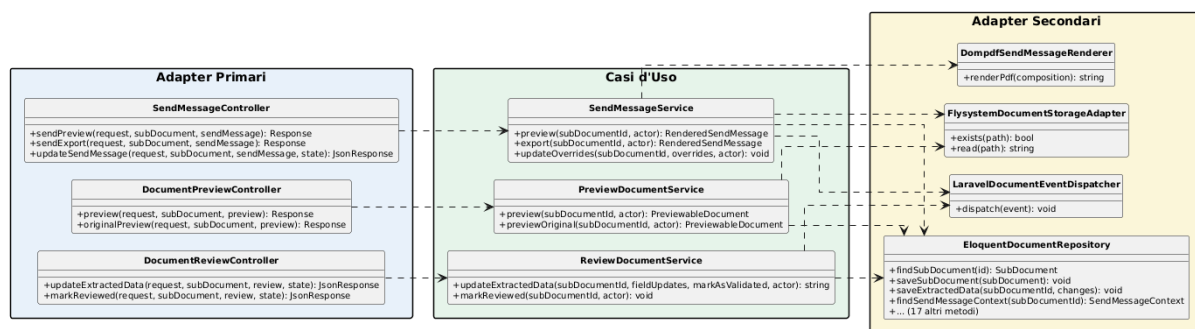


Figura 10: Diagramma classi parziale: Flusso di revisione e invio

Il quinto diagramma completa il quadro con il flusso di consultazione ed eliminazione: `ListDocumentsService` e `PollDocumentProgressService` leggono lo stato tramite `EloquentDocumentRepository`, mentre `DeleteDocumentService` ne orchestra anche la cancellazione del file fisico e la pubblicazione dell'evento di dominio. È qui che compare per la prima volta l'entità `SubDocument`.

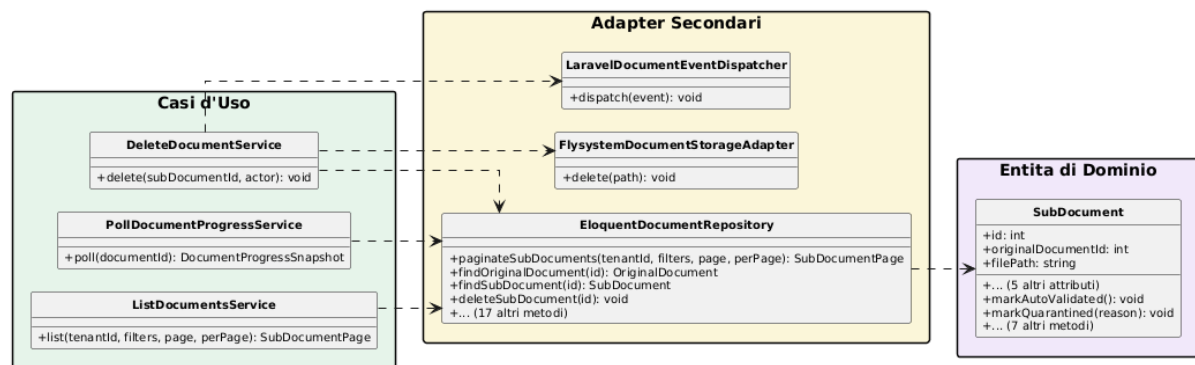


Figura 11: Diagramma classi parziale: Flusso di consultazione ed eliminazione

5.1.2 Diagrammi di sequenza: caricamento ed elaborazione di un documento

La sequenza è divisa in due diagrammi lungo il confine reale della pipeline: la parte sincrona, servita direttamente dall'endpoint HTTP, e la parte asincrona, orchestrata da Step Functions ed eseguita dal Worker.

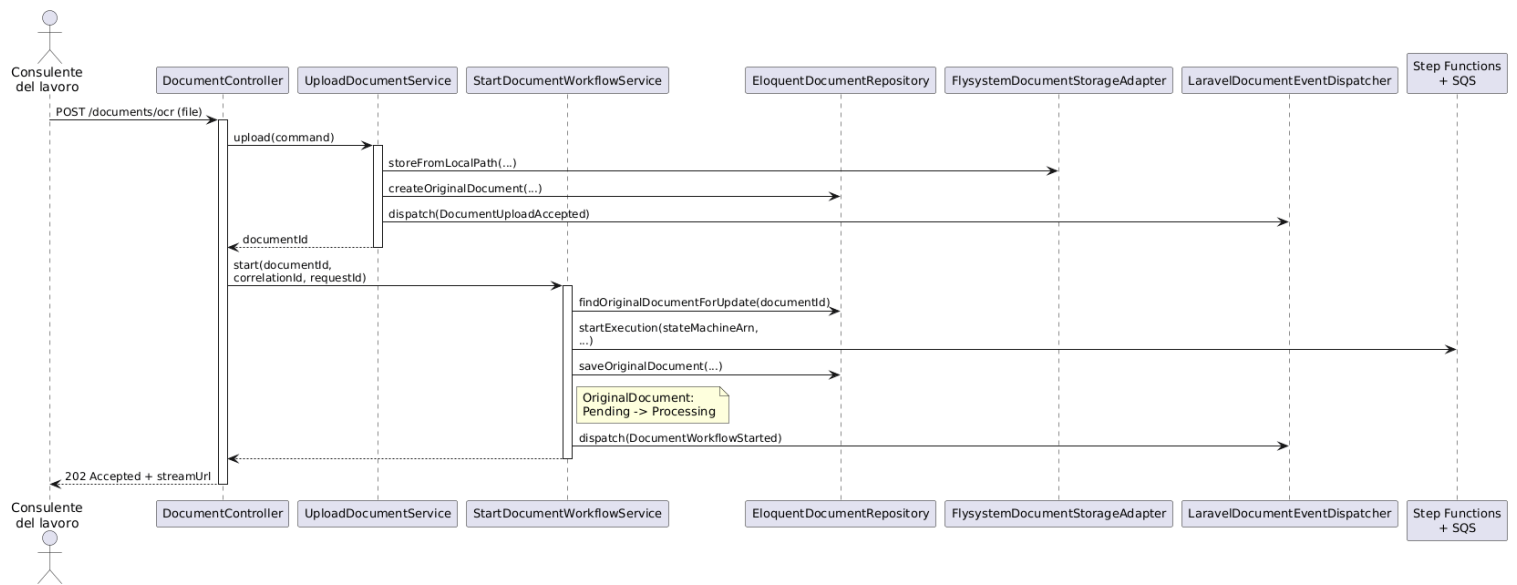


Figura 12: Diagramma di sequenza: Upload e avvio della pipeline

Il primo diagramma descrive la sequenza delle operazioni che avvengono quando un consulente del lavoro carica un documento tramite l'endpoint dedicato. Il `DocumentController` riceve la richiesta, delega il salvataggio del file a `UploadDocumentService` e, subito dopo, l'avvio della pipeline a `StartDocumentWorkflowUseCase`; la risposta HTTP torna al client con l'URL di streaming, senza attendere l'elaborazione.

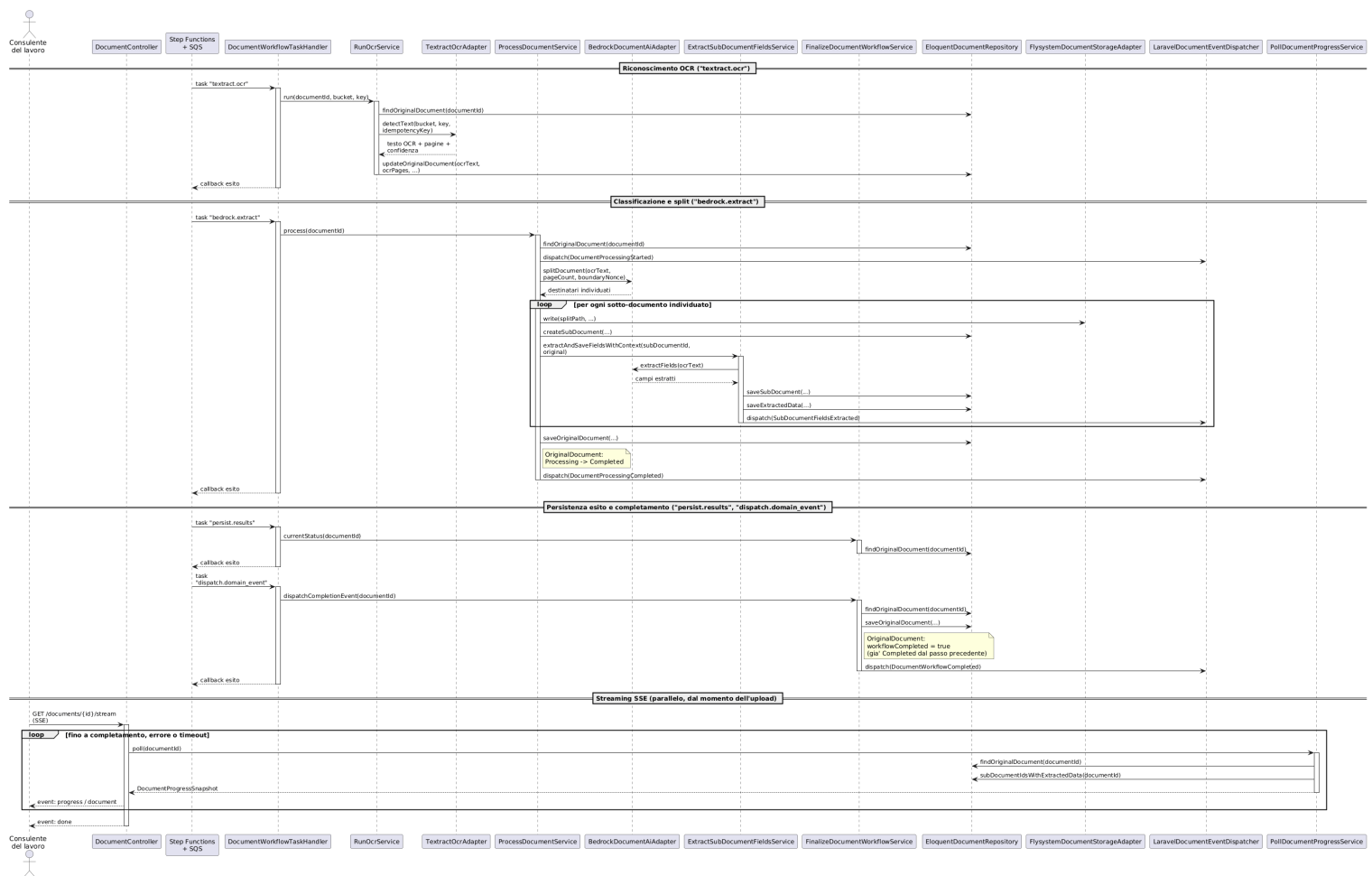


Figura 13: Diagramma di sequenza: Elaborazione asincrona e streaming SSE

Da quel momento la sequenza prosegue in modo asincrono, come mostra il secondo diagramma. Il Worker riceve dalla state machine Step Functions un task alla volta e lo smista al caso d'uso corrispondente, un passo per fase della pipeline: riconoscimento ottico, classificazione, estrazione dei campi per ogni destinatario individuato, infine persistenza dei risultati e pubblicazione dell'evento di completamento.

In termini di classi, quello smistamento passa per `DocumentWorkflowTaskHandler`, che delega rispettivamente a `RunOcrService` (`textextract.ocr`), `ProcessDocumentService` (`bedrock.extract`, che a sua volta delega l'estrazione dei campi per ogni destinatario a `ExtractSubDocumentFieldsService`, Sezione 5.1) e `FinalizeDocumentWorkflowUseCase` (`persist.results`, `dispatch.domain_event`). La transizione di stato *Processing* → *Completed* avviene già durante `bedrock.extract`, quando `ProcessDocumentService` completa lo split: `persist.results` si limita a leggerlo tramite `DocumentRepository`, senza scriverlo; `dispatch.domain_event` marca invece separatamente il workflow come concluso (`workflowCompleted`) e pubblica l'evento finale.

In parallelo, dal momento dell'upload, la SPA resta connessa all'endpoint SSE esposto da `DocumentController::stream()`, che interroga `PollDocumentProgressUseCase`

a ogni iterazione e inoltra al client i sotto-documenti man mano che vengono estratti, fino all'evento finale di completamento o di errore: anch'essa illustrata nel secondo diagramma.

5.1.3 Tutte le classi del modulo Documents

Dopo aver mostrato i diagrammi delle classi e di sequenza, si riportano di seguito le descrizioni delle classi principali del modulo Documents (adapter primari, entità di dominio, casi d'uso applicativi e adapter secondari), con i dettagli di attributi e metodi per ciascuna. Eventi, listener, value object, enum ed eccezioni sono citati nel contesto delle classi che li usano, non descritti separatamente.

5.1.3.1 DocumentController

Adapter primario HTTP del dominio: riceve le richieste, le traduce in chiamate ai casi d'uso tramite le rispettive porte primarie e restituisce la risposta al client. Non contiene alcuna regola di business: quella vive nel dominio e nell'applicazione.

`store()` incatena due casi d'uso distinti, caricamento e avvio del workflow. `stream()` espone invece l'avanzamento della pipeline via Server-Sent Events: il controller resta responsabile solo del protocollo SSE (loop, heartbeat, timeout), leggendo lo stato tramite `PollDocumentProgressUseCase` invece di interrogare direttamente il database a ogni iterazione.

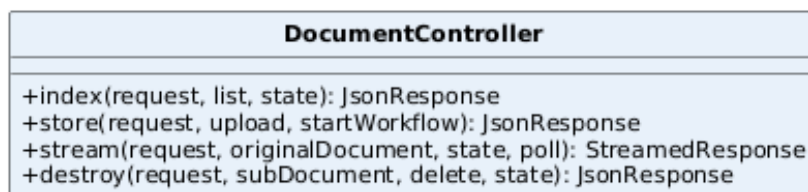


Figura 14: Classe: DocumentController

Attributi

- Nessuno.

Metodi

- `index(request: ListDocumentsRequest, list: ListDocumentsUseCase, state: MvpStateService): JsonResponse`
- `store(request: UploadDocumentRequest, upload: UploadDocumentUseCase, startWorkflow: StartDocumentWorkflowUseCase): JsonResponse`
- `stream(request: Request, originalDocument: OriginalDocument, state: MvpStateService, poll: PollDocumentProgressUseCase): StreamedResponse`
- `destroy(request: Request, subDocument: SubDocument, delete: DeleteDocumentUseCase, state: MvpStateService): JsonResponse`

5.1.3.2 DocumentPreviewController, SendMessageController

Due Controller separati da `DocumentController`, sullo stesso principio di traduzione pura verso una porta primaria: `DocumentPreviewController` espone l'anteprima del file del sotto-documento e, con `originalPreview()`, anche quella del documento originale completo non splittato, come alternativa indipendente quando la prima non è disponibile; `SendMessageController` copre l'intero ciclo del messaggio di invio (anteprima PDF composta, esportazione, correzione delle sovrascritture manuali). `DocumentPreviewController` traduce un fallimento applicativo in una risposta HTTP appropriata per entrambi i suoi metodi, invece di lasciarlo propagare: converte `DocumentPreviewUnavailableException` in un 404 e un `RuntimeException` generico (storage non raggiungibile) in un 503. `SendMessageController` fa l'opposto di proposito: non cattura più `SendMessageNotConfirmedException`/`SendMessageAttachmentUnavailableException`, lasciandole risalire al gestore centralizzato in `bootstrap/app.php` come ogni altro errore di dominio, così la risposta porta sempre `requestId/correlationId`, che una risposta costruita a mano qui non includerebbe.

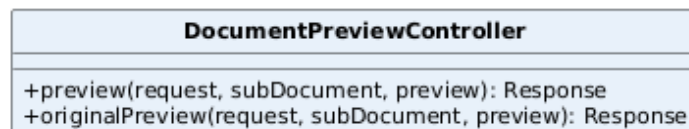


Figura 15: Classe: DocumentPreviewController



Figura 16: Classe: SendMessageController

Attributi

- Nessuno.

Metodi

- `preview(request: Request, subDocument: SubDocument, preview: PreviewDocumentUseCase): Response`
- `originalPreview(request: Request, subDocument: SubDocument, preview: PreviewDocumentUseCase): Response`
- `sendPreview(request: Request, subDocument: SubDocument, sendMessage: SendMessageUseCase): Response`

- `sendExport(request: Request, subDocument: SubDocument, sendMessage: SendMessageUseCase): Response`
- `updateSendMessage(request: UpdateSendMessageRequest, subDocument: SubDocument, sendMessage: SendMessageUseCase, state: MvpStateService): JsonResponse`

5.1.3.3 DocumentWorkflowTaskHandler

Adapter primario guidato da Step Functions/SQS invece che da HTTP: sullo stesso piano concettuale del Controller, traduce ogni task di callback in arrivo dalla pipeline in una chiamata al caso d'uso corrispondente, tramite la sua porta primaria. Non contiene regole di business: l'idempotenza verso la riconsegna di un task (*documento già completato*) vive nei singoli casi d'uso, non in questo adapter.

Risolve e aggiorna l'aggregato documentale attraverso lo stesso `DocumentRepository` usato dai casi d'uso, mai attraverso Eloquent diretto. Comunica con il runner condiviso passando solo un identificativo e un tenant, così da restare agnostico rispetto al dominio.

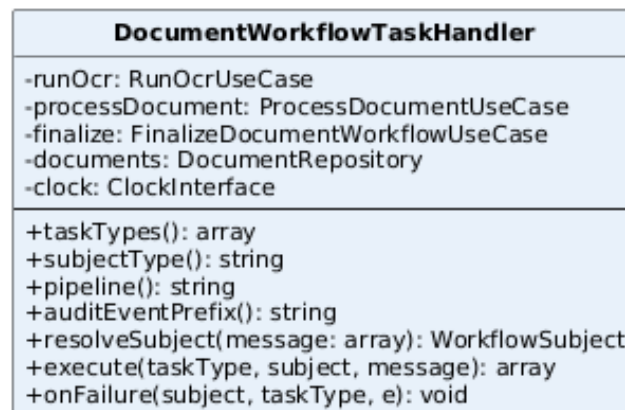


Figura 17: Classe: DocumentWorkflowTaskHandler

Attributi

- `runOcr: RunOcrUseCase, processDocument: ProcessDocumentUseCase, finalize: FinalizeDocumentWorkflowUseCase, documents: DocumentRepository, clock: ClockInterface.`

Metodi

- `taskTypes(): array`
- `subjectType(): string`
- `pipeline(): string`: restituisce l'identificativo della pipeline di appartenenza ("documents").
- `auditEventPrefix(): string`: prefisso ("mvp-document-workflow-task-") usato dal runner condiviso per nominare gli eventi di audit di questo dominio.

- `resolveSubject(message: array): WorkflowSubject`
- `execute(taskType: string, subject: WorkflowSubject, message: array): array`
- `onFailure(subject: WorkflowSubject, taskType: string, e: Throwable): void`

5.1.3.4 OriginalDocument

Entità di dominio che rappresenta il documento caricato dall'utente e il suo avanzamento nella pipeline di elaborazione. Non è una semplice proiezione dei dati salvati: governa da sé le proprie transizioni di stato, invece di lasciare a ogni caso d'uso il compito di scriverle correttamente a mano.

`startProcessing()` e `fail()` consolidano in un unico posto le transizioni di stato della lavorazione: la prima porta il documento in elaborazione registrando l'esecuzione avviata, la seconda lo porta a fallito distinguendo il messaggio pensato per l'operatore dal dettaglio tecnico usato per il troubleshooting. Non governa invece i campi restituiti dal riconoscimento ottico (testo, pagine, confidenza): non sono una transizione di stato, solo dati letti dal gateway OCR e scritti direttamente dal caso d'uso che li riceve.

OriginalDocument
+id: int +tenantId: string +filePath: string +manualDocumentType: ?string +manualCompanyName: ?string +manualReferenceMonth: ?int +manualReferenceYear: ?int -processingStatus: ProcessingStatus +ocrText: ?string +ocrPages: array +ocrConfidenceAvg: ?float -s3Bucket: ?string -s3Key: ?string -workflowCompleted: bool -workflowExecutionArn: ?string -errorMessage: ?string +originalFilename: ?string
+fromRecord(record: OriginalDocumentRecord): self +processingStatus(): ProcessingStatus +errorMessage(): ?string +s3Bucket(): ?string +s3Key(): ?string +workflowExecutionArn(): ?string +workflowCompleted(): bool +hasManualUploadMetadata(): bool +startProcessing(executionArn, s3Bucket, s3Key, startedAt): void +fail(errorMessage, technicalReason, failedAt): void +complete(): void +markWorkflowCompleted(completedAt: DateTimeImmutable): void +pendingChanges(): OriginalDocumentChanges

Figura 18: Classe: OriginalDocument

Attributi

- id, tenantId, filePath, originalFilename, i metadati manuali di caricamento, processingStatus, il risultato OCR, i riferimenti all'esecuzione Step Functions.

Metodi

- fromRecord(record: OriginalDocumentRecord): self
- processingStatus(): ProcessingStatus, errorMessage(): ?string, s3Bucket(): ?string, s3Key(): ?string, workflowExecutionArn(): ?string, workflowCompleted(): bool: accessori di sola lettura sui rispettivi attributi privati.
- hasManualUploadMetadata(): bool: vero se l'utente ha fornito almeno uno dei metadati manuali di caricamento (tipo documento, ragione sociale, mese/anno di riferimento).
- startProcessing(executionArn: string, s3Bucket: string, s3Key: string, startedAt: DateTimeImmutable): void

- `fail(errorMessage: string, technicalReason: ?string, failedAt: DateTimeImmutable): void`
- `complete(): void`
- `markWorkflowCompleted(completedAt: DateTimeImmutable): void`
- `pendingChanges(): OriginalDocumentChanges`

5.1.3.5 SubDocument

Entità di dominio che rappresenta un sotto-documento generato dallo split di un documento originale, e governa la propria revisione: può essere validato automaticamente dal sistema, messo in quarantena con un motivo esplicito quando l'estrazione ha una confidenza troppo bassa, oppure validato manualmente da un operatore.

Governa anche il proprio stato di invio (`setStatus()`, `markSent()`): il download del PDF esportato tramite `SendMessageService::export()` lo marca *Sent* in una transizione a senso unico (Sezione 5.1, `SendMessageService`). Non governa invece le eventuali sovrascritture apportate al messaggio destinato al destinatario (destinatario, oggetto, corpo): quei valori vivono a parte in `SendMessageContext`, letto insieme ai dati estratti al momento della composizione, non come stato proprio dell'entità.

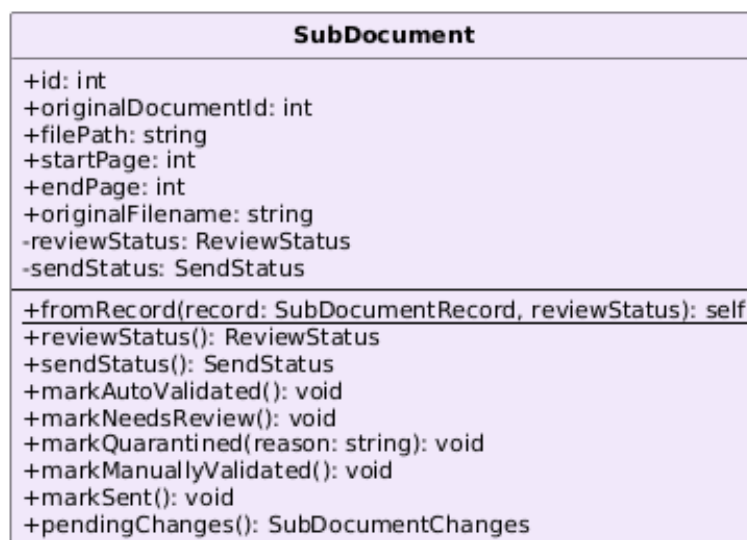


Figura 19: Classe: SubDocument

Attributi

- `id`, `originalDocumentId`, `filePath`, il range di pagine, `originalFilename`, `reviewStatus`, `setStatus`.

Metodi

- `fromRecord(record: SubDocumentRecord, reviewStatus: ReviewStatus): self`

- `reviewStatus(): ReviewStatus, sendStatus(): SendStatus`: accessori di sola lettura sui rispettivi attributi privati.
- `markAutoValidated(): void`
- `markNeedsReview(): void`
- `markQuarantined(reason: string): void`
- `markManuallyValidated(): void`
- `markSent(): void`
- `pendingChanges(): SubDocumentChanges`

5.1.3.6 UploadDocumentService

Implementa `UploadDocumentUseCase` ed è il punto di ingresso della pipeline documentale: riceve il file già validato dall'adapter HTTP, lo salva su storage e crea il record del documento originale con stato *Pending*. Deliberatamente non avvia la pipeline di elaborazione: quella è una decisione distinta, delegata a `StartDocumentWorkflowUseCase` e invocata in sequenza dallo stesso Controller.

L'audit passa dal dispatch dell'evento di dominio `DocumentUploadAccepted` (id documento, attore, nome file, metadati manuali), registrato da un listener dedicato: stesso principio in uso per il resto degli eventi di dominio.



Figura 20: Classe: UploadDocumentService

Attributi

- `documents: DocumentRepository, storage: DocumentStoragePort, events: DocumentEventDispatcherPort`.

Metodi

- `upload(command: UploadDocumentCommand): int`

5.1.3.7 StartDocumentWorkflowService

Implementa `StartDocumentWorkflowUseCase`: avvia l'esecuzione Step Functions della pipeline per un documento già persistito, subito dopo `UploadDocumentService` nello stesso ciclo di richiesta. È idempotente rispetto a un documento già in elaborazione: il controllo "è già in corso?" e la scrittura che lo rende vero (oltre alla

chiamata a Step Functions) girano dentro un'unica transazione, con lettura tramite `DocumentRepository::findOriginalDocumentForUpdate()` (lock pessimistico sulla riga): senza, due richieste quasi simultanee potrebbero superare entrambe il controllo e avviare due esecuzioni per lo stesso documento, una delle quali resterebbe orfana. Gli eventi di dominio (avvio riuscito o fallito) sono dispatchati solo dopo che la transazione ha commesso, altrimenti un rollback sul ramo di fallimento annullerebbe anche il salvataggio dello stato "fallito" appena scritto. Esegue anche un controllo esplicito a runtime: quando Textract reale è abilitato, verifica che i documenti risiedano effettivamente su S3 reale e non sul disco LocalStack, fallendo subito con un messaggio azionabile invece di lasciare che Textract fallisca più a valle con un errore criptico. In caso di errore nell'avvio, marca il documento come fallito tramite `OriginalDocument::fail()` prima di rilanciare l'eccezione.

Non legge configurazione tramite `config()` al proprio interno: ARN della state machine, URL della coda, flag Textract, disco e bucket di storage sono risolti una sola volta in `AppServiceProvider` e iniettati come scalari nel costruttore, lo stesso trattamento riservato a soglia di confidenza e prefisso storage altrove nel dominio. Poiché anche `WorkflowContext` è una classe pura senza dipendenze da Illuminate, `start()` è interamente istanziabile ed eseguibile in un test Pest puro, senza avviare Laravel.

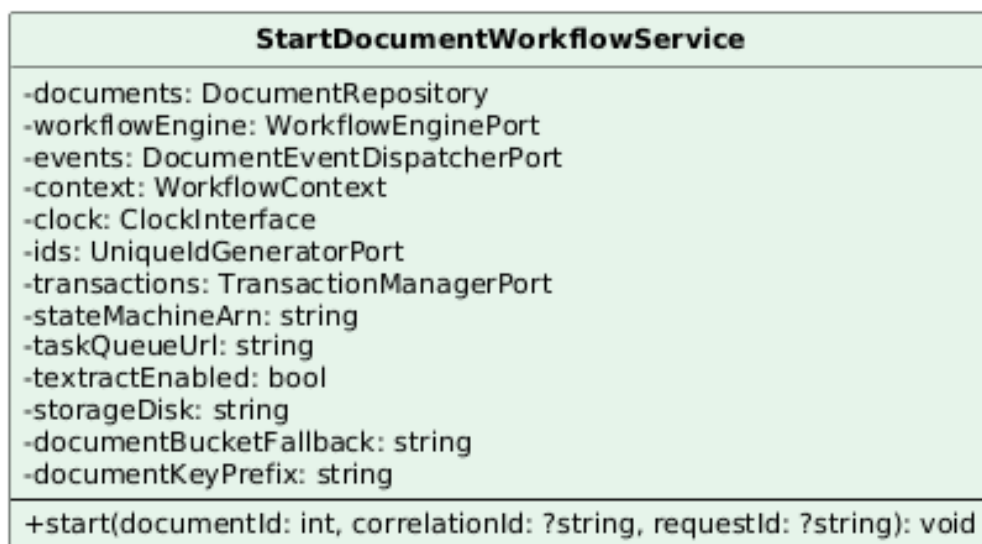


Figura 21: Classe: StartDocumentWorkflowService

Attributi

- documents: DocumentRepository, workflowEngine: WorkflowEnginePort, events: DocumentEventDispatcherPort, context: WorkflowContext, clock: ClockInterface, ids: UniqueIdGeneratorPort, transactions: TransactionManagerPort.
- stateMachineArn: string, taskQueueUrl: string, textractEnabled: bool, storageDisk: string, documentBucketFallback: string, documentKeyPrefix: string: scalari risolti da configurazione in AppServiceProvider.

Metodi

- `start(documentId: int, correlationId: ?string, requestId: ?string): void`

5.1.3.8 RunOcrService

Implementa `RunOcrUseCase`: invoca il gateway OCR per il documento e ne salva il risultato grezzo (testo, pagine, confidenza media, id del job Textract) tramite `OriginalDocumentChanges`. In caso di errore marca il documento come fallito prima di rilanciare l'eccezione.

È idempotente verso la riconsegna del task workflow (un documento già completato restituisce immediatamente senza richiamare Textract) e deriva la chiave di idempotenza anche dall'oggetto S3, non solo dall'id del documento: dopo un reset seguito da un nuovo upload lo stesso id può puntare a un file diverso, e una chiave fissa farebbe scattare un conflitto contro il job precedente.

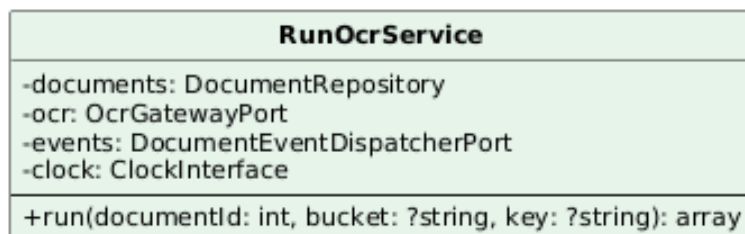


Figura 22: Classe: RunOcrService

Attributi

- `documents: DocumentRepository`, `ocr: OcrGatewayPort`, `events: DocumentEventDispatcherPort`, `clock: ClockInterface`.

Metodi

- `run(documentId: int, bucket: ?string, key: ?string): array`

5.1.3.9 ProcessDocumentService

Coordina la fase più corposa della pipeline documentale: la classificazione del documento tramite Bedrock e, per ogni destinatario individuato, la delega dell'estrazione dei campi a `ExtractSubDocumentFieldsService`.

Le sue dipendenze sono fra le più numerose del dominio, perché riflettono quante integrazioni coordina in un solo passo: un repository, uno storage, un gateway AI, un dispatcher di eventi, un heartbeat verso Step Functions (per segnalare che l'elaborazione è ancora viva durante chiamate AI che possono durare minuti), oltre a un generatore di identificativi e un lettore OCR condiviso.

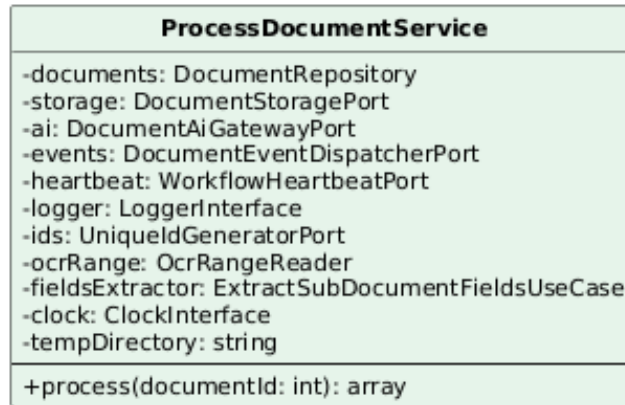


Figura 23: Classe: ProcessDocumentService

Attributi

- documents: DocumentRepository, storage: DocumentStoragePort, ai: DocumentAiGatewayPort, events: DocumentEventDispatcherPort, heartbeat: WorkflowHeartbeatPort, logger: LoggerInterface, ids: UniqueIdGeneratorPort, ocrRange: OcrRangeReader, fieldsExtractor: ExtractSubDocumentFieldsUseCase, clock: ClockInterface, tempDirectory: string.

Metodi

- process(documentId: int): array

5.1.3.10 ExtractSubDocumentFieldsService

Implementa `ExtractSubDocumentFieldsUseCase`: estrae nove campi (nome, cognome, azienda, data documento, tipo documento, descrizione, email destinatario, codice fiscale, matricola) di un singolo sotto-documento tramite Bedrock, un destinatario alla volta. Separata da `ProcessDocumentService` (che orchestra lo split PDF e invoca questa porta per ciascun destinatario appena creato), per isolare questa logica dalla manipolazione PDF: è così testabile indipendentemente da quest'ultima.

I metadati dichiarati manualmente in fase di upload prevalgono sempre su quelli estratti dall'AI, ed escludono il campo corrispondente dal calcolo della confidenza (un'azienda o una data dichiarate dall'utente non hanno bisogno di essere lette dall'AI per essere affidabili). Codice fiscale, email e matricola sono identificativi, non semplici estrazioni: un valore che non supera il proprio controllo formale (email tramite `filter_var`, codice fiscale tramite `CodiceFiscale::isValid()`) torna a `null` invece di essere salvato: un dato falso che l'operatore non avrebbe modo di riconoscere come tale. La matricola non ha una forma dichiarata e passa così com'è.

Il punteggio di confidenza del sotto-documento è la confidenza più bassa fra i suoi campi chiave (nome, cognome, azienda, data, non la media: un documento è affidabile quanto il suo dato meno affidabile), ciascuna letta dalla riga OCR specifica da cui il campo proviene tramite `FieldConfidence` (Sezione 5.1, `TextextractOcrAdapter`), con la confidenza media di pagina come ripiego quando un campo non è rintracciabile fra le righe. Un

campo chiave assente azzerà il punteggio. Sopra soglia (`confidenceThreshold`, default 80) il sotto-documento è validato automaticamente, salvo che porti un codice fiscale sotto la propria soglia dedicata (`fiscalCodeConfidenceThreshold`, default 95, più alta perché un carattere letto male assegnerebbe il documento alla persona sbagliata), altrimenti è marcato per revisione manuale. Un output AI non conforme allo schema atteso mette il sotto-documento in quarantena invece di propagare l'errore.

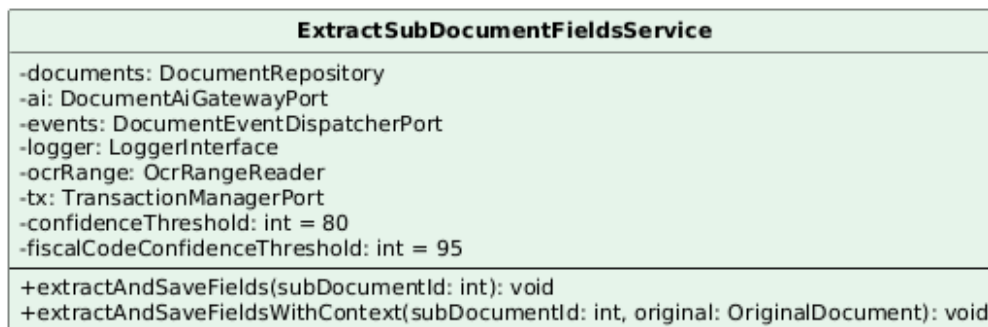


Figura 24: Classe: ExtractSubDocumentFieldsService

Attributi

- `documents: DocumentRepository`, `ai: DocumentAiGatewayPort`, `events: DocumentEventDispatcherPort`, `logger: LoggerInterface`, `ocrRange: OcrRangeReader`, `tx: TransactionManagerPort`, `confidenceThreshold: int` (default 80), `fiscalCodeConfidenceThreshold: int` (default 95).

Metodi

- `extractAndSaveFields(subDocumentId: int): void`
- `extractAndSaveFieldsWithContext(subDocumentId: int, original: OriginalDocument): void`

5.1.3.11 FinalizeDocumentWorkflowService

Implementa `FinalizeDocumentWorkflowUseCase`: chiude la pipeline documentale, marcando il workflow come completato sull'entità `OriginalDocument` quando lo stato di elaborazione è *Completed*, e pubblicando l'evento di completamento che la SPA osserva tramite lo stream SSE.

`currentStatus()` espone lo stato corrente per i punti della pipeline che devono decidere se procedere; `dispatchCompletionEvent()` distingue nel proprio risultato un completamento effettivo da un semplice avanzamento, così che il chiamante possa scegliere l'evento SSE corretto da inoltrare al client.

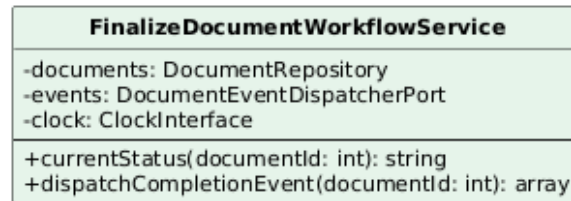


Figura 25: Classe: FinalizeDocumentWorkflowService

Attributi

- documents: DocumentRepository, events: DocumentEventDispatcherPort, clock: ClockInterface.

Metodi

- currentStatus(documentId: int): string
- dispatchCompletionEvent(documentId: int): array

5.1.3.12 PollDocumentProgressService

Implementa PollDocumentProgressUseCase: legge un'istantanea dello stato di avanzamento di un documento (stato di elaborazione, eventuale messaggio d'errore, identificativi dei sotto-documenti per cui l'estrazione è già disponibile), consumata da DocumentController::stream() a ogni iterazione del loop SSE invece di interrogare direttamente il repository dal controller.



Figura 26: Classe: PollDocumentProgressService

Attributi

- documents: DocumentRepository.

Metodi

- poll(documentId: int): DocumentProgressSnapshot

5.1.3.13 DocumentReviewController

Adapter primario HTTP dedicato alla revisione human-in-the-loop: traduce le due operazioni di correzione manuale verso ReviewDocumentUseCase, senza regole di business proprie. updateExtractedData() accetta anche un flag opzionale che marca contestualmente il sotto-documento come validato manualmente, oltre a correggerne i campi.

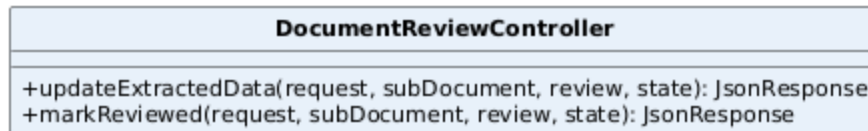


Figura 27: Classe: DocumentReviewController

Attributi

- Nessuno.

Metodi

- `updateExtractedData(request: UpdateExtractedDataRequest, subDocument: SubDocument, review: ReviewDocumentUseCase, state: MvpStateService): JsonResponse`
- `markReviewed(request: Request, subDocument: SubDocument, review: ReviewDocumentUseCase, state: MvpStateService): JsonResponse`

5.1.3.14 ReviewDocumentService

Implementa `ReviewDocumentUseCase`: gestisce la revisione manuale di un sotto-documento da parte dell'operatore, con due metodi che coprono due percorsi distinti. Entrambi verificano per primi che il tenant dell'attore coincida con quello del documento originale del sotto-documento (`DocumentNotAuthorizedException` altrimenti), difesa in profondità dietro il controllo già fatto a livello HTTP.

`updateExtractedData()` applica correzioni ai campi estratti; scrive comunque i dati estratti se il sotto-documento non ne ha ancora, anche in assenza di modifiche. Lo stato di revisione che ne segue non è un semplice riflesso del parametro `markAsValidated` ricevuto dal chiamante: se *true* valida manualmente il sotto-documento, ma se *false* lo rimanda in revisione solo quando non era già validato manualmente: una correzione non declassa una scheda che una persona aveva già confermato, perché quei dati sono appena passati un'altra volta sotto i suoi occhi, e declassarli chiederebbe di confermare due volte la stessa scheda. `markReviewed()` valida invece un sotto-documento senza modificarne i campi, e rifiuta l'operazione se non esistono ancora dati estratti da validare.

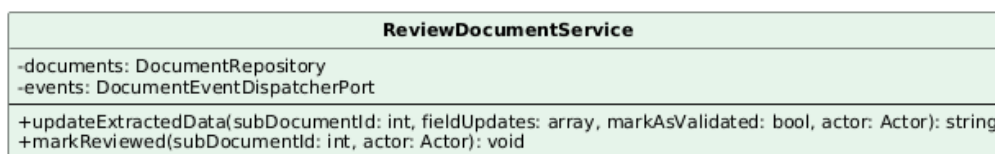


Figura 28: Classe: ReviewDocumentService

Attributi

- `documents: DocumentRepository, events: DocumentEventDispatcherPort.`

Metodi

- `updateExtractedData(subDocumentId: int, fieldUpdates: array, markAsValidated: bool, actor: Actor): string`
- `markReviewed(subDocumentId: int, actor: Actor): void`

5.1.3.15 PreviewDocumentService

Implementa `PreviewDocumentUseCase`: restituisce il contenuto grezzo del file di un sotto-documento per l'anteprima nella SPA, verificando prima che esista ancora sullo storage. Distinta da `SendMessageService::preview()`, che genera invece un PDF composto a partire dai dati estratti, non il file originale.

`previewOriginal()` risale invece dal sotto-documento al suo `OriginalDocument` e ne restituisce il file completo, non splittato: un'alternativa indipendente dallo stato dell'anteprima del sotto-documento, utile quando quest'ultima non è disponibile ma il documento originale caricato è ancora leggibile dallo storage.

Verifica anche l'autorizzazione: se il tenant dell'attore non coincide con quello del documento (originale o del sotto-documento, a seconda del metodo), solleva `DocumentNotAuthorizedException` prima di leggere il file.

PreviewDocumentService
-documents: DocumentRepository -storage: DocumentStoragePort
+preview(subDocumentId: int, actor: Actor): PreviewableDocument +previewOriginal(subDocumentId: int, actor: Actor): PreviewableDocument

Figura 29: Classe: PreviewDocumentService

Attributi

- `documents: DocumentRepository, storage: DocumentStoragePort.`

Metodi

- `preview(subDocumentId: int, actor: Actor): PreviewableDocument`
- `previewOriginal(subDocumentId: int, actor: Actor): PreviewableDocument`

5.1.3.16 SendMessageService

Implementa `SendMessageUseCase`: compone il messaggio di invio precompilato a partire dai dati già estratti (nessuna generazione AI coinvolta) e ne orchestra antepri-
ma, esportazione e correzione manuale. Destinatario, corpo e oggetto hanno un valore proposto di default, sovrascrivibile dall'operatore tramite le sovrascritture salvate sul sotto-documento; l'oggetto proposto nomina il tipo di documento e il suo periodo di riferimento (mese/anno, se dichiarati), non l'invio in sé, che a quel punto non

è ancora avvenuto. Sia l'anteprima sia l'esportazione accodano al PDF composto anche il sotto-documento del destinatario prodotto dallo split (letto dallo storage tramite `DocumentStoragePort`, `SendMessageAttachmentUnavailableException` se assente): chi riceve il messaggio riceve anche il documento a cui si riferisce, non solo il testo.

Il download è vincolato a una revisione umana già avvenuta: `export()` rifiuta con `SendMessageNotConfirmedException` se il sotto-documento non è ancora *ManuallyValidated*: il pannello tiene spento il comando finché non lo è, ma la API resterebbe altrimenti chiamabile anche senza passare dal pannello. Il recapito avviene fuori dalla piattaforma: poiché il sistema non può osservare l'invio reale, è proprio il download del PDF esportato a marcare lo stato come *Sent*, in una transizione a senso unico: un secondo download non la ripete. Tutti e tre i metodi pubblici verificano l'autorizzazione tramite un metodo privato condiviso (`assertActorOwnsSubDocument()`), che solleva `DocumentNotAuthorizedException` in caso di tenant non corrispondente.

SendMessageService
-documents: DocumentRepository -renderer: SendMessageRendererPort -storage: DocumentStoragePort -events: DocumentEventDispatcherPort
+preview(subDocumentId: int, actor: Actor): RenderedSendMessage +export(subDocumentId: int, actor: Actor): RenderedSendMessage +updateOverrides(subDocumentId: int, overrides: array, actor: Actor): void

Figura 30: Classe: SendMessageService

Attributi

- documents: DocumentRepository, renderer: SendMessageRendererPort, storage: DocumentStoragePort, events: DocumentEventDispatcherPort.

Metodi

- preview(subDocumentId: int, actor: Actor): RenderedSendMessage
- export(subDocumentId: int, actor: Actor): RenderedSendMessage
- updateOverrides(subDocumentId: int, overrides: array, actor: Actor): void

5.1.3.17 ListDocumentsService, DeleteDocumentService

Due casi d'uso minimi, simmetrici ai rispettivi sul dominio Communications. `ListDocumentsService` implementa `ListDocumentsUseCase` e inoltra la paginazione al repository tramite `DocumentListFilters` (value object tipizzato), senza logica propria.

`DeleteDocumentService` implementa `DeleteDocumentUseCase`: verifica che l'attore appartenga allo stesso tenant del documento originale (`DocumentNotAuthorizedException` altrimenti), elimina un sotto-documento e, se non resta alcun altro sotto-documento collegato allo stesso documento originale, elimina

anche quest'ultimo insieme a tutti i task di workflow che lo referenziano, indipendentemente dal loro stato. In entrambi i casi il record resta rimosso dal database anche se la pulizia dello storage fallisce: un guasto di rete verso S3 non deve far fallire un'eliminazione già avvenuta dal punto di vista dell'utente, a costo di lasciare temporaneamente un file orfano.

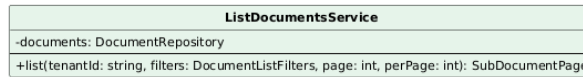


Figura 31: Classe: ListDocumentsService

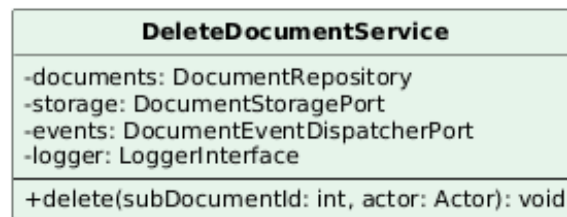


Figura 32: Classe: DeleteDocumentService

Attributi

- ListDocumentsService: documents: DocumentRepository.
- DeleteDocumentService: documents: DocumentRepository, storage: DocumentStoragePort, events: DocumentEventDispatcherPort, logger: LoggerInterface.

Metodi

- list(tenantId: string, filters: DocumentListFilters, page: int, perPage: int): SubDocumentPage
- delete(subDocumentId: int, actor: Actor): void

5.1.3.18 EloquentDocumentRepository

Adapter secondario che implementa DocumentRepository sopra Eloquent: è il punto in cui l'aggregato documentale (documento originale e sotto-documenti) viene effettivamente letto e scritto sul database. Consuma i value object tipizzati del dominio (NewOriginalDocument, OriginalDocumentChanges, SubDocumentChanges...) invece di array associativi grezzi, così che un refuso nel nome di una colonna venga intercettato dall'analisi statica invece che a runtime.

EloquentDocumentRepository
<pre> +createOriginalDocument(document: NewOriginalDocument): int +findOriginalDocument(id: int): OriginalDocument +findOriginalDocumentForUpdate(id: int): OriginalDocument +updateOriginalDocument(id: int, changes: OriginalDocumentChanges): void +saveOriginalDocument(document: OriginalDocument): void +deleteOriginalDocumentWithWorkflowTasks(id: int): void +paginateSubDocuments(tenantId, filters, page, perPage): SubDocumentPage +findSubDocument(id: int): SubDocument +saveSubDocument(subDocument: SubDocument): void +createSubDocument(subDocument: NewSubDocument): int +updateSubDocument(id: int, changes: SubDocumentChanges): void +deleteSubDocument(id: int): void +originalDocumentIdForSubDocument(subDocumentId: int): int +originalDocumentHasRemainingSubDocuments(originalDocumentId): bool +subDocumentIdsWithExtractedData(originalDocumentId): array +deleteExistingSubDocuments(originalDocumentId): array +saveExtractedData(subDocumentId: int, changes: ExtractedDataChanges): void +deleteExtractedData(subDocumentId: int): void +findSendMessageContext(subDocumentId: int): SendMessageContext +subDocumentHasExtractedData(subDocumentId: int): bool +countSubDocuments(originalDocumentId: int): int </pre>

Figura 33: Classe: EloquentDocumentRepository

Attributi

- Nessuno.

Metodi

- `createOriginalDocument(document: NewOriginalDocument): int`
- `findOriginalDocument(id: int): OriginalDocument`
- `findOriginalDocumentForUpdate(id: int): OriginalDocument`: come `findOriginalDocument()`, ma con `lockForUpdate()`, usato dai casi d'uso che racchiudono un controllo "è già in corso?" e la scrittura che lo rende vero in un'unica transazione (`StartDocumentWorkflowService`).
- `updateOriginalDocument(id: int, changes: OriginalDocumentChanges): void`
- `saveOriginalDocument(document: OriginalDocument): void`
- `deleteOriginalDocumentWithWorkflowTasks(id: int): void`
- `paginateSubDocuments(tenantId: string, filters: DocumentListFilters, page: int, perPage: int): SubDocumentPage`
- `findSubDocument(id: int): SubDocument`

- `saveSubDocument(subDocument: SubDocument): void`
- `createSubDocument(subDocument: NewSubDocument): int`
- `updateSubDocument(id: int, changes: SubDocumentChanges): void`
- `deleteSubDocument(id: int): void`
- `originalDocumentIdForSubDocument(subDocumentId: int): int`
- `originalDocumentHasRemainingSubDocuments(originalDocumentId: int): bool`
- `subDocumentIdsWithExtractedData(originalDocumentId: int): array`
- `deleteExistingSubDocuments(originalDocumentId: int): array`: legge ed elimina in un'unica transazione, restituendo i percorsi storage da ripulire.
- `saveExtractedData(subDocumentId: int, changes: ExtractedDataChanges): void`
- `deleteExtractedData(subDocumentId: int): void`
- `findSendMessageContext(subDocumentId: int): SendMessageContext`
- `subDocumentHasExtractedData(subDocumentId: int): bool`
- `countSubDocuments(originalDocumentId: int): int`

5.1.3.19 Adapter AI e OCR: `TextractOcrAdapter`, `BedrockDocumentAiAdapter`

I due adapter che collegano il dominio ai servizi AI esterni. `TextractOcrAdapter` implementa `OcrGatewayPort` sopra Amazon Textract e restituisce il risultato grezzo del riconoscimento ottico, senza toccare la persistenza: è il caso d'uso chiamante a decidere cosa salvarne. Se Textract è disattivato (`TEXTTRACT_ENABLED=false`, il default), non contatta AWS: restituisce un risultato vuoto (`enabled: false`) e il caso d'uso chiamante non persiste alcun testo OCR. Il passo successivo della pipeline non ha quindi nulla da analizzare e si interrompe con un errore esplicito prima di invocare il modello generativo: senza un Textract attivo il modulo Documents non porta a termine un'elaborazione.

`BedrockDocumentAiAdapter` implementa `DocumentAiGatewayPort` sopra il servizio condiviso `BedrockService` (Sezione 2.7), per lo split del documento e l'estrazione dei campi.

TextractOcrAdapter
-client: TextractClient -heartbeat: WorkflowTaskHeartbeat
+detectText(bucket: string, key: string, idempotencyKey: string): array

Figura 34: Classe: `TextractOcrAdapter`

BedrockDocumentAiAdapter
-bedrock: BedrockService
+splitDocument(ocrText: string, pageCount: int, boundaryNonce: string): array +extractFields(ocrText: string): array +pageBoundaryMarker(page: int, nonce: string): string +formatUserError(e: Throwable, defaultMessage: string): string

Figura 35: Classe: BedrockDocumentAiAdapter

Attributi

- `TexttractOcrAdapter: client: TexttractClient, heartbeat: WorkflowTaskHeartbeat.`
- `BedrockDocumentAiAdapter: bedrock: BedrockService.`

Metodi

- `detectText(bucket: string, key: string, idempotencyKey: string): array`
- `splitDocument(ocrText: string, pageCount: int, boundaryNonce: string): array`
- `extractFields(ocrText: string): array`
- `pageBoundaryMarker(page: int, nonce: string): string`: delega al marcatore di confine pagina di `BedrockService`, usato per far riconoscere al modello dove finisce ogni pagina nel testo OCR concatenato.
- `formatUserError(e: Throwable, defaultMessage: string): string`: delega alla formattazione errori di `BedrockService`, per un messaggio utente leggibile a partire da un'eccezione tecnica.

5.1.3.20 FlysystemDocumentStorageAdapter, DompdfSendMessageRenderer

Due adapter secondari di supporto. `FlysystemDocumentStorageAdapter` implementa `DocumentStoragePort` sopra il disco Laravel/Flysystem configurato per i documenti, generando per ogni file caricato un nome univoco (UUID) nella directory di destinazione, per evitare collisioni fra upload concorrenti.

`DompdfSendMessageRenderer` implementa `SendMessageRendererPort` sopra `Dompdf` e la vista Blade dedicata, applicando lo stesso timbro di piè di pagina (`PdfFooterStamper`) condiviso con l'esportazione delle comunicazioni. Accoda al messaggio anche il documento del destinatario, quando fornito: lo unisce tramite `Fpdi` (la stessa libreria che ha già tagliato il documento originale nei sotto-documenti, quindi il file è già importabile per costruzione), copiando le pagine del documento così come sono, senza sovrastamparci numeri o marcatori: è un documento di terzi. Il piè di pagina numera l'intero file risultante (messaggio più documento accodato), non la sola lettera.

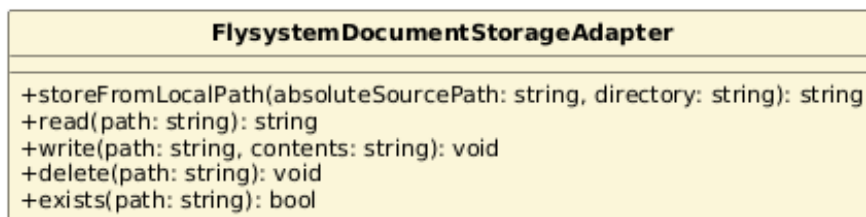


Figura 36: Classe: FlysystemDocumentStorageAdapter



Figura 37: Classe: DompdfSendMessageRenderer

Attributi

- DompdfSendMessageRenderer: footerStamper: PdfFooterStamper.
- FlysystemDocumentStorageAdapter: nessuno (il disco è risolto da configurazione a ogni chiamata).

Metodi

- storeFromLocalPath(absoluteSourcePath: string, directory: string): string
- read(path: string): string, write(path: string, contents: string): void, delete(path: string): void, exists(path: string): bool
- renderPdf(composition: SendMessageComposition, attachmentPdf: ?string): string

5.1.3.21 LaravelDocumentEventDispatcher

Adapter secondario minimo: implementa DocumentEventDispatcherPort appoggiandosi al dispatcher di eventi nativo di Laravel. I 13 listener del dominio (su 14 eventi dispatchati: DocumentWorkflowCompleted non ha un listener registrato) sono registrati nel service provider, non in questa classe, che si limita a inoltrare l'evento.

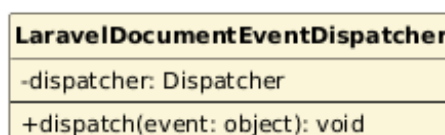


Figura 38: Classe: LaravelDocumentEventDispatcher

Attributi

- `dispatcher: Dispatcher.`

Metodi

- `dispatch(event: object): void`

5.2 Modulo Communications (AI Assistant Generativo)

Il modulo Communications è la componente del sistema dedicata alla generazione assistita di comunicazioni interne, dal titolo al testo fino all'immagine di copertina. Il suo obiettivo è offrire ai redattori aziendali uno strumento che produca contenuti coerenti con il tono e lo stile dell'organizzazione a partire da un semplice prompt, riducendo il tempo dedicato alla scrittura senza sottrarre all'utente il controllo editoriale: ogni bozza generata resta modificabile finché non viene scartata, anche dopo l'approvazione, rigenerabile alle stesse condizioni ma solo quando la generazione precedente si è conclusa (stato *completed* o *failed*), e valutabile una sola volta nell'intero suo ciclo di vita. Il modulo si integra con Amazon Bedrock sia per la generazione testuale sia per quella delle immagini di copertina, orchestrando la pipeline in modo asincrono tramite AWS Step Functions, sullo stesso schema del modulo Documents.

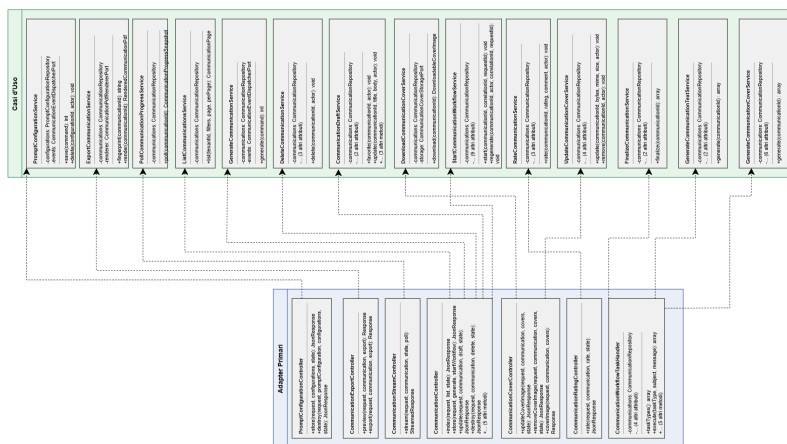
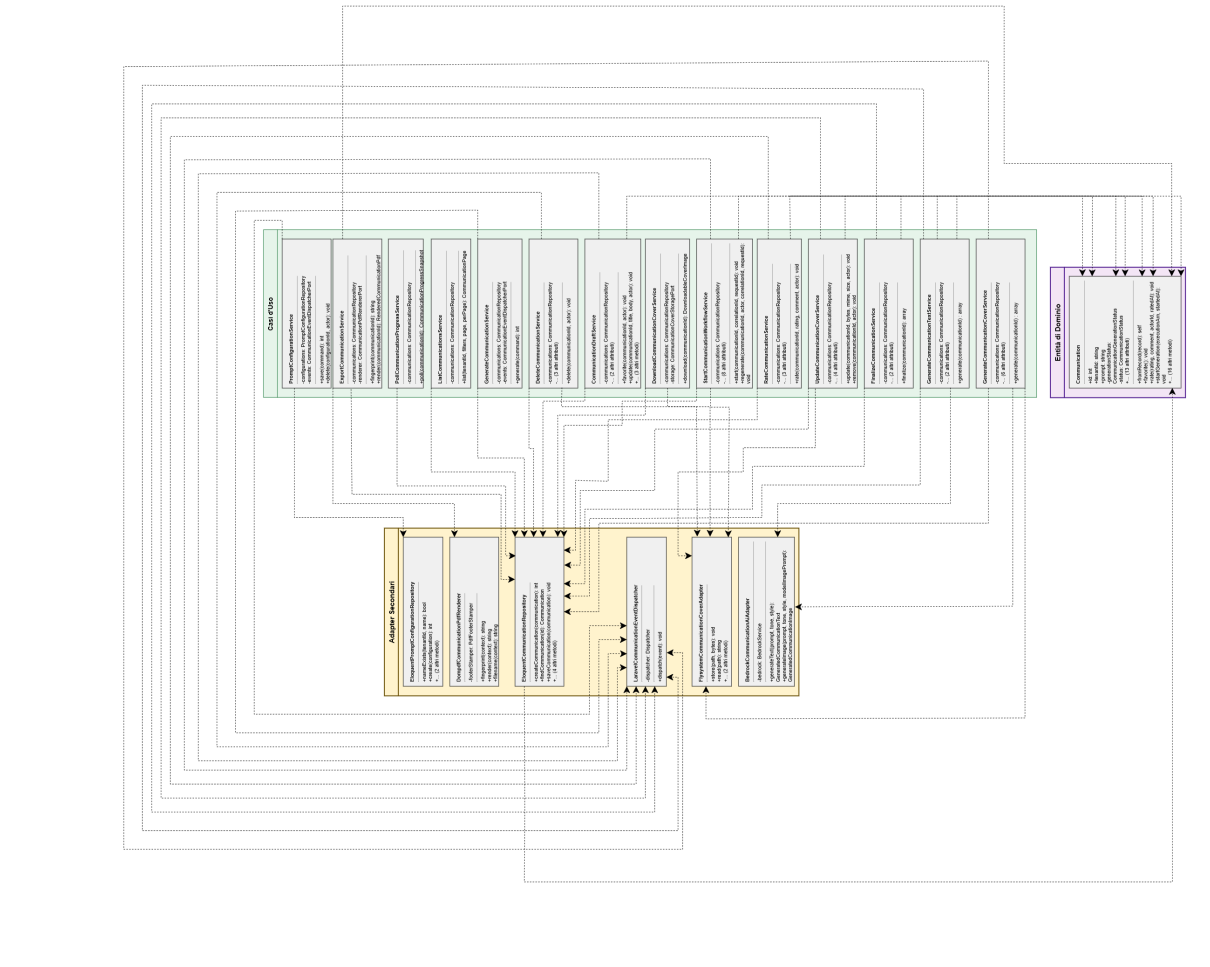


Figura 39: Diagramma delle classi: Modulo Communications

5.2.1 Diagrammi delle classi parziali del modulo Communications

I diagrammi seguenti mostrano porzioni del diagramma delle classi complessivo, raggruppate per flusso applicativo, sullo stesso principio già applicato al modulo

Documents.

Il primo diagramma mostra **CommunicationController**, l'adapter primario che riceve la maggior parte delle richieste HTTP del modulo e le smista verso i cinque casi d'uso corrispondenti.

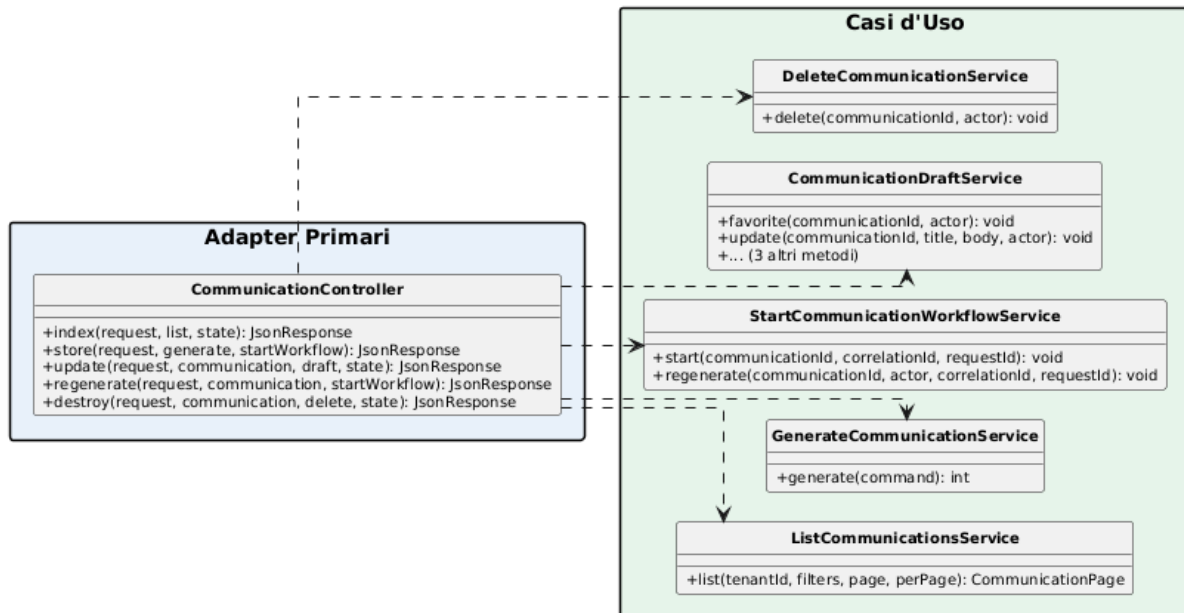


Figura 40: Diagramma classi parziale: CommunicationController

Il secondo diagramma rappresenta il flusso di generazione e avvio della pipeline: **GenerateCommunicationService** crea la comunicazione, mentre **StartCommunicationWorkflowService** avvia l'esecuzione Step Functions tramite **SfnWorkflowEngineAdapter** (componente condiviso del modulo Workflow, non specifico di Communications) ed elimina l'eventuale copertina precedente in caso di rigenerazione.

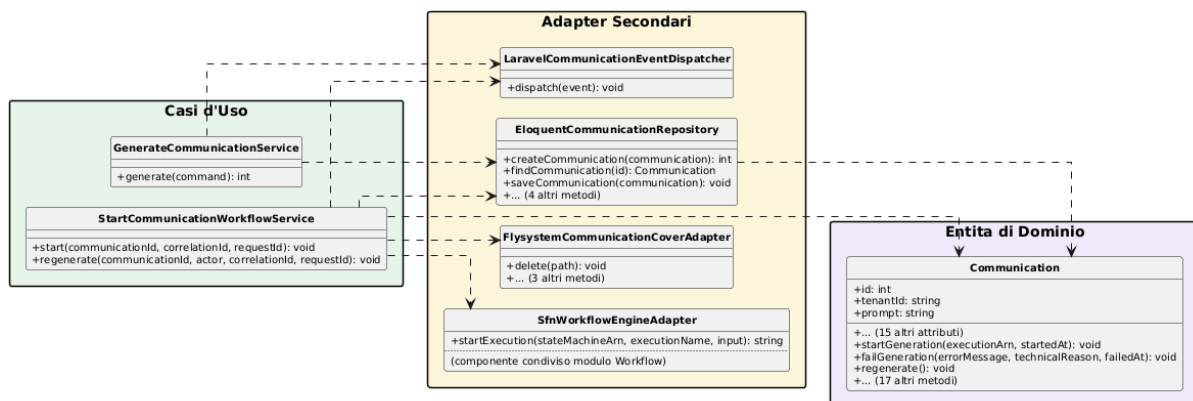


Figura 41: Diagramma classi parziale: Flusso di generazione e avvio

Il terzo diagramma evidenzia la catena di generazione asincrona orchestrata da **CommunicationWorkflowTaskHandler**: generazione del testo, generazione della copertina ed aggiornamento finale dello stato, entrambe le generazioni tramite Bedrock.

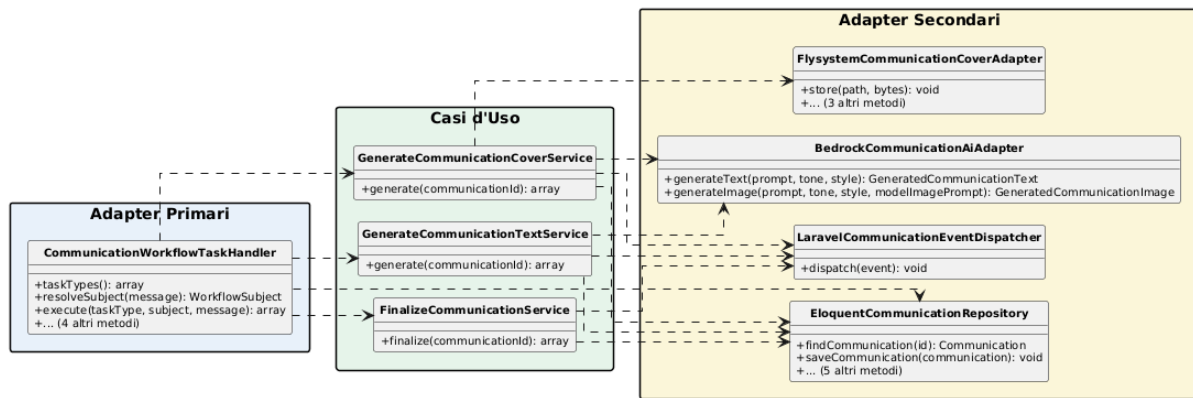


Figura 42: Diagramma classi parziale: Flusso di generazione asincrona

Il quarto diagramma mostra il percorso di copertina, esportazione e valutazione: i tre adapter primari dedicati (CommunicationCoverController, CommunicationExportController, CommunicationRatingController) delegano rispettivamente a UpdateCommunicationCoverService/DownloadCommunicationCoverService, ExportCommunicationService e RateCommunicationService.

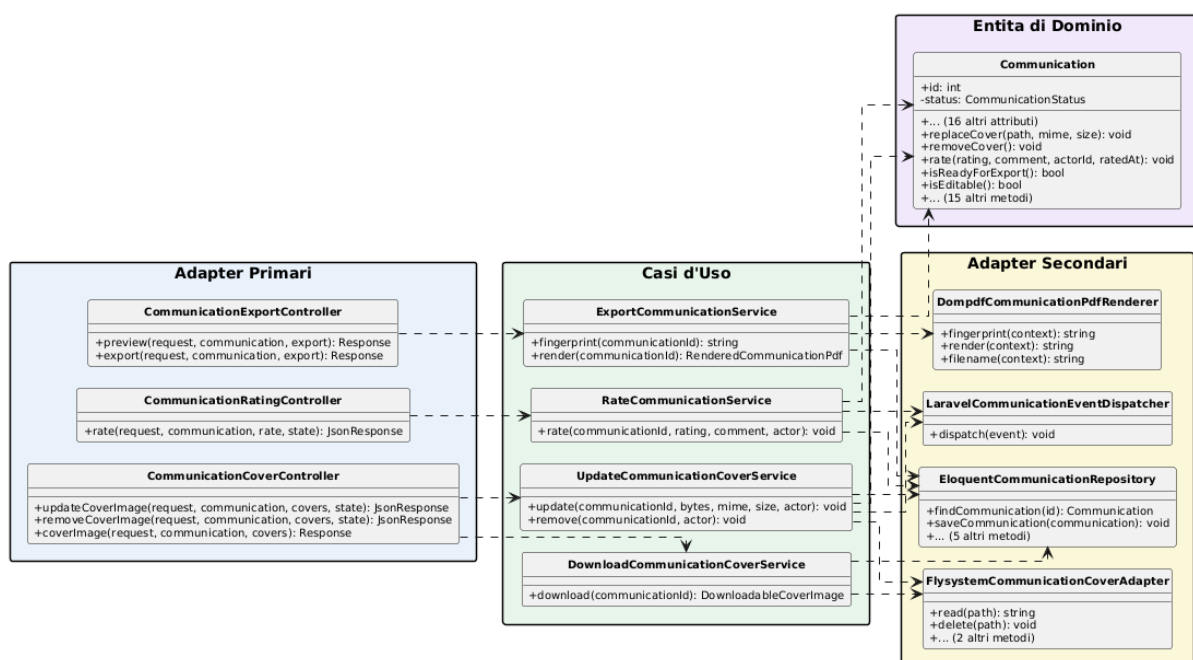


Figura 43: Diagramma classi parziale: Flusso di copertina, esportazione e valutazione

Il quinto diagramma completa il quadro con il flusso di consultazione, streaming ed eliminazione: ListCommunicationsService e PollCommunicationProgressService leggono lo stato tramite EloquentCommunicationRepository, mentre DeleteCommunicationService ne orchestra anche la cancellazione della copertina e la pubblicazione dell'evento di dominio.

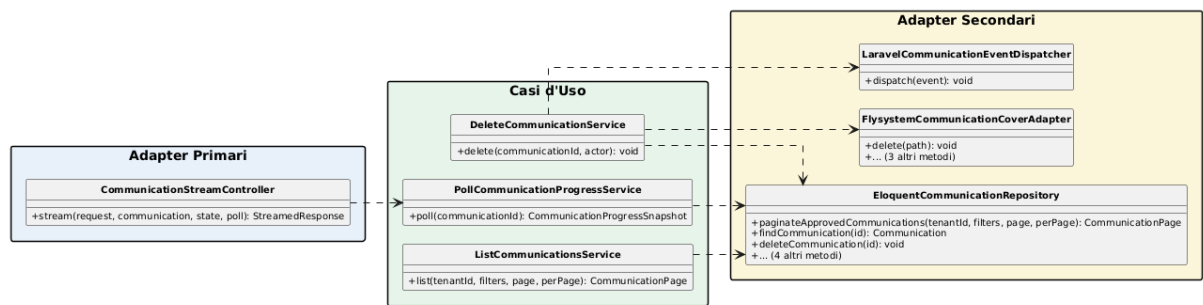


Figura 44: Diagramma classi parziale: Flusso di consultazione ed eliminazione

Il sesto diagramma isola la gestione delle configurazioni di prompt salvate, un aggregato distinto da **Communication** con un proprio repository dedicato.

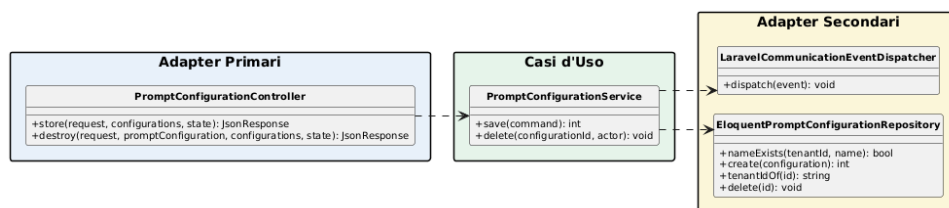


Figura 45: Diagramma classi parziale: Configurazioni di prompt

5.2.2 Diagramma di sequenza: generazione di una comunicazione

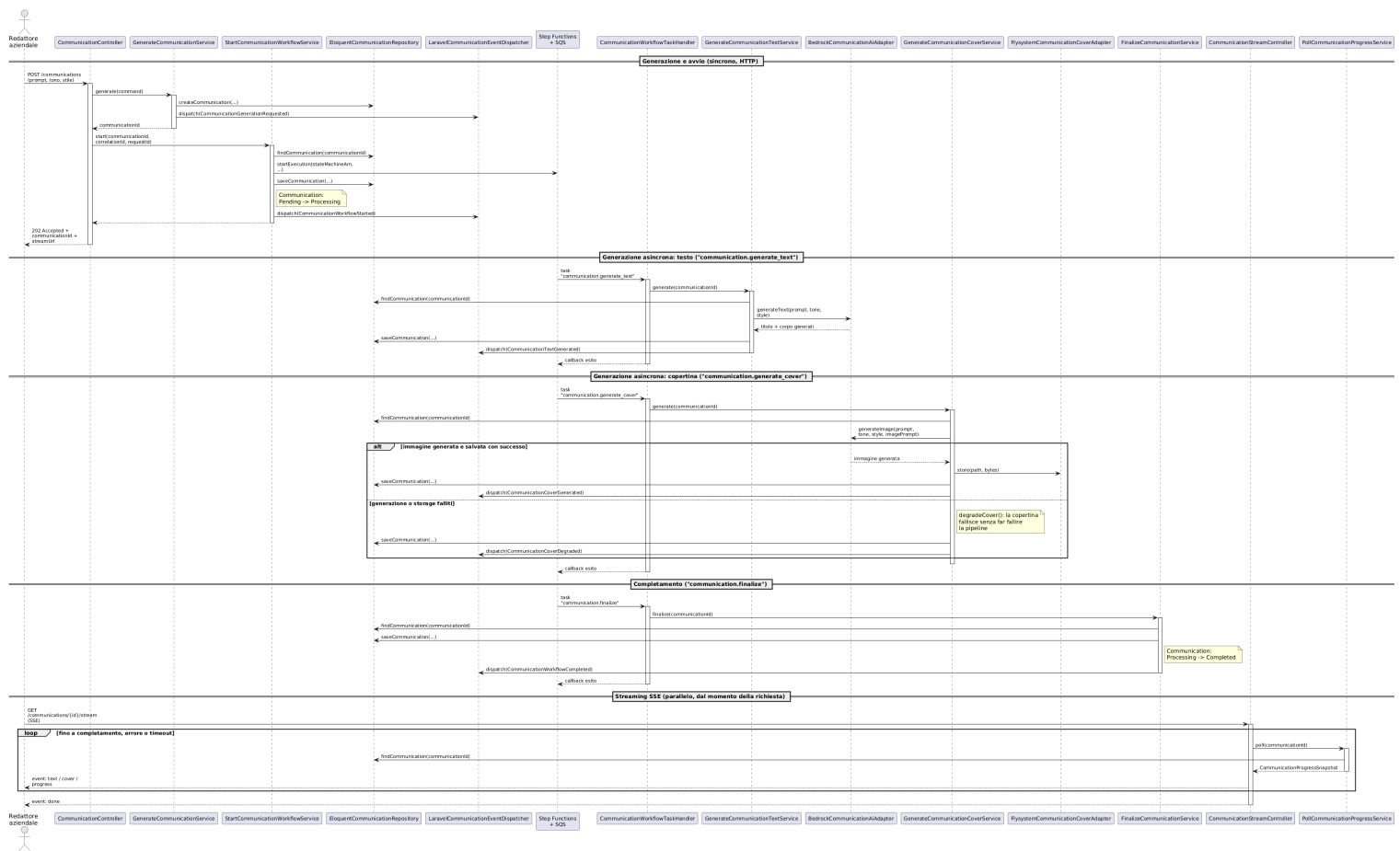


Figura 46: Diagramma di sequenza: Generazione di una comunicazione

Il diagramma descrive la sequenza delle operazioni che avvengono quando un redattore richiede la generazione di una comunicazione tramite l'endpoint dedicato. Il `CommunicationController` riceve prompt, tono e stile, delega la creazione della bozza a `GenerateCommunicationService` e, subito dopo, l'avvio della pipeline a `StartCommunicationWorkflowUseCase`; la risposta HTTP torna al client con l'identificativo della comunicazione e l'URL di streaming, senza attendere la generazione.

Da quel momento la sequenza prosegue in modo asincrono. Il Worker riceve dalla state machine Step Functions un task alla volta: prima la generazione del testo, poi quella della copertina a partire dal prompt visivo prodotto dal passo precedente (un passo che può fallire senza invalidare la comunicazione, degradando semplicemente lo stato della copertina), infine la pubblicazione dell'evento di completamento.

In termini di classi, quei tre passi passano rispettivamente per `CommunicationWorkflowTaskHandler`, che delega a `GenerateCommunicationTextService`, `GenerateCommunicationCoverService` e `FinalizeCommunicationUseCase`. Ogni passo aggiorna lo stato dell'entità `Communication` tramite `EloquentCommunicationRepository`.

In parallelo, la SPA resta connessa all'endpoint SSE esposto da `CommunicationStreamController`, che interroga `PollCommunicationProgressUseCase` a ogni iterazione e inoltra al client l'avanzamento di testo e copertina, fino all'evento finale di completamento o di errore.

5.2.3 Tutte le classi del modulo Communications

Dopo aver mostrato i diagrammi delle classi e di sequenza, si riportano di seguito le descrizioni delle classi principali del modulo Communications (adapter primari, entità di dominio, casi d'uso applicativi e adapter secondari), con i dettagli di attributi e metodi per ciascuna. Eventi, listener, value object, enum ed eccezioni sono citati nel contesto delle classi che li usano, non descritti separatamente.

5.2.3.1 CommunicationController

Adapter primario HTTP del dominio: raccoglie le operazioni sul ciclo di vita della bozza (creazione, modifica, rigenerazione, salvataggio, scarto, eliminazione, preferiti) che nel dominio Documents sono invece distribuite su più Controller specializzati, perché qui condividono lo stesso aggregato e la stessa autorizzazione di base.

Le operazioni più specifiche (copertina, esportazione, valutazione, streaming SSE) restano su Controller dedicati (`CommunicationCoverController`, `CommunicationExportController`, `CommunicationRatingController`, `CommunicationStreamController`), sullo stesso principio di traduzione pura verso una porta primaria, senza regole di business proprie.

CommunicationController
<pre>+index(request, list, state): JsonResponse +store(request, generate, startWorkflow): JsonResponse +update(request, communication, draft, state): JsonResponse +favorite(request, communication, draft, state): JsonResponse +unfavorite(request, communication, draft, state): JsonResponse +regenerate(request, communication, startWorkflow): JsonResponse +save(request, communication, draft, state): JsonResponse +discard(request, communication, draft, state): JsonResponse +destroy(request, communication, delete, state): JsonResponse</pre>

Figura 47: Classe: CommunicationController

Attributi

- Nessuno.

Metodi

- `index(request: ListCommunicationsRequest, list: ListCommunicationsUseCase, state: MvpStateService): JsonResponse`
- `store(request: GenerateCommunicationRequest, generate: GenerateCommunicationUseCase, startWorkflow: StartCommunicationWorkflowUseCase): JsonResponse`

- `update(request: UpdateCommunicationRequest, communication: Communication, draft: CommunicationDraftUseCase, state: MvpStateService): JsonResponse`
- `favorite(request: Request, communication: Communication, draft: CommunicationDraftUseCase, state: MvpStateService): JsonResponse`
- `unfavorite(request: Request, communication: Communication, draft: CommunicationDraftUseCase, state: MvpStateService): JsonResponse`
- `regenerate(request: Request, communication: Communication, startWorkflow: StartCommunicationWorkflowUseCase): JsonResponse`
- `save(request: Request, communication: Communication, draft: CommunicationDraftUseCase, state: MvpStateService): JsonResponse`
- `discard(request: Request, communication: Communication, draft: CommunicationDraftUseCase, state: MvpStateService): JsonResponse`
- `destroy(request: Request, communication: Communication, delete: DeleteCommunicationUseCase, state: MvpStateService): JsonResponse`

5.2.3.2 CommunicationCoverController

Adapter primario HTTP dedicato all'immagine di copertina della comunicazione: sostituzione manuale, rimozione e download. L'audit della mutazione passa dagli eventi `CommunicationCoverReplaced/CommunicationCoverRemoved` dispatchati da `UpdateCommunicationCoverService`, non scritto qui direttamente. Il download passa dalla porta primaria `DownloadCommunicationCoverUseCase`, stesso schema di `DocumentPreviewController` sul dominio `Documents`.

CommunicationCoverController
+updateCoverImage(request, communication, covers, state): JsonResponse +removeCoverImage(request, communication, covers, state): JsonResponse +coverImage(request, communication, covers): Response

Figura 48: Classe: CommunicationCoverController

Attributi

- Nessuno.

Metodi

- `updateCoverImage(request: UpdateCommunicationCoverRequest, communication: Communication, covers: UpdateCommunicationCoverUseCase, state: MvpStateService): JsonResponse`

- `removeCoverImage(request: Request, communication: Communication, covers: UpdateCommunicationCoverUseCase, state: MvpStateService): JsonResponse`
- `coverImage(request: Request, communication: Communication, covers: DownloadCommunicationCoverUseCase): Response`

5.2.3.3 CommunicationExportController

Adapter primario HTTP per il documento finale impaginato: anteprima inline ed esportazione come allegato. Entrambi i metodi condividono la stessa logica privata di risposta PDF, che usa l'impronta (`fingerprint()`) come ETag per evitare di rigenerare il documento se il client possiede già l'ultima versione.



Figura 49: Classe: CommunicationExportController

Attributi

- Nessuno.

Metodi

- `preview(request: Request, communication: Communication, export: ExportCommunicationUseCase): Response`
- `export(request: Request, communication: Communication, export: ExportCommunicationUseCase): Response`

5.2.3.4 CommunicationRatingController

Adapter primario HTTP per la valutazione a stelle (1-5) con commento opzionale, assegnabile una sola volta per l'intera vita della bozza: traduce la richiesta verso `RateCommunicationUseCase` senza regole di business proprie.



Figura 50: Classe: CommunicationRatingController

Attributi

- Nessuno.

Metodi

- `rate(request: RateCommunicationRequest, communication: Communication, rate: RateCommunicationUseCase, state: MvpStateService): JsonResponse`

5.2.3.5 CommunicationStreamController

Adapter primario HTTP dedicato allo stream SSE dell'avanzamento della generazione: legge lo stato tramite `PollCommunicationProgressUseCase` (porta primaria), restando responsabile solo del protocollo SSE e dello shaping via `MvpStateService`, ricaricando dal modello Eloquent solo quando deve emettere gli eventi *cover/done* che richiedono l'intero aggregato.

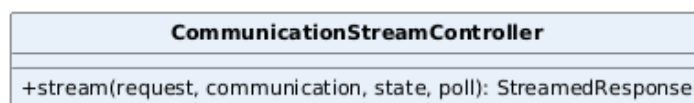


Figura 51: Classe: CommunicationStreamController

Attributi

- Nessuno.

Metodi

- `stream(request: Request, communication: Communication, state: MvpStateService, poll: PollCommunicationProgressUseCase): StreamedResponse`

5.2.3.6 CommunicationWorkflowTaskHandler

Adapter primario guidato da Step Functions/SQS, equivalente concettuale del `DocumentWorkflowTaskHandler` sul dominio gemello: traduce i task di callback della pipeline di generazione in chiamate ai casi d'uso di dominio tramite le loro porte primarie, senza contenere regole di business proprie.

Per costruire un messaggio di ripiego usa direttamente il messaggio dell'eccezione, stesso comportamento del suo equivalente sul dominio Documents.

CommunicationWorkflowTaskHandler
-generateText: GenerateCommunicationTextUseCase -generateCover: GenerateCommunicationCoverUseCase -finalize: FinalizeCommunicationUseCase -communications: CommunicationRepository -clock: ClockInterface
+taskTypes(): array +subjectType(): string +pipeline(): string +auditEventPrefix(): string +resolveSubject(message: array): WorkflowSubject +execute(taskType, subject, message): array +onFailure(subject, taskType, e): void

Figura 52: Classe: CommunicationWorkflowTaskHandler

Attributi

- generateText: GenerateCommunicationTextUseCase, generateCover: GenerateCommunicationCoverUseCase, finalize: FinalizeCommunicationUseCase, communications: CommunicationRepository, clock: ClockInterface.

Metodi

- taskTypes(): array
- subjectType(): string
- pipeline(): string: restituisce l'identificativo della pipeline di appartenenza ("communications").
- auditEventPrefix(): string: prefisso ("mvp-communication-workflow-task-") usato dal runner condiviso per nominare gli eventi di audit di questo dominio.
- resolveSubject(message: array): WorkflowSubject
- execute(taskType: string, subject: WorkflowSubject, message: array): array
- onFailure(subject: WorkflowSubject, taskType: string, e: Throwable): void

5.2.3.7 Communication

Entità di dominio che governa l'intero ciclo di vita di una bozza di comunicazione, ed è la più estesa dei due domini perché accumula responsabilità distinte (preferiti, approvazione e scarto della bozza, valutazione a stelle, copertina manuale o generata dall'AI, l'intera pipeline di generazione fino alla rigenerazione) invece delle una o due tipiche di SubDocument e OriginalDocument.

Verifica anche un'invariante propria: una copertina non può mai essere applicata prima che il testo sia stato generato, controllata ogni volta che l'entità riceve l'esito della generazione AI. Come le altre due entità, i metodi che rappresentano un'azione dell'utente verificano la propria guardia (una comunicazione già scartata non può essere approvata, una valutazione già assegnata non può essere ripetuta), mentre quelli che registrano l'esito della pipeline si limitano ad applicarlo e accumula il proprio delta in un oggetto di modifiche interno, letto una sola volta dall'adapter di persistenza.

Communication
+id: int +tenantId: string +prompt: string +tone: string +style: string -generatedTitle: ?string -generatedBody: ?string -imagePrompt: ?string -generationStatus: CommunicationGenerationStatus -coverImagePath: ?string -coverImageMime: ?string -coverStatus: CoverImageStatus -status: CommunicationStatus -isFavorite: bool -rating: ?int -workflowExecutionArn: ?string -coverError: ?string -errorMessage: ?string
+fromRecord(record: CommunicationRecord): self +favorite(): void +unfavorite(): void +updateDraft(title: string, body: string): void +approve(): void +discard(): void +rate(rating, comment, actorId, ratedAt): void +replaceCover(path, mime, size): void +removeCover(): void +applyGeneratedText(generated: GeneratedCommunicationText): void +applyGeneratedCover(image: GeneratedCommunicationImage, path: string): void +degradeCover(warning: string): void +startGeneration(executionArn: string, startedAt: DateTimeImmutable): void +failGeneration(errorMessage, technicalReason, failedAt): void +completeGeneration(completedAt: DateTimeImmutable): void +regenerate(): void +hasGeneratedText(): bool +isReadyForExport(): bool +isEditable(): bool +pendingChanges(): CommunicationChanges

Figura 53: Classe: Communication

Attributi

- **id**, **tenantId**, i dati della richiesta, l'output della generazione, gli stati di generazione/copertina/bozza, **isFavorite**, **rating**, i riferimenti all'esecuzione Step Functions ed eventuali errori.

Metodi

- **fromRecord(record: CommunicationRecord): self**
- **favorite(): void, unfavorite(): void**

- `updateDraft(title: string, body: string): void, approve(): void, discard(): void`
- `rate(rating: int, comment: ?string, actorId: string, ratedAt: DateTimeImmutable): void`
- `replaceCover(path: string, mime: string, size: int): void, removeCover(): void`
- `applyGeneratedText(generated: GeneratedCommunicationText): void, applyGeneratedCover(image: GeneratedCommunicationImage, path: string): void, degradeCover(warning: string): void`
- `startGeneration(executionArn: string, startedAt: DateTimeImmutable): void, failGeneration(errorMessage: string, technicalReason: ?string, failedAt: DateTimeImmutable): void, completeGeneration(completedAt: DateTimeImmutable): void`
- `regenerate(): void`
- `hasGeneratedText(): bool`: l'invariante testo-prima-di-copertina descritta sopra: usato da `GenerateCommunicationCoverService` come guardia prima di applicare e persistere la copertina, quindi dopo la chiamata al modello generativo, non prima.
- `isReadyForExport(): bool, isEditable(): bool`
- `pendingChanges(): CommunicationChanges`

5.2.3.8 GenerateCommunicationService

Implementa `GenerateCommunicationUseCase` ed è il punto di ingresso della pipeline di generazione: crea la comunicazione con stato *Draft* e generazione *Pending* a partire da prompt, tono e stile, e restituisce l'identificativo appena creato. Come il suo equivalente sul dominio Documents, non avvia da sé la pipeline: quella responsabilità è delegata a `StartCommunicationWorkflowUseCase`, invocato subito dopo dallo stesso Controller.

L'audit passa dal dispatch dell'evento `CommunicationGenerationRequested`, non da un `AuditLogger` diretto, stesso principio già applicato a `UploadDocumentService` sul dominio Documents.

GenerateCommunicationService
-communications: CommunicationRepository
-events: CommunicationEventDispatcherPort
+generate(command: GenerateCommunicationCommand): int

Figura 54: Classe: GenerateCommunicationService

Attributi

- `communications: CommunicationRepository, events: CommunicationEventDispatcherPort.`

Metodi

- `generate(command: GenerateCommunicationCommand): int`

5.2.3.9 StartCommunicationWorkflowService

Implementa `StartCommunicationWorkflowUseCase`: avvia l'esecuzione Step Functions della pipeline di generazione, equivalente concettuale di `StartDocumentWorkflowService` sul dominio Documents, con la stessa idempotenza verso un'esecuzione già in corso: il controllo "è già in corso?" e la scrittura che lo rende vero girano dentro un'unica transazione, con lettura tramite `CommunicationRepository::findCommunicationForUpdate()` (lock pessimistico sulla riga): senza, due richieste quasi simultanee potrebbero superare entrambe il controllo e avviare due esecuzioni per la stessa comunicazione. Gli eventi di dominio (avvio riuscito o fallito) sono dispatchati solo dopo che la transazione ha commesso, stesso schema di `CommunicationDraftService::favorite()/unfavorite()`. Espone anche `regenerate()`: verifica il tenant dell'attore (`CommunicationNotAuthorizedException` altrimenti), riporta la comunicazione allo stato di rigenerazione, elimina l'eventuale copertina precedente dallo storage e riavvia la pipeline richiamando internamente `start()`, così che prima generazione e rigenerazione condividano sempre la stessa logica di avvio. Non ha bisogno di un lock proprio: nessun controllo di stato concorrente da proteggere qui, il lock su `start()` (chiamato subito dopo) resta la difesa contro un doppio avvio.

Stesso trattamento della configurazione del suo equivalente Documents: ARN della state machine e URL della coda sono risolti una volta in `AppServiceProvider` e iniettati come scalari, non letti tramite `config()` internamente: stessa istanziabilità in un test Pest puro, senza avviare Laravel.

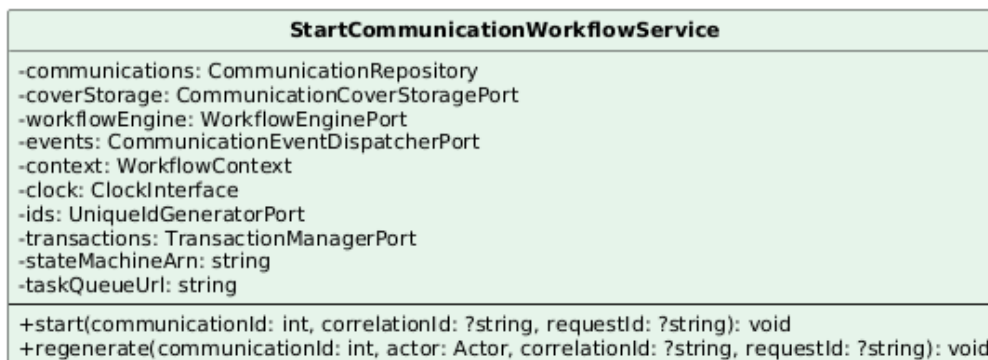


Figura 55: Classe: StartCommunicationWorkflowService

Attributi

- `communications: CommunicationRepository, coverStorage: CommunicationCoverStoragePort, workflowEngine: WorkflowEnginePort, events: CommunicationEventDispatcherPort, context: WorkflowContext, clock: ClockInterface, ids: UniqueIdGeneratorPort, transactions: TransactionManagerPort.`

- `stateMachineArn: string, taskQueueUrl: string`: scalari risolti da configurazione in `AppServiceProvider`.

Metodi

- `start(communicationId: int, correlationId: ?string, requestId: ?string): void`
- `regenerate(communicationId: int, actor: Actor, correlationId: ?string, requestId: ?string): void`

5.2.3.10 `GenerateCommunicationTextService`, `GenerateCommunicationCoverService`

Le due classi che orchestrano la generazione vera e propria, invocate dal Worker durante la pipeline asincrona: la prima genera titolo e corpo del testo tramite Bedrock, la seconda l'immagine di copertina a partire dal prompt visivo prodotto dalla prima. Sono rimaste distinte invece di essere unificate in un unico servizio, perché rappresentano due passi della state machine Step Functions con criticità diverse: se il testo non viene generato la bozza è fallita, mentre una copertina non disponibile viene degradata e segnalata, lasciando la comunicazione comunque valida.

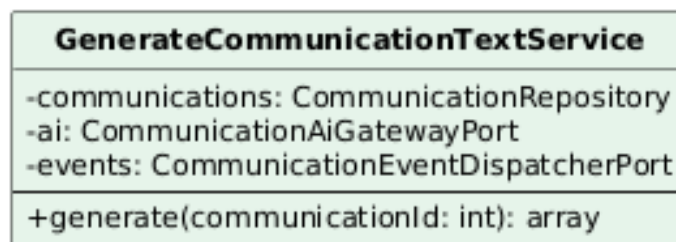


Figura 56: Classe: `GenerateCommunicationTextService`

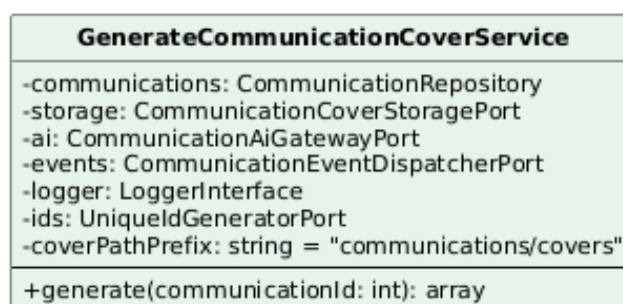


Figura 57: Classe: `GenerateCommunicationCoverService`

Attributi

- `GenerateCommunicationTextService`: `communications: CommunicationRepository`, `ai: CommunicationAiGatewayPort`, `events: CommunicationEventDispatcherPort`.

- `GenerateCommunicationCoverService`: gli stessi tre più `storage`:
`CommunicationCoverStoragePort`, `logger`: `LoggerInterface`, `ids`:
`UniqueIdGeneratorPort`, `coverPathPrefix`: `string` (prefisso del percorso di
storage, default "communications/covers").

Metodi

- `generate(communicationId: int): array`

5.2.3.11 FinalizeCommunicationService

Implementa `FinalizeCommunicationUseCase`, equivalente concettuale di `FinalizeDocumentWorkflowService`: chiude la pipeline di generazione, marcando la comunicazione come completata e pubblicando l'evento di completamento osservato dallo stream SSE. È idempotente verso la riconsegna del task: una comunicazione già completata non ripubblica l'evento.

Se la copertina è rimasta bloccata in *Pending/Processing* (il ramo di degrado della state machine può saltare il task in caso di timeout o worker caduto), questo caso d'uso la degrada esplicitamente a *Failed* prima di chiudere la pipeline, così la SPA non resta in attesa indefinita.

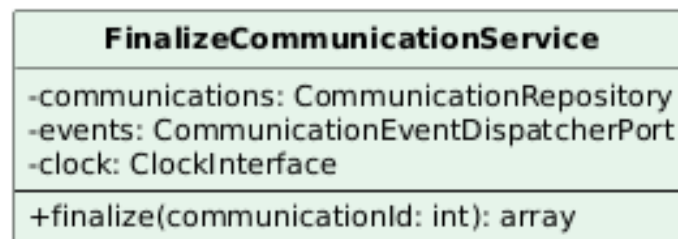


Figura 58: Classe: FinalizeCommunicationService

Attributi

- `communications`: `CommunicationRepository`, `events`:
`CommunicationEventDispatcherPort`, `clock`: `ClockInterface`.

Metodi

- `finalize(communicationId: int): array`

5.2.3.12 PollCommunicationProgressService

Implementa `PollCommunicationProgressUseCase`, equivalente concettuale di `PollDocumentProgressService`: legge un'istantanea dell'avanzamento (stato di generazione, stato copertina, testo generato finora, eventuali errori di copertina o di generazione), consumata da `CommunicationStreamController` a ogni iterazione dello stream SSE.

PollCommunicationProgressService
-communications: CommunicationRepository
+poll(communicationId: int): CommunicationProgressSnapshot

Figura 59: Classe: PollCommunicationProgressService

Attributi

- communications: CommunicationRepository.

Metodi

- poll(communicationId: int): CommunicationProgressSnapshot

5.2.3.13 CommunicationDraftService

Implementa **CommunicationDraftUseCase**: raccoglie le operazioni sul ciclo di vita editoriale della bozza (preferiti, modifica, approvazione, scarto) dietro cui il **CommunicationController** delega senza contenere logica propria. Ogni metodo segue lo stesso schema: recupera l'entità con lock pessimistico, verifica che il tenant dell'attore coincida con quello della comunicazione (**CommunicationNotAuthorizedException** altrimenti; difesa in profondità: il controllo HTTP già protegge la rotta, ma il caso d'uso non si affida solo a chi lo chiama), invoca la transizione corrispondente (che verifica da sé le proprie guardie), persiste e pubblica l'evento di dominio associato.

Tutti e cinque i metodi racchiudono lettura-per-aggiornamento e scrittura in un'unica transazione tramite **TransactionManagerPort**: senza il lock, un **save()** e un **discard()** quasi simultanei sulla stessa bozza leggerebbero entrambi lo stato prima che l'altro scriva, supererebbero entrambi il proprio controllo in **Communication**, e l'ultimo a scrivere vincerebbe: una bozza appena scartata potrebbe tornare "approvata" per puro timing, bypassando l'invariante di dominio.

CommunicationDraftService
-communications: CommunicationRepository -events: CommunicationEventDispatcherPort -transactions: TransactionManagerPort
+favorite(communicationId: int, actor: Actor): void +unfavorite(communicationId: int, actor: Actor): void +update(communicationId: int, title: string, body: string, actor: Actor): void +save(communicationId: int, actor: Actor): void +discard(communicationId: int, actor: Actor): void

Figura 60: Classe: CommunicationDraftService

Attributi

- communications: CommunicationRepository, events: CommunicationEventDispatcherPort, transactions: TransactionManagerPort.

Metodi

- `favorite(communicationId: int, actor: Actor): void`,
`unfavorite(communicationId: int, actor: Actor): void`
- `update(communicationId: int, title: string, body: string, actor: Actor): void`
- `save(communicationId: int, actor: Actor): void`,
`discard(communicationId: int, actor: Actor): void`

5.2.3.14 RateCommunicationService

Implementa `RateCommunicationUseCase`: registra la valutazione a stelle di una comunicazione insieme a un commento opzionale, normalizzando un commento composto solo da spazi bianchi a null prima di passarlo all'entità. Lettura-per-aggiornamento, verifica del tenant dell'attore (`CommunicationNotAuthorizedException` altrimenti, difesa in profondità dietro il controllo già fatto a livello HTTP), mutazione e salvataggio avvengono dentro una transazione (`TransactionManagerPort`), stessa cautela di `CommunicationDraftService::favorite()/unfavorite()` contro valutazioni concorrenti sulla stessa comunicazione.

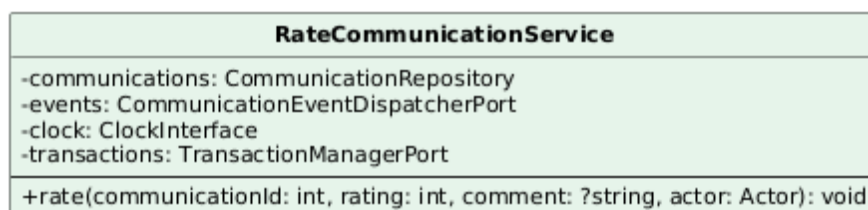


Figura 61: Classe: `RateCommunicationService`

Attributi

- `communications: CommunicationRepository`, `events: CommunicationEventDispatcherPort`, `clock: ClockInterface`,
`transactions: TransactionManagerPort`.

Metodi

- `rate(communicationId: int, rating: int, comment: ?string, actor: Actor): void`

5.2.3.15 UpdateCommunicationCoverService

Implementa `UpdateCommunicationCoverUseCase`: gestisce la sostituzione manuale della copertina da parte dell'utente, in alternativa a quella generata dall'AI. Entrambi i metodi verificano per primi che il tenant dell'attore coincida con quello della comunicazione (`CommunicationNotAuthorizedException` altrimenti, difesa in profondità dietro il controllo già fatto a livello HTTP). `update()` rifiuta poi l'operazione solo se la bozza è già

scartata (`Communication::isEditable()`): una bozza approvata resta modificabile anche dopo il salvataggio nello storico, come previsto dal perimetro MVP: testo, rigenerazione e copertina restano tutti ammessi finché la bozza non è scartata.

Genera un percorso univoco per ogni nuova copertina caricata e rimuove quella precedente dallo storage solo dopo che la nuova è stata salvata con successo, per non lasciare la comunicazione temporaneamente senza copertina in caso di errore a metà operazione.

UpdateCommunicationCoverService
-communications: CommunicationRepository -storage: CommunicationCoverStoragePort -events: CommunicationEventDispatcherPort -ids: UniqueIdGeneratorPort -coverPathPrefix: string = "communications/covers"
+update(communicationId: int, bytes: string, mime: string, size: int, actor: Actor): void +remove(communicationId: int, actor: Actor): void

Figura 62: Classe: UpdateCommunicationCoverService

Attributi

- `communications`: CommunicationRepository, `storage`: CommunicationCoverStoragePort.
- `events`: CommunicationEventDispatcherPort, `ids`: UniqueIdGeneratorPort, `coverPathPrefix`: string (default communications/covers).

Metodi

- `update(communicationId: int, bytes: string, mime: string, size: int, actor: Actor): void`
- `remove(communicationId: int, actor: Actor): void`

5.2.3.16 DownloadCommunicationCoverService, ExportCommunicationService

Due casi d'uso di lettura per l'output finale di una comunicazione. `DownloadCommunicationCoverService` implementa `DownloadCommunicationCoverUseCase` e restituisce i byte grezzi della sola immagine di copertina.

`ExportCommunicationService` implementa `ExportCommunicationUseCase` e produce il PDF completo della comunicazione tramite `CommunicationPdfRendererPort`, rifiutando l'operazione se la bozza non è ancora pronta per l'esportazione (`isReadyForExport()`); espone anche un'impronta (`fingerprint()`) usata dall'adapter di rendering per la cache su disco, senza che il caso d'uso sappia che quella cache esiste.

DownloadCommunicationCoverService
-communications: CommunicationRepository -storage: CommunicationCoverStoragePort
+download(communicationId: int): DownloadableCoverImage

Figura 63: Classe: DownloadCommunicationCoverService

ExportCommunicationService
-communications: CommunicationRepository -renderer: CommunicationPdfRendererPort
+fingerprint(communicationId: int): string +render(communicationId: int): RenderedCommunicationPdf

Figura 64: Classe: ExportCommunicationService

Attributi

- DownloadCommunicationCoverService: communications: CommunicationRepository, storage: CommunicationCoverStoragePort.
- ExportCommunicationService: communications: CommunicationRepository, renderer: CommunicationPdfRendererPort.

Metodi

- download(communicationId: int): DownloadableCoverImage
- fingerprint(communicationId: int): string, render(communicationId: int): RenderedCommunicationPdf

5.2.3.17 ListCommunicationsService, DeleteCommunicationService

Due casi d'uso minimi, equivalenti concettuali di ListDocumentsService e DeleteDocumentService. ListCommunicationsService implementa ListCommunicationsUseCase e inoltra la paginazione al repository.

DeleteCommunicationService implementa DeleteCommunicationUseCase: verifica che il tenant dell'attore coincida con quello della comunicazione (CommunicationNotAuthorizedException altrimenti, difesa in profondità dietro il controllo già fatto a livello HTTP), poi elimina la comunicazione e, se presente, la sua copertina dallo storage: con lo stesso trattamento delle eliminazioni sul dominio Documents, un guasto nella pulizia dello storage viene solo loggato, non propagato, perché il record è già rimosso dal punto di vista dell'utente.



Figura 65: Classe: ListCommunicationsService

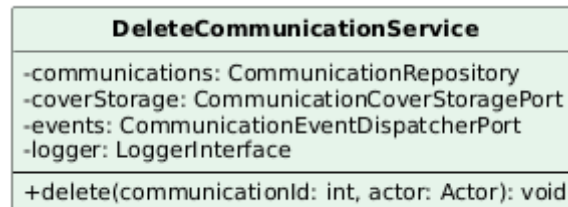


Figura 66: Classe: DeleteCommunicationService

Attributi

- ListCommunicationsService: communications: CommunicationRepository.
- DeleteCommunicationService: communications: CommunicationRepository, coverStorage: CommunicationCoverStoragePort, events: CommunicationEventDispatcherPort, logger: LoggerInterface.

Metodi

- list(tenantId: string, filters: CommunicationListFilters, page: int, perPage: int): CommunicationPage
- delete(communicationId: int, actor: Actor): void

5.2.3.18 PromptConfigurationController

Adapter primario HTTP per i preset di prompt riutilizzabili: traduce creazione ed eliminazione verso PromptConfigurationUseCase, senza regole di business proprie: l'unico controllo qui è di trasporto, la verifica di ownership del tenant prima di autorizzare l'eliminazione.

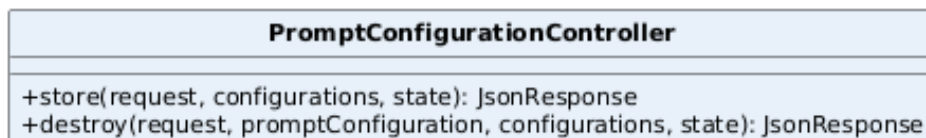


Figura 67: Classe: PromptConfigurationController

Attributi

- Nessuno.

Metodi

- `store(request: SavePromptConfigurationRequest, configurations: PromptConfigurationUseCase, state: MvpStateService): JsonResponse`
- `destroy(request: Request, promptConfiguration: PromptConfiguration, configurations: PromptConfigurationUseCase, state: MvpStateService): JsonResponse`

5.2.3.19 PromptConfigurationService

Implementa `PromptConfigurationUseCase`: gestisce le configurazioni di prompt salvate dall'utente (combinazioni di prompt, tono e stile riutilizzabili). Se il nome richiesto è vuoto o già in uso per il tenant, `save()` non restituisce un errore all'utente ma assegna un'etichetta progressiva ("*Senza nome (1)*", "*(2)*", ...) tramite un resolver privato, scegliendo sempre di lasciar salvare la configurazione piuttosto che bloccare l'utente su un conflitto di nome.

Il nome è protetto anche da un vincolo di unicità a livello di database: `save()` tenta il salvataggio fino a 8 volte, e se l'adapter di persistenza segnala una collisione (`PromptConfigurationNameTakenException`, tradotta da una violazione del vincolo unico) rigenera un nome e riprova. È una difesa contro la corsa fra due richieste concorrenti che il resolver, da solo, non può escludere: solo dopo 8 tentativi falliti l'eccezione risale al chiamante.

`delete()` verifica invece l'autorizzazione: se il tenant della configurazione non coincide con quello dell'attore, solleva `PromptConfigurationNotAuthorizedException`.

PromptConfigurationService
-configurations: PromptConfigurationRepository -events: CommunicationEventDispatcherPort
+save(command: SavePromptConfigurationCommand): int +delete(configurationId: int, actor: Actor): void

Figura 68: Classe: PromptConfigurationService

Attributi

- `configurations: PromptConfigurationRepository, events: CommunicationEventDispatcherPort.`

Metodi

- `save(command: SavePromptConfigurationCommand): int`
- `delete(configurationId: int, actor: Actor): void`

5.2.3.20 EloquentCommunicationRepository

Adapter secondario che implementa `CommunicationRepository` sopra `Eloquent`, equivalente concettuale dell'`EloquentDocumentRepository`: consuma i value object tipizzati del

dominio (`NewCommunication`, `CommunicationChanges`) invece di array grezzi. È l'unico punto di scrittura dell'aggregato `Communication`, condiviso da tutti i casi d'uso che lo creano, lo modificano o lo eliminano.

EloquentCommunicationRepository
<pre> +createCommunication(communication: NewCommunication): int +findCommunication(id: int): Communication +findCommunicationForUpdate(id: int): Communication +updateCommunication(id: int, changes: CommunicationChanges): void +saveCommunication(communication: Communication): void +deleteCommunication(id: int): void +paginateApprovedCommunications(tenantId, filters, page, perPage): CommunicationPage </pre>

Figura 69: Classe: `EloquentCommunicationRepository`

Attributi

- Nessuno.

Metodi

- `createCommunication(communication: NewCommunication): int`
- `findCommunication(id: int): Communication`
- `findCommunicationForUpdate(id: int): Communication`: con `lockForUpdate()`, usato dai casi d'uso che racchiudono lettura e scrittura in una transazione.
- `updateCommunication(id: int, changes: CommunicationChanges): void`
- `saveCommunication(communication: Communication): void`
- `deleteCommunication(id: int): void`
- `paginateApprovedCommunications(tenantId: string, filters: CommunicationListFilters, page: int, perPage: int): CommunicationPage`

5.2.3.21 `EloquentPromptConfigurationRepository`

Adapter secondario che implementa `PromptConfigurationRepository` sopra `Eloquent`, per la persistenza delle configurazioni di prompt salvate dall'utente: un aggregato distinto da `Communication`, con un proprio repository invece di essere annegato in quello principale. `create()` riceve il value object tipizzato `NewPromptConfiguration` invece di un array grezzo, e traduce una violazione del vincolo unico a livello database in `PromptConfigurationNameTakenException`: è questa traduzione che `PromptConfigurationService::save()` intercetta nel proprio ciclo di retry.

EloquentPromptConfigurationRepository
+nameExists(tenantId: string, name: string): bool +create(configuration: NewPromptConfiguration): int +tenantIdOf(id: int): string +delete(id: int): void

Figura 70: Classe: EloquentPromptConfigurationRepository

Attributi

- Nessuno.

Metodi

- `nameExists(tenantId: string, name: string): bool`
- `create(configuration: NewPromptConfiguration): int`
- `tenantIdOf(id: int): string`
- `delete(id: int): void`

5.2.3.22 BedrockCommunicationAiAdapter

Adapter secondario che implementa `CommunicationAiGatewayPort` sopra il servizio condiviso `BedrockService` (Sezione 2.7), equivalente concettuale del `BedrockDocumentAiAdapter` sul dominio Documents: la generazione testuale e quella dell'immagine di copertina restano due chiamate distinte, coerentemente con il fatto che sono due passi separati della pipeline.

BedrockCommunicationAiAdapter
-bedrock: BedrockService
+generateText(prompt: string, tone: string, style: string): GeneratedCommunicationText +generateImage(prompt, tone, style, modelImagePrompt): GeneratedCommunicationImage

Figura 71: Classe: BedrockCommunicationAiAdapter

Attributi

- `bedrock: BedrockService.`

Metodi

- `generateText(prompt: string, tone: string, style: string): GeneratedCommunicationText`
- `generateImage(prompt: string, tone: string, style: string, modelImagePrompt: ?string): GeneratedCommunicationImage`

5.2.3.23 FlysystemCommunicationCoverAdapter, DompdfCommunicationPdfRenderer

Due adapter secondari di supporto, equivalenti concettuali di `FlysystemDocumentStorageAdapter` e `DompdfSendMessageRenderer` sul dominio `Documents`. `FlysystemCommunicationCoverAdapter` implementa `CommunicationCoverStoragePort` sopra `Flysystem`, registrando nel log applicativo i fallimenti di scrittura e di cancellazione; un fallimento in lettura viene invece propagato al chiamante come errore, senza essere registrato.

`DompdfCommunicationPdfRenderer` implementa `CommunicationPdfRendererPort` sopra `Dompdf`: prima di renderizzare calcola un'impronta SHA-1 del contenuto (identificativo della comunicazione, titolo, corpo, stato, percorso e tipo della copertina, più una versione del renderer che permette di invalidare le impronte già materializzate quando il layout cambia) e, se un PDF con la stessa impronta è già stato materializzato su disco, lo restituisce direttamente invece di rigenerarlo; applica inoltre una filigrana diagonale ("*Creato da AI Assistant*") e lo stesso timbro di piè di pagina condiviso con `DompdfSendMessageRenderer`.

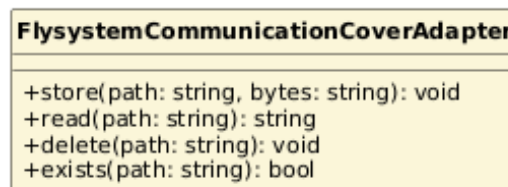


Figura 72: Classe: `FlysystemCommunicationCoverAdapter`

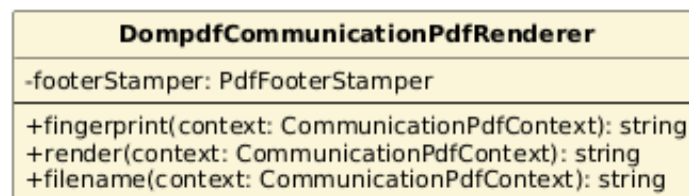


Figura 73: Classe: `DompdfCommunicationPdfRenderer`

Attributi

- `FlysystemCommunicationCoverAdapter`: nessuno, nessun costruttore proprio (usa il disco storage tramite la facade `Storage`).
- `DompdfCommunicationPdfRenderer`: `footerStamper`: `PdfFooterStamper`.

Metodi

- `store(path: string, bytes: string): void`, `read(path: string): string`, `delete(path: string): void`, `exists(path: string): bool`

- `fingerprint(context: CommunicationPdfContext): string`
- `render(context: CommunicationPdfContext): string, filename(context: CommunicationPdfContext): string`

5.2.3.24 LaravelCommunicationEventDispatcher

Adapter secondario minimo, equivalente concettuale del `LaravelDocumentEventDispatcher`: implementa `CommunicationEventDispatcherPort` appoggiandosi al dispatcher di eventi nativo di Laravel. I 20 listener del dominio (su 21 eventi dispatchati: `CommunicationWorkflowCompleted` non ha un listener registrato, simmetricamente a `DocumentWorkflowCompleted`) sono registrati nel service provider.

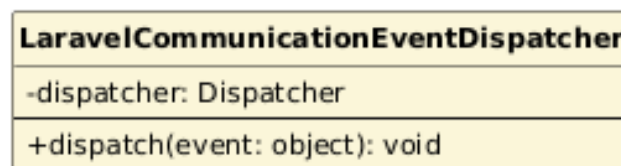


Figura 74: Classe: `LaravelCommunicationEventDispatcher`

Attributi

- `dispatcher: Dispatcher.`

Metodi

- `dispatch(event: object): void`

5.3 Classi condivise

Infrastruttura trasversale ai due domini, non riorganizzata in porte e adapter propri (si veda la Sezione 4.7 per la motivazione del perimetro).

5.3.1 Le principali classi condivise

Si riportano di seguito le descrizioni delle principali classi condivise fra i due domini, con i dettagli di attributi e metodi per ciascuna. Non sono documentate qui le interfacce di porta e i contratti già trattati nelle sezioni dei rispettivi moduli, né le classi di supporto minori.

5.3.1.1 WorkflowTaskRegistry

Instrada un tipo di task (`communication.generate_text`, `document.split`, ...) verso l'handler del dominio competente: ogni `WorkflowTaskHandler` di dominio (Sezioni 5.1 e 5.2) viene registrato qui nel binding del singleton in `AppServiceProvider::register()`, così che `WorkflowTaskRunner` non debba conoscere staticamente quali domini esistono.

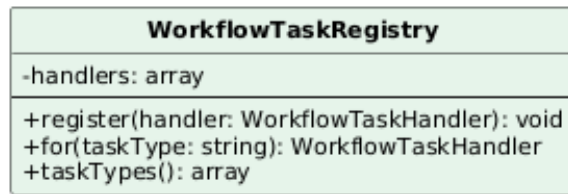


Figura 75: Classe: WorkflowTaskRegistry

Attributi

- **handlers:** array: handler registrati, indicizzati per tipo di task gestito.

Metodi

- **register(handler: WorkflowTaskHandler): void:** registra un handler per i tipi di task che dichiara di gestire.
- **for(taskType: string): WorkflowTaskHandler:** restituisce l'handler registrato per un tipo di task, lanciando un'eccezione se nessuno è registrato.
- **taskTypes(): array:** restituisce l'elenco di tutti i tipi di task registrati.

5.3.1.2 WorkflowTaskRunner

Punto di ingresso generico invocato da `ConsumeWorkflowTasks` per ogni messaggio SQS: deduplica il task tramite il suo token hash, lo prende in carico atomicamente a database, delega l'esecuzione all'handler risolto tramite `WorkflowTaskRegistry` e registra l'audit in modo indipendente dal dominio del task.

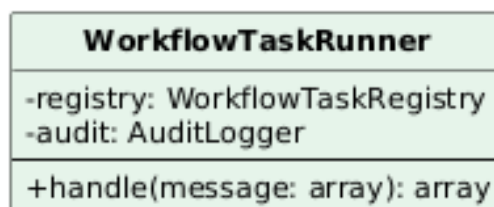


Figura 76: Classe: WorkflowTaskRunner

Attributi

- **registry:** WorkflowTaskRegistry.
- **audit:** AuditLogger.

Metodi

- **handle(message: array): array:** esegue un singolo task a partire dal messaggio SQS decodificato, restituendo l'esito, incluso se e come va inviato il callback verso Step Functions.

5.3.1.3 ConsumeWorkflowTasks

Il comando che il Worker esegue in loop, consumando i messaggi SQS e invocando `WorkflowTaskRunner` per ciascuno. Nel callback di esito a Step Functions distingue gli errori permanenti (`TaskTimedOut`, `TaskDoesNotExist`, `InvalidToken`: il token non è più valido, nessun retry potrebbe riuscire) da quelli transitori (`throttling`, servizio non disponibile): sui primi rimuove comunque il messaggio dalla coda, sui secondi lo lascia per un nuovo tentativo, evitando di popolare la DLQ con falsi fallimenti per un task già concluso a database.

ConsumeWorkflowTasks
-PERMANENT_CALLBACK_ERRORS: array
+handle(sqs, stepFunctions, runner, heartbeat, context): int

Figura 77: Classe: ConsumeWorkflowTasks

Attributi

- `PERMANENT_CALLBACK_ERRORS`: array (costante): codici di errore Step Functions per cui un retry con lo stesso token non può mai riuscire.

Metodi

- `handle(sqs, stepFunctions, runner, heartbeat, context)`: int: ricevuti per dependency injection dal metodo (non dal costruttore, come da convenzione dei comandi Artisan): riceve i messaggi da SQS, delega l'esecuzione a `WorkflowTaskRunner`, invia il callback di esito a Step Functions e cancella il messaggio dalla coda solo a callback confermato.

5.3.1.4 WorkflowTaskHeartbeat

Unico adapter condiviso della porta `WorkflowHeartbeatPort`, invocata dai casi d'uso di lunga durata (split PDF, OCR) per segnalare a Step Functions che l'elaborazione è ancora viva, senza che quei casi d'uso conoscano `SfnClient`.

WorkflowTaskHeartbeat
-stepFunctions: SfnClient -taskToken: string -taskType: string -lastBeatAt: float -degraded: bool
+activate(taskToken: string, taskType: string): void +deactivate(): void +beat(force: bool): void

Figura 78: Classe: WorkflowTaskHeartbeat

Attributi

- `stepFunctions: SfnClient`.
- `taskToken: string`: token del task attivo, impostato da `activate()`.
- `taskType: string`.
- `lastBeatAt: float`: timestamp Unix (`microtime(true)`) dell'ultimo segnale inviato.
- `degraded: bool`.

Metodi

- `activate(taskToken: string, taskType: string): void`: attiva l'heartbeat per il task corrente.
- `deactivate(): void`: disattiva l'heartbeat al termine del task.
- `beat(force: bool): void`: invia un segnale di attività a Step Functions se necessario (o sempre, se `force` è vero).

5.3.1.5 SfnWorkflowEngineAdapter

Unico adapter che implementa la porta condivisa verso Step Functions per entrambi i domini.

SfnWorkflowEngineAdapter
-client: SfnClient
+startExecution(stateMachineArn: string, executionName: string, input: array): string

Figura 79: Classe: SfnWorkflowEngineAdapter

Attributi

- `client: SfnClient`.

Metodi

- `startExecution(stateMachineArn: string, executionName: string, input: array): string`: avvia un'esecuzione Step Functions, restituendo l'ARN dell'esecuzione.

5.3.1.6 WorkflowContext

Contenitore puro degli identificativi di correlazione (request id, correlation id, tenant) propagati lungo l'esecuzione di un caso d'uso, senza alcuna dipendenza da Illuminate: il tagging dei log (`Log::withContext()`) vive nel middleware `HTTP CorrelateRequests` e, per il worker SQS, in `ConsumeWorkflowTasks`.

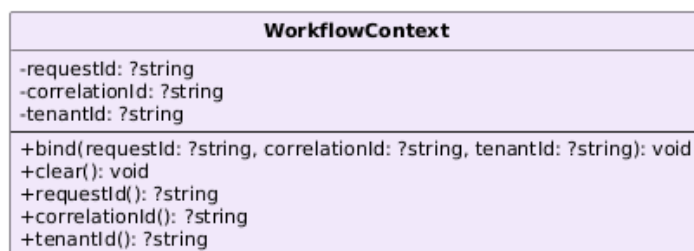


Figura 80: Classe: WorkflowContext

Attributi

- `requestId: ?string`.
- `correlationId: ?string`.
- `tenantId: ?string`.

Tutti e tre nullable: inizializzati a `null` e azzerati da `clear()`, valorizzati solo fra un `bind()` e la fine dell'esecuzione corrente.

Metodi

- `bind(?requestId, ?correlationId, ?tenantId): void`: imposta gli identificativi di correlazione per l'esecuzione corrente; parametri nullable, una stringa vuota è normalizzata a `null`.
- `clear(): void`: azzerà il contesto a fine esecuzione.
- `requestId(): ?string`, `correlationId(): ?string`, `tenantId(): ?string`: accessori di sola lettura.

5.3.1.7 Actor

Chi ha compiuto un'azione (id, tenant, ruoli), tradotto una sola volta al bordo HTTP da `MvpUser`; porte, eventi e comandi di dominio vedono solo questa classe, mai il contratto `Authenticatable` di Laravel.

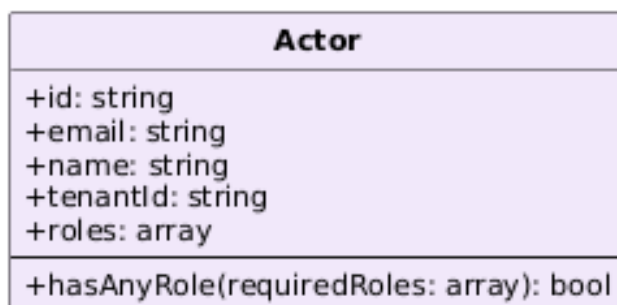


Figura 81: Classe: Actor

Attributi

- `id: string, email: string, name: string, tenantId: string, roles: array`: tutti readonly, impostati una sola volta alla costruzione.

Metodi

- `hasAnyRole(requiredRoles: array): bool`: vero se l'attore possiede almeno uno dei ruoli richiesti.

5.3.1.8 MvpUserProvider

`UserProvider` esplicito per il guard `mvp`, registrato in `AppServiceProvider::boot()`. Poiché `MvpUser` non ha un archivio persistente (viene ricostruito a ogni richiesta da `ResolveMvpIdentity`, mai risolto da un provider), ogni metodo di questa classe fallisce deliberatamente con un messaggio esplicito invece di lasciare che una risoluzione inattesa fallisca con un errore criptico.

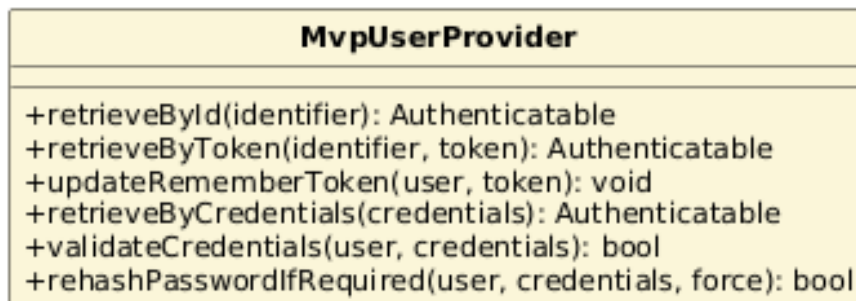


Figura 82: Classe: `MvpUserProvider`

Metodi

- `retrieveById(identifier)`, `retrieveByToken(identifier, token)`, `updateRememberToken(user, token)`, `retrieveByCredentials(credentials)`, `validateCredentials(user, credentials)`, `rehashPasswordIfRequired(user, credentials, force)`: i sei metodi del contratto `UserProvider`, tutti implementati per lanciare una `LogicException` esplicativa.

5.3.1.9 SystemClock

Unico adapter condiviso dell'interfaccia standard PSR-20 `ClockInterface`, usata al posto di un helper temporale custom.

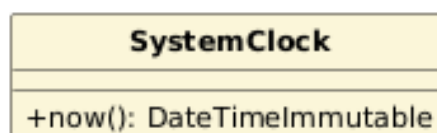


Figura 83: Classe: `SystemClock`

Metodi

- `now(): DateTimeImmutable`: restituisce l'istante corrente.

5.3.1.10 RandomUuidGenerator

Unico adapter condiviso della porta `UniqueIdGeneratorPort`, per la generazione di identificativi univoci.

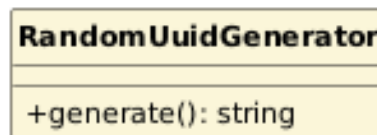


Figura 84: Classe: RandomUuidGenerator

Metodi

- `generate(): string`: genera un nuovo UUID.

5.3.1.11 LaravelTransactionManager

Unico adapter condiviso della porta `TransactionManagerPort`, per le sequenze di scritture DB pure che richiedono un confine transazionale.

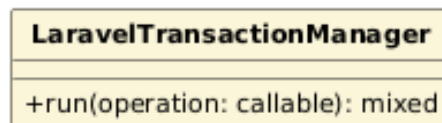


Figura 85: Classe: LaravelTransactionManager

Metodi

- `run(operation: callable): mixed`: esegue l'operazione all'interno di una transazione DB.

5.3.1.12 AuditLogger

Registra gli eventi di audit dispatchati dai listener di dominio.

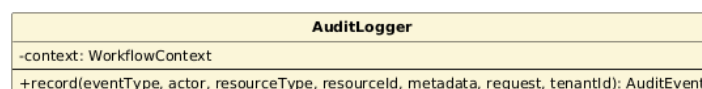


Figura 86: Classe: AuditLogger

Attributi

- `context: WorkflowContext.`

Metodi

- `record(eventType, actor, resourceType, resourceId, metadata, request, tenantId): AuditEvent`: registra un evento di audit; fuori da una richiesta HTTP (worker SQS) gli id di correlazione arrivano da `WorkflowContext` anziché dalla request.

6 Architettura Frontend

6.1 Principi di progettazione

La SPA Angular è organizzata per feature (`features/assistant`, `features/copilot`, `features/overview`), affiancate da tre cartelle trasversali: `core` (stato applicativo, interceptor HTTP, osservabilità), `shared` (componenti UI riutilizzabili) e `layout` (guscio dell'applicazione, intestazioni, navigazione). Ogni feature raggruppa a sua volta una pagina (View) e il proprio ViewModel, affiancati dai propri componenti (`components/`) e dai propri servizi e modelli (`data/`) dove servono: `overview`, che si limita a comporre dati già presenti nello Store, non ha né gli uni né gli altri. La struttura rispecchia la stessa separazione per dominio applicativo già adottata nel backend (Sezione 4.3).

Tutti i componenti sono standalone (nessun `NgModule`) e usano `ChangeDetectionStrategy.OnPush` con le API a segnali di Angular 21 (`signal`, `computed`, `input()`) al posto delle proprietà decorate con `@Input()`: un componente si ridisegna solo quando un segnale che consuma cambia valore, non a ogni ciclo di change detection globale.

Il routing e i template usano la sintassi di controllo nativa (`@if`, `@for`) introdotta nelle versioni recenti del framework, senza le direttive strutturali `*ngIf`/`*ngFor`. Le tre pagine principali (`AssistantPage`, `CopilotPage`, `OverviewPage`) separano inoltre la logica di presentazione dalla vista tramite un ViewModel dedicato (Sezione 6.2), un principio applicato solo lì: i componenti di feature e quelli condivisi restano invece componenti Angular ordinari, senza un ViewModel proprio.

6.2 Il pattern MVVM

La figura seguente illustra il pattern e la comunicazione fra i suoi tre componenti; un esempio concreto tratto dal progetto segue in Sezione 6.2.1.

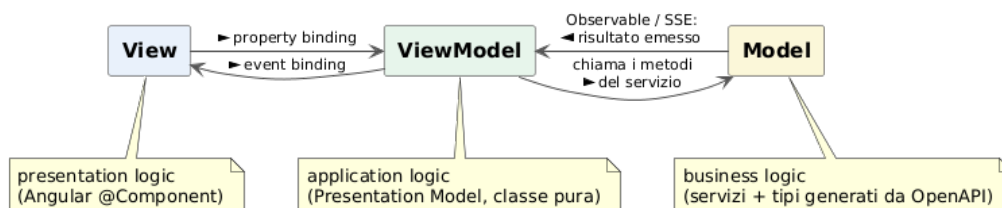


Figura 87: Il pattern MVVM

Le tre pagine principali applicano MVVM in senso classico: un *ViewModel* (più precisamente un *Presentation Model* nel senso di Fowler) è una classe TypeScript che non tocca il DOM né l'infrastruttura di rendering o di dependency injection di Angular (nessun `@Component`, `inject()`, decoratore, injection context, chiamata diretta a `document/window`); riceve le proprie dipendenze dal costruttore, ed è quindi istanziabile con un semplice `new` e testabile senza `TestBed`, con dipendenze fittizie create a mano. L'unica eccezione apparente è l'uso di `signal/computed` di `@angular/core`: restano nel *ViewModel* non come parte del motore di rendering, ma come primitiva reattiva di piattaforma, quindi non compromettono l'istanziabilità con `new` né la testabilità senza Angular avviato; è questa distinzione, non un conteggio letterale degli import da `@angular/core`, a rendere la classe un *Presentation Model* genuino.

Il *Model* resta quanto descritto in Sezione 6.3: i tipi generati dal contratto OpenAPI e i modelli di dominio specifici di ciascuna feature. La *View* è la pagina stessa (`AssistantPage`, `CopilotPage`, `OverviewPage`): l'unico punto accoppiato ad Angular e al DOM, che si procura le dipendenze con `inject()` nel proprio costruttore e le passa al *ViewModel* per costruirlo, poi lega il template ai segnali e ai metodi pubblici che il *ViewModel* espone (`vm.stato()`, `(evento)="vm.metodo()"`). Anche l'unica operazione DOM richiesta da alcuni *ViewModel* (lo scorrimento verso una sezione dopo un'azione) resta un'operazione della *View*, ma non tramite una funzione-callback ricevuta dal costruttore: `AssistantPageViewModel` e `OverviewPageViewModel` scrivono un segnale osservabile, `pendingScrollTarget: WritableSignal<string | null>`, e un `effect()` nella *View* lo legge, chiama `scrollToElement` (definita in `shared/util/scroll.ts`) quando non è nullo, poi lo azzerà. Un segnale osservato è un fatto dichiarativo puro, che la *View* può scegliere di leggere o ignorare senza che il *ViewModel* sappia se lo ha fatto. Se non azzerasse il segnale dopo averlo consumato, la stessa destinazione richiesta due volte di seguito non ritriggererebbe l'`effect()`, che reagisce solo a un cambio di valore.

Lo stato condiviso fra le pagine non vive nel *ViewModel*: una sola istanza di `MvpStateStore`, fornita a livello di applicazione (`providedIn: "root"`), conserva lo stato restituito dal backend e sopravvive ai cambi di rotta. Ogni *ViewModel* riceve lo Store come una qualunque altra dipendenza nel proprio costruttore e lo consuma in sola lettura per derivare i propri segnali `computed`; le mutazioni (generazione, upload, revisione, eliminazione) non aggiornano lo stato localmente, ma rimpiazzano l'intero stato con quello autorevole restituito dal backend nella risposta, evitando divergenze fra ciò che la SPA mostra e ciò che il backend ha effettivamente persistito.

Un'eccezione è documentata esplicitamente nel codice: `effect()` e `takeUntilDestroyed()` restano nella *View* perché richiedono un injection context che il *ViewModel* non ha per costruzione. Gli `effect()` però si limitano a leggere i segnali sorgente (lo storico o i documenti condivisi, i filtri attivi, il documento selezionato) e a chiamare un metodo del *ViewModel* (`vm.reload()`, `vm.loadPreviewStatus()`...): la chiamata vera e propria (stessa forma di ogni altra azione del *ViewModel*, come `generate()` o `discard()`) vive nel *ViewModel*, non nella *View*. I metodi che inoltrano il risultato (`setFilteredCommunications()`, `handleHistoryError()`) sono privati: dettagli interni di `reload()`, non superficie pubblica del *ViewModel*. Per lo stesso motivo il template delle tre pagine legge sempre e solo `vm.*`, mai `store.*` direttamente: anche il banner d'errore principale chiama un metodo del *ViewModel*: `vm.reloadState()` su `AssistantPage` e `CopilotPage`, `vm.reload()` su `OverviewPage` (l'unica delle tre il

cui ViewModel non distingue fra ricarica dello stato condiviso e ricarica di uno storico proprio, non avendone uno). Nessuna delle tre pagine permette a una View di raggiungere lo Store direttamente dal markup: il campo `store`, `inject()`ato su tutte e tre, serve solo a costruire il ViewModel nel costruttore e, su `AssistantPage/CopilotPage`, come segnale sorgente letto dall'`effect()` di ricerca, mai referenziato nel template.

I segnali `computed` pass-through sul ViewModel seguono lo stesso schema su tutte e tre le pagine: tutte e tre espongono `vm.error()` e `vm.loading()`; `AssistantPage` e `CopilotPage` espongono inoltre `vm.metrics()` (l'elenco di metriche del proprio modulo, filtrato delle voci che il pannello non mostra come scheda a sé) e `vm.metricsPresentation()` (la mappa che indica a `MetricsPanelComponent`, Sezione 6.7, come disegnare ciascuna scheda). `OverviewPage` non monta un pannello di metriche generico: il suo ViewModel espone segnali propri più mirati (`vm.priorities()`, `vm.assistantQuality()`, `vm.copilotQuality()`, `vm.quarantined()`), descritti nella sua sottosezione più sotto.

Angular offre già disaccoppiamento e testabilità tramite i propri servizi e la dependency injection, quindi un ViewModel separato potrebbe sembrare ridondante rispetto a testare direttamente il componente con `TestBed`. Il motivo per cui si è scelto comunque questo confine è pratico, non dogmatico: un test su `TestBed` richiede l'avvio del modulo, la compilazione del template e la risoluzione dell'albero di dependency injection a ogni esecuzione, mentre un test sul ViewModel istanzia la classe con `new` passando dipendenze fittizie create a mano: più veloce da eseguire e più semplice da scrivere, perché isola la logica applicativa dal ciclo di vita di Angular. Per questo il confine è tracciato solo sulle tre pagine principali, dove quella logica è più densa, e non esteso a ogni componente della SPA.

L'assenza di `inject()` e dell'infrastruttura di rendering nel ViewModel non è un'omissione stilistica: è la condizione che rende possibile l'obiettivo dichiarato sopra. `inject()` funziona solo dentro un injection context, quindi una classe che lo usasse sarebbe istanziabile solo da Angular: per testarla servirebbe comunque `TestBed.runInInjectionContext()` o un `TestBed.inject()`, lo stesso costo che l'estrazione del ViewModel vuole evitare. Lo stesso vale per il rendering: un ViewModel che manipolasse il DOM richiederebbe un ambiente browser (o `jsdom`) anche solo per verificarne la logica. Estendere questo pattern a ogni componente della SPA, non solo alle tre pagine principali, non è però la conclusione corretta: il beneficio (test framework-indipendenti su logica applicativa densa) va confrontato con il costo (un file e un'indirection in più). I componenti condivisi (Sezione 6.7) non hanno quasi logica propria (ricevono un `input()` e lo mostrano), e un ViewModel dedicato aggiungerebbe solo un file vuoto da mantenere, senza nulla di significativo da isolare che il codice non mostri già direttamente.

6.2.1 Esempio di pattern MVVM nel progetto

Il diagramma seguente illustra il pattern con un esempio concreto tratto dalla pagina di generazione (`AssistantPage`): per chiarezza è semplificato a una sola delle componenti riutilizzabili della pagina.

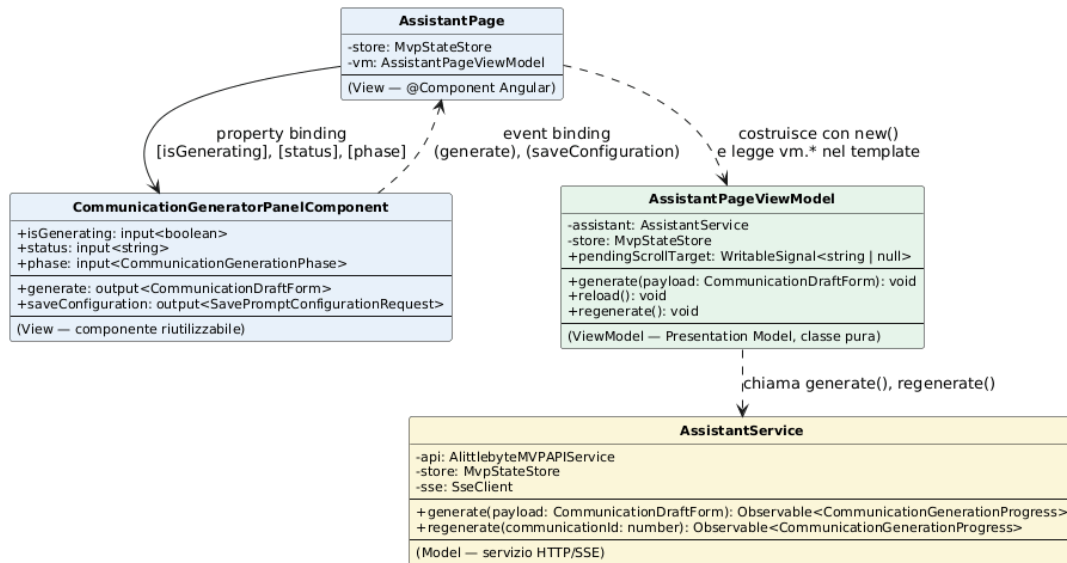


Figura 88: Esempio di pattern MVVM: AssistantPage

La classe **AssistantPage** (View) costruisce **AssistantPageViewModel** (ViewModel) con `new()` nel proprio costruttore e nel template legge solo i suoi segnali e metodi pubblici (`vm.*`). **AssistantPageViewModel**, a sua volta, non tocca mai Angular né il DOM: risponde alle azioni richieste dalla View chiamando le operazioni esposte da **AssistantService** (Model), il servizio che comunica con l'API generata e con lo stream SSE.

Per completezza si mostra anche la relazione con il componente riutilizzabile **CommunicationGeneratorPanelComponent**: **AssistantPage** lo importa nel proprio template e gli passa lo stato tramite property binding (`[isGenerating]`, `[status]`, `[phase]`); il componente notifica le azioni dell'utente alla View tramite event binding (`(generate)`, `(saveConfiguration)`), che le inoltra al ViewModel (`vm.generate($event)`): lo stesso schema vale per qualunque altro componente riutilizzabile della SPA.

6.2.2 Two-way data binding

Il progetto non usa il binding bidirezionale nativo di Angular (`[(ngModel)]`), equivalente meccanicamente alla stessa coppia `[input]/(outputChange)` già usata altrove, o l'API `model()`: il motivo è strutturale, non stilistico. `model()` richiede che sia il componente stesso a possedere il segnale bidirezionale, ma il ViewModel resta volutamente una classe pura fuori dall'injector di Angular (Sezione 6.2), quindi non potrebbe comunque parteciparvi. La relazione fra View e ViewModel non è inoltre il caso d'uso naturale del two-way binding, pensato per rispecchiare un valore grezzo: il componente riceve stato derivato e `computed` dal ViewModel, ed emette azioni (`generate`, `saveConfiguration`) che devono passare per logica applicativa, non per una scrittura diretta, prima di cambiare qualcosa.

Questo spiega anche la scelta di MVVM invece di MVP: un Presenter MVP dovrebbe aggiornare la View direttamente tramite un riferimento esplicito a essa, il che comprometterebbe l'istanziabilità con `new()` e la testabilità senza **TestBed** del ViewModel. Con MVVM è invece Angular stesso a occuparsi della sincronizzazione: **ChangeDetectionStrategy.OnPush** ridisegna la View a ogni cambiamento di un segnale letto nel template, senza che il ViewModel debba mai sapere che la View esiste.

6.2.3 AssistantPageViewModel

Il ViewModel più corposo dei tre, perché copre l'intero ciclo di vita di una bozza (generazione, rigenerazione, valutazione, copertina, storico, configurazioni di prompt salvate) oltre alla presentazione delle proprie metriche. Il test unitario associato (`assistant-page.view-model.spec.ts`) lo costruisce con `new` passando un `AssistantService` e un `MvpStateStore` simulati a mano (senza avviare Angular), a riprova diretta che qui MVVM non è solo nominale.

AssistantPageViewModel
-assistant: AssistantService -store: MvpStateStore +phase: CommunicationGenerationPhase "idle" +isGenerating: boolean +isUpdatingCover: boolean +isDiscarding: boolean +isSavingToHistory: boolean +confirmingDeleteld: number null +isDeletingHistoryItem: boolean +confirmingConfigDeleteld: number null +isDeletingConfig: boolean +isRating: boolean +rateError: string null +status: string +selectedDraftId: number null +latestDraft: GeneratedDraft null +isSavingDraft: boolean +saveDraftError: string null +isSavingConfiguration: boolean +saveConfigurationError: string null +prefillPayload: CommunicationDraftForm null +promptConfigurations: PromptConfiguration[] +filteredPromptConfigurations: PromptConfiguration[] +togglingFavoriteId: number null +pendingScrollTarget: string null +previewDraft: GeneratedDraft null +activeFilters: CommunicationFilters +hasActiveFilters: boolean +filteredCommunications: Communication[] +historyError: string null +error: string null +loading: boolean +metricsFilters: AssistantMetricsFilters +hasActiveMetricsFilters: boolean +metrics: Metric[] +metricsPresentation: Record<string, MetricPresentation> +recentFeedback: Communication[] +metricsReportExportUrl: string +draftComposition: CompositionSlice[]
+setActiveFilters(filters: CommunicationFilters): void +setMetricsFilters(filters: AssistantMetricsFilters): void +refreshFilteredMetrics(): void +reload(): void +reloadState(): void +destroy(): void +generate(payload: CommunicationDraftForm): void +regenerate(): void +rateDraft(payload: RateDraftPayload): void +selectDraft(communicationId: number): void +uploadCover(file: File): void +saveDraft(event: {communicationId, payload}): void +removeCover(): void +discard(): void +saveToHistory(): void +saveConfiguration(payload: SavePromptConfigurationRequest): void +useConfiguration(configuration: PromptConfiguration): void +deleteConfiguration(configurationId: number): void +deleteHistoryItem(communicationId: number): void +toggleFavorite(communication: Communication): void

Attributi

- **assistant:** `AssistantService`, **store:** `MvpStateStore`: dipendenze ricevute dal costruttore, non da `inject()`.
- **phase**, **isGenerating**, **latestDraft**, **previewDraft**, **status**: segnali sullo stato della generazione in corso e della bozza visualizzata.
- **pendingScrollTarget:** `WritableSignal<string | null>`: destinazione di uno scorrimento richiesto dal `ViewModel`, non ancora consumato dalla `View` (si veda l'inizio della Sezione 6.2); scritto da `selectDraft()`, `discard()`, `useConfiguration()` e da `handleProgress()`, azzerato dalla `View` dopo averlo eseguito.
- **error:** `Signal<string | null>`, **loading:** `Signal<boolean>`: combinano `store.error()/store.loading()` con `store.filteredAssistantError()/store.filteredAssistantLoading()`, così il banner e lo spinner principali riflettono anche un fallimento o un caricamento della sola sezione Metriche filtrata, non solo dello stato condiviso.
- **metricsFilters:** `WritableSignal<AssistantMetricsFilters>`, **hasActiveMetricsFilters:** `Signal<boolean>`: filtri della sezione Metriche (tono, stile, intervallo di date, RF38-OB..RF41-OB), separati da `activeFilters` sotto: filtrare le metriche non deve filtrare anche lo storico, e viceversa.
- **metrics:** pass-through su `store.filteredAssistantState()?.metrics`, esclusa la voce `assistant.drafts` (le bozze da valutare sono una priorità e vivono nella Overview; qui sono già la parte "in bozza" di `draftComposition`, sotto). Legge dalla fetta filtrata dello Store, non da `store.assistantMetrics()`: quest'ultima resterebbe sempre sul totale non filtrato.
- **metricsPresentation:** `Signal<Record<string, MetricPresentation>>`: per ciascuna delle quattro metriche mostrate (`assistant.total`, `assistant.prompt_configurations`, `assistant.rated`, `assistant.rating_average`: due andamenti, una quota e una media a stelle, tutte a due celle), la forma della scheda e il suo ingombro nella griglia, letta da `MetricsPanelComponent` (Sezione 6.7).
- **recentFeedback:** `Signal<Communication[]>`: pass-through su `store.filteredAssistantState()?.recentFeedback` (UC-27.3), gli ultimi contenuti valutati con voto e commento testuale, ricalcolati secondo `metricsFilters` come il resto della sezione, non solo la media, che resta in `metricsPresentation` sopra.
- **metricsReportExportUrl:** `string`: URL statico del download del report riepilogativo delle metriche (`/api/v1/assistant/metrics/export`, RF34-OB), letto direttamente dal template come `href` del link di esportazione.

- **draftComposition:** `Signal<CompositionSlice[]>`: ripartizione delle comunicazioni fra "In bozza" e "Decise" (il totale meno le bozze), derivata dalla stessa fetta filtrata di **metrics** sopra.
- **activeFilters**, **filteredCommunications**, **historyError**: segnali sullo storico, letti dal template; scritti solo internamente da **reload()**.

Metodi

- **generate(payload: CommunicationDraftForm): void, regenerate(): void**
- **rateDraft(payload: RateDraftPayload): void, selectDraft(communicationId: number): void**
- **uploadCover(file: File): void, removeCover(): void**
- **saveDraft(event: {communicationId: number, payload: UpdateCommunicationRequest}): void**
- **discard(): void, saveToHistory(): void, deleteHistoryItem(communicationId: number): void**
- **toggleFavorite(communication: Communication): void**
- **saveConfiguration(payload: SavePromptConfigurationRequest): void, useConfiguration(configuration: PromptConfiguration): void, deleteConfiguration(configurationId: number): void**
- **setActiveFilters(filters: CommunicationFilters): void**
- **setMetricsFilters(filters: AssistantMetricsFilters): void**
- **refreshFilteredMetrics(): void**: rilegge la sezione Metriche con il filtro attivo corrente, tramite **store.reloadFilteredAssistantMetrics()** (RF38-OB).
- **reload(): void**: rilegge lo storico con i filtri correnti; stessa forma delle altre azioni, chiamata dall'**effect()** della View a ogni cambio di filtro o di stato condiviso. Annulla la propria sottoscrizione precedente prima di avviarne una nuova: senza questa cancellazione, una risposta di ricerca arrivata in ritardo potrebbe sovrascrivere un risultato più recente.
- **reloadState(): void**: ricarica lo stato condiviso tramite **store.reload()** e la sezione Metriche filtrata tramite **refreshFilteredMetrics()**, invocato dal pulsante "Riprova" del banner d'errore principale. Distinto da **reload()**, che rilegge solo lo storico comunicazioni: confonderli lascerebbe lo stato globale in errore pur avendo ricaricato l'elenco.
- **destroy(): void**: chiamato dalla View alla distruzione del componente (tramite **DestroyRef**): annulla solo la sottoscrizione di lettura in corso, non le azioni di scrittura (**generate()**, **discard()**...), che devono completare lato server anche se l'utente ha già navigato altrove.

6.2.4 CopilotPageViewModel

Stessa forma di `AssistantPageViewModel` applicata al Co-Pilot, con una superficie più piccola: copre upload, eliminazione, revisione dei dati estratti e correzione del messaggio di invio. Non ha bisogno di una dipendenza di scorrimento (nessuna delle sue azioni richiede di spostare la pagina verso una sezione), ma possiede una fetch propria oltre allo storico documenti: la raggiungibilità dell'anteprima del documento selezionato.

CopilotPageViewModel
<pre>-workflow: DocumentWorkflowService -store: MvpStateStore +selectedDocumentId: string null +selectedDocument: SubDocument null +selectedDocumentIdForList: string null +previewStatus: DocumentPreviewStatus +isUploading: boolean +uploadStatus: string +uploadPhase: DocumentUploadPhase null +isDeleting: boolean +isSavingReview: boolean +reviewError: string null +isSavingSendMessage: boolean +sendMessageError: string null +reviewFieldErrors: Record<string, string> null +activeFilters: DocumentFilters +hasActiveFilters: boolean +filteredDocuments: SubDocument[] +documentsError: string null +currentPage: number +totalDocuments: number +pageSize: number +totalPages: number +hasPreviousPage: boolean +hasNextPage: boolean +error: string null +loading: boolean +metrics: Metric[] +metricsPresentation: Record<string, MetricPresentation></pre>
<pre>+setActiveFilters(filters: DocumentFilters): void +goToPage(page: number): void +reload(): void +loadPreviewStatus(previewUrl: string null): void +reloadState(): void +destroy(): void +selectDocument(documentId: string null): void +upload(request: DocumentUploadRequest): void +deleteDocument(documentId: string): void +markReviewed(documentId: string): void +saveReview(event: {documentId, payload}): void +saveSendMessage(event: {documentId, payload}): void</pre>

Figura 90: Classe: CopilotPageViewModel

Attributi

- `workflow`: `DocumentWorkflowService`, `store`: `MvpStateStore`: dipendenze ricevute dal costruttore, non da `inject()`.

- `selectedDocumentId`, `selectedDocument`, `selectedDocumentIdForList`: segnali sul sotto-documento attualmente in revisione.
- `previewStatus`: `WritableSignal<DocumentPreviewStatus>`: raggiungibilità dell'anteprima del documento selezionato (*idle*, *loading*, *available*, *unavailable*, *unreachable*), verificata prima che il dettaglio monti l'iframe del PDF; segue `selectedDocument`, richiesta di nuovo a ogni cambio tramite `loadPreviewStatus()`.
- `isUploading`, `uploadStatus`, `uploadPhase`, `isDeleting`, `isSavingReview`, `reviewError`, `reviewFieldErrors`, `isSavingSendMessage`, `sendMessageError`: segnali di stato per le rispettive azioni asincrone.
- `activeFilters`, `hasActiveFilters`, `filteredDocuments`, `documentsError`: segnali sullo storico documenti, scritti solo internamente da `reload()`.
- `currentPage`: `WritableSignal<number>` (default 1), `totalDocuments`: `WritableSignal<number>` (default 0), `pageSize`: paginazione dello storico, dieci righe a pagina.
- `totalPages`: `Signal<number>`, `hasPreviousPage`: `Signal<boolean>`, `hasNextPage`: `Signal<boolean>`: segnali derivati da `currentPage/totalDocuments`. Alla View viene passato `totalPages`, da cui `DocumentListComponent` ricava da sé lo stato dei propri controlli di paginazione; `hasPreviousPage` e `hasNextPage` restano disponibili sul `ViewModel` ma nessun template li consuma.
- `error`, `loading`: pass-through su `store.error()/store.loading()`, così il template non legge mai lo Store direttamente.
- `metrics`: pass-through su `store.copilotMetrics()`, escluse le voci che il pannello non mostra come scheda a sé (`copilot.ocr_confidence`, `copilot.validated`, `copilot.quarantined`, `copilot.sub_documents`): `validated` e `quarantined` sono esiti di validazione, `quarantined` vive anche come priorità nella Overview; `sub_documents` è il totale su cui si misurano le quote; `ocr_confidence` non risponde a nessun caso d'uso UC-56, ma resta nel contratto perché la Overview la legge (`vm.copilotQuality()`, Sezione 6.2).
- `metricsPresentation`: `Signal<Record<string, MetricPresentation>>`: stesso ruolo di `AssistantPageViewModel.metricsPresentation`, per le cinque metriche mostrate (`copilot.documents`, `copilot.processing_seconds` (due andamenti stretti) e tre quote larghe: `copilot.auto_classified` per UC-56.2, `copilot.needs_review` per UC-56.3, `copilot.recipient_auto_matched` per UC-56.4). L'ordine delle chiavi non è arbitrario: la griglia CSS non ricompone le righe per colmare i vuoti, quindi le due schede strette devono precedere le tre larghe perché le righe si chiudano senza celle vuote (due celle, poi sei, chiudono due righe da quattro).

Metodi

- `setActiveFilters(filters: DocumentFilters): void`: azzera anche `currentPage` a 1: cambiare filtro rimescola i risultati, e restare su una pagina che con il nuovo filtro non esiste più mostrerebbe una tabella vuota senza spiegazione.
- `goToPage(page: number): void`: va alla pagina indicata, vincolata fra 1 e `totalPages()`; se cambia, aggiorna `currentPage` e richiama `reload()`.
- `reload(): void`: stessa cancellazione della sottoscrizione precedente vista in `AssistantPageViewModel.reload()`; legge anche `currentPage/pageSize` nella richiesta al backend. Il corpo gira dentro `untracked(): CopilotPage` lo chiama da un `effect()` che legge solo `store.documents()/activeFilters()` apposta, per non far ripartire la ricerca due volte a ogni cambio pagina (`goToPage()` richiama già `reload()` da sé): senza `untracked()`, la lettura di `currentPage()` qui dentro diventerebbe comunque una dipendenza nascosta di quell'effect, e ogni "pagina successiva" scatenerrebbe due richieste invece di una.
- `loadPreviewStatus(previewUrl: string | null): void`: richiede di nuovo la raggiungibilità dell'anteprima, stessa forma di `reload()` ma sul documento selezionato invece che sullo storico; annulla la propria sottoscrizione precedente, così una risposta tardiva su un documento ormai deselezionato non sovrascrive lo stato di quello corrente. Con `previewUrl` nullo azzera lo stato a *idle* senza chiamare il backend.
- `reloadState(): void`: ricarica lo stato condiviso tramite `store.reload()`, invocato dal pulsante "Riprova" del banner d'errore principale; distinto da `reload()` come nel `ViewModel` dell'Assistant.
- `destroy(): void`: annulla sia la sottoscrizione di ricerca sia quella di anteprima.
- `selectDocument(documentId: string | null): void`
- `upload(request: DocumentUploadRequest): void`,
`deleteDocument(documentId: string): void`
- `markReviewed(documentId: string): void`
- `saveReview(event: {documentId: string, payload: UpdateExtractedDataRequest}): void`
- `saveSendMessage(event: {documentId: string, payload: UpdateSendMessageRequest}): void`

6.2.5 OverviewPageViewModel

Il più semplice dei tre nella forma del costruttore (nessuna azione di scrittura verso il backend): sintetizza priorità operative e indicatori di qualità per entrambi i moduli, così il template legge solo `vm.*` invece di raggiungere direttamente lo Store. Tutti i valori derivano da metriche sull'intero tenant tramite `store.metricEntry()`, non da un ricalcolo sulle liste locali: `history` e `documents` nello Store sono finestre limitate (rispettivamente

10 e 40 elementi), e un valore derivato da quelle finestre sarebbe silenziosamente sbagliato appena i dati reali le superano.

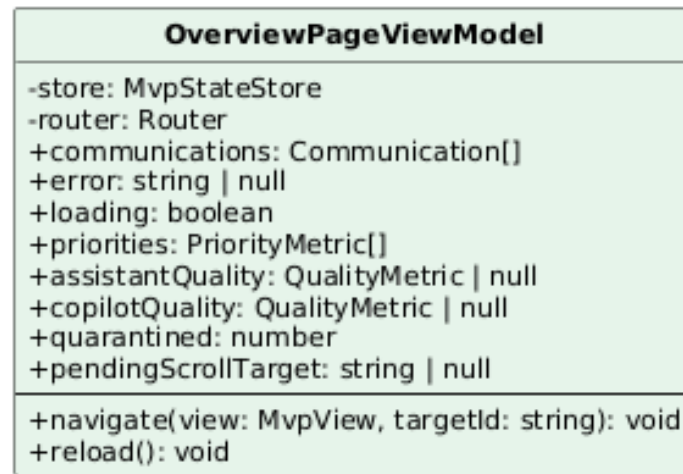


Figura 91: Classe: OverviewPageViewModel

Due tipi di supporto, definiti in questo stesso file, formattano i dati prima che raggiungano il template: **QualityMetric** (*label*, *value*, *unit*, *numeric*, *threshold*) per un indicatore di qualità già pronto per una scheda su scala; **PriorityMetric** (*key*, *label*, *value*, *tone*, *history*, *context*) per una priorità operativa con il proprio andamento a sette giorni.

Attributi

- **store**: **MvpStateStore**, **router**: **Router**: dipendenze ricevute dal costruttore (nessuna funzione di scorrimento: si veda **pendingScrollTarget** sotto).
- **communications**: pass-through su **store.history()**.
- **error**: **Signal<string | null>**, **loading**: **Signal<boolean>**: pass-through su **store.error()/store.loading()**, come già su **AssistantPageViewModel** e **CopilotPageViewModel**.
- **priorities**: **Signal<PriorityMetric[]>**: le tre metriche su cui si agisce (*da verificare*, *in quarantena*, *bozze da valutare*), non un riassunto di tutte: i conteggi puramente descrittivi restano nelle pagine dei rispettivi moduli.
- **assistantQuality**: **Signal<QualityMetric | null>**, **copilotQuality**: **Signal<QualityMetric | null>**: un solo indicatore di qualità per modulo (voto medio delle bozze per l'Assistant, confidenza media OCR per il Co-Pilot), con rimando alla pagina che ne porta il dettaglio.
- **quarantined**: **Signal<number>**: numero di sotto-documenti in quarantena, letto da **store.metricEntry("copilot.quarantined")**, usato per decidere se mostrare la segnalazione d'attenzione in cima alla sezione priorità.

- `pendingScrollTarget: WritableSignal<string | null>`: stesso ruolo e stesso protocollo di consumo/azzeramento di `AssistantPageViewModel.pendingScrollTarget`, scritto da `navigate()`.

Metodi

- `navigate(view: MvpView, targetId: string): void`: naviga verso un'altra pagina e, a navigazione completata, chiede lo scorrimento verso la sezione target scrivendo `pendingScrollTarget`.
- `reload(): void`: ricarica lo stato condiviso tramite `store.reload()`, invocato dal pulsante "Riprova" del banner d'errore principale. A differenza degli altri due `ViewModel` non esiste un `reloadState()` distinto: la `Overview` non ha uno storico proprio da rileggere separatamente dallo stato condiviso.

6.2.6 MvpStateStore (stato condiviso fra i ViewModel)

Sorgente unica dello stato applicativo (`Assistant` e `Co-Pilot`), condivisa da tutte le viste tramite i rispettivi `ViewModel`. Espone esclusivamente segnali di sola lettura, derivati dai propri segnali privati (lo stato condiviso, i suoi indicatori di caricamento ed errore e la fetta filtrata delle metriche `Assistant`), così che né i `ViewModel` né i componenti possano scrivere lo stato direttamente, ma solo tramite i metodi dedicati.



Figura 92: Classe: `MvpStateStore`

Attributi

- `api: AlittlebyteMVPAIService`: client HTTP generato dal contratto `OpenAPI`, iniettato tramite `inject()` (unico punto della classe accoppiato ad Angular, essendo un vero `@Injectable` e non un `ViewModel` puro).
- `state: Signal<MvpState | null>`, `loading: Signal<boolean>`, `error: Signal<string | null>`: segnali di sola lettura.

- `documents`, `history`, `recentFeedback`, `promptConfigurations`, `assistantMetrics`, `copilotMetrics`: segnali derivati (`computed`) sulle diverse viste dello stato; `recentFeedback` è il più recente (UC-27.3), gli ultimi contenuti valutati con voto e commento testuale.
- `filteredAssistantState`: `Signal<MvpState["assistant"] | null>`, `filteredAssistantLoading`: `Signal<boolean>`, `filteredAssistantError`: `Signal<string | null>`: fetta di stato separata da `state` per la sezione Metriche filtrata dell'AI Assistant (RF38-OB..RF41-OB, tono/stile/intervallo di date): applicare un filtro qui non cambia i numeri non filtrati che Overview e Co-Pilot leggono da `state`.

Metodi

- `loadOnce()`: `void`: carica lo stato al primo montaggio della shell applicativa; deriva la guardia dallo stato reale dei segnali (`loading` o `state` già valorizzato), così un primo tentativo fallito può essere ritentato da una chiamata successiva invece di restare bloccato per sempre.
- `reload()`: `void`: ricarica lo stato dal backend, con un tentativo di retry per errori temporanei. Ignora una chiamata mentre una richiesta è già in volo, la stessa guardia già applicata da `loadOnce()`: senza di essa un doppio click su "Riprova" avvierebbe due `GET /state` in parallelo, l'ultima risposta vincerebbe comunque ma la richiesta in più sarebbe sprecata.
- `reloadFilteredAssistantMetrics(filters: AssistantMetricsFilters)`: `void`: ricarica solo la fetta Assistant della sezione Metriche, filtrata, su uno stato separato da `state`. Annulla la richiesta filtrata precedente ancora in volo prima di avviarne una nuova, stesso schema di `AssistantPageViewModel::reload()` per lo storico: un cambio di filtro mentre la risposta al precedente non è ancora arrivata non deve andare perso, altrimenti il pannello resterebbe fermo su un filtro che i controlli non mostrano più. Chiamato da un `effect()` della View a ogni cambio filtro: l'intero corpo gira dentro `untracked()`, perché una lettura di segnale al suo interno diventerebbe altrimenti una dipendenza di quell'effect, innescando un ciclo di richieste che non si ferma mai.
- `setState(state: MvpState)`: `void`: rimpiazza lo stato con quello autorevole restituito da una mutazione.
- `upsertDocument(document: SubDocument)`: `void`: inserisce o aggiorna in testa un sotto-documento ricevuto dallo stream SSE, senza ricaricare l'intero stato.
- `metric(key: string)`: `number`: legge il valore di una metrica per chiave stabile, evitando di ricalcolarla su elenchi parzialmente caricati; 0 se la chiave non esiste o lo stato non è ancora caricato.
- `metricEntry(key: string)`: `Metric | null`: come `metric()` ma restituisce la metrica completa (compresa `history`, che il solo valore non porta) o `null` finché lo stato non è disponibile, invece di collasare l'assenza su 0: una scheda che mostrasse 0 durante il caricamento affermerebbe un dato che non ha ancora. `metric()` lo chiama internamente invece di duplicarne la logica di ricerca.

6.3 Definizione dei modelli di dati

Il livello Model si articola su due strati. Il primo è generato automaticamente da Orval a partire dal contratto OpenAPI (Sezione 2.4): interfacce TypeScript per ogni schema (`Communication`, `SubDocument`, `MvpState`...), non modificabili a mano. Il secondo è specifico di ciascuna feature e arricchisce i tipi generati con concetti che esistono solo lato UI (fasi di una pipeline, etichette, vincoli dei form) senza duplicarne i campi.

6.3.1 `assistant.model.ts`

Arricchisce i tipi generati per il modulo Assistant. `CommunicationGenerationPhase` traduce le combinazioni di `generationStatus/coverStatus` restituite dal backend in un'unica fase discreta, usata dal template per etichette e stato dei controlli senza replicare quella logica in più punti; `GeneratedDraft` è la proiezione della bozza usata dal pannello di anteprima, con solo i campi che quella vista consuma.

6.3.1.1 `CommunicationDraftForm`

Alias del tipo di richiesta generato da OpenAPI, usato dal form di generazione senza introdurre una forma propria.



Figura 93: Tipo: `CommunicationDraftForm`

6.3.1.2 `CommunicationGenerationPhase`

Unione di stringhe letterali che traduce le combinazioni di `generationStatus/coverStatus` restituite dal backend in un'unica fase discreta, usata dal template per etichette e stato dei controlli senza replicare quella logica in più punti.

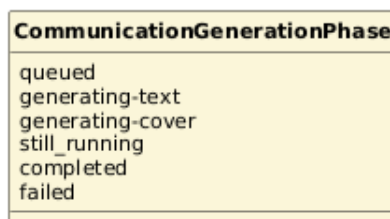


Figura 94: Tipo: `CommunicationGenerationPhase`

6.3.1.3 `CommunicationGenerationProgress`

Proiezione dell'evento di avanzamento SSE ricevuto durante la generazione: testo e copertina restano opzionali perché arrivano in momenti diversi della pipeline.

CommunicationGenerationProgress
+status: string +phase: CommunicationGenerationPhase +communicationId: number +text?: {title, body} +cover?: {coverImageUrl, coverStatus, coverError} +communication?: Communication

Figura 95: Tipo: CommunicationGenerationProgress

6.3.1.4 GeneratedDraft

Proiezione della bozza usata dal pannello di anteprima, con solo i campi che quella vista consuma, non l'intera **Communication** generata da OpenAPI.

GeneratedDraft
+id: number +body: string +coverImageUrl?: string +coverStatus: string +coverError?: string +status: string +statusValue?: string +title: string +previewUrl?: string +exportUrl?: string +generationStatus?: string +rating?: number null +ratingComment?: string null +ratedAt?: string null

Figura 96: Tipo: GeneratedDraft

6.3.1.5 RateDraftPayload

Payload minimo per la valutazione di una bozza: stelle e commento facoltativo.

RateDraftPayload
+rating: number +comment?: string

Figura 97: Tipo: RateDraftPayload

6.3.1.6 communicationTones e communicationStyles

Le opzioni di tono e stile selezionabili nel form di generazione, condivise fra form e validazione così da non poter divergere.

«const array» communicationTones
"Chiaro e diretto"
"Più istituzionale"
"Più sintetico"
"Empatico"
"Tecnico"

Figura 98: Tipo: communicationTones

«const array» communicationStyles
"Testo informativo"
"Avviso operativo"
"Aggiornamento breve"

Figura 99: Tipo: communicationStyles

6.3.2 Tipi del modulo Co-Pilot

L'equivalente Co-Pilot di `assistant.model.ts`, distribuito fra `document-workflow.service.ts` (i tipi) e `document-filters.ts` (il form dei filtri e la sua conversione). `DocumentUploadPhase` traduce l'evento SSE `progress` in una fase discreta per la barra di avanzamento, analogamente a `CommunicationGenerationPhase` sul modulo Assistant.

`toDocumentFilters()` converte i valori grezzi del form nei criteri inviati all'API scaricando quelli vuoti, evitando percorsi che possano sollevare eccezioni: un valore inatteso nel form non deve poter interrompere silenziosamente lo stream dei filtri.

6.3.2.1 DocumentUploadPhase

Fase dell'elaborazione usata dalla barra di progressione a step; *still_running* riflette l'omonima fase SSE del backend (Sezione 6.4, `SseClient`) quando lo stream supera il timeout senza che la pipeline sia effettivamente fallita.

DocumentUploadPhase
uploading
queued
processing
extracting
still_running
completed
failed

Figura 100: Tipo: DocumentUploadPhase

6.3.2.2 DocumentFilters

Criteri di filtro per l'elenco documenti mostrato nel Co-Pilot: ricerca testuale, stato di invio, soglia di confidenza col relativo criterio di confronto, mese e anno.

DocumentFilters
+search?: string
+sendStatus?: SubDocumentSendStatus
+confidenceThreshold?: number
+confidenceCriterion?: ConfidenceCriterion
+month?: number
+year?: number

Figura 101: Tipo: DocumentFilters

6.3.2.3 ConfidenceCriterion

Criterio di confronto per il filtro di confidenza: sopra o sotto la soglia impostata.

ConfidenceCriterion
above
below

Figura 102: Tipo: ConfidenceCriterion

6.3.2.4 DocumentUploadMetadata

Metadati manuali inviati insieme al file in upload: tipo di documento, ragione sociale, mese e anno di riferimento: gli stessi che il backend espone tramite `OriginalDocument::hasManualUploadMetadata()` (Sezione 5.1).

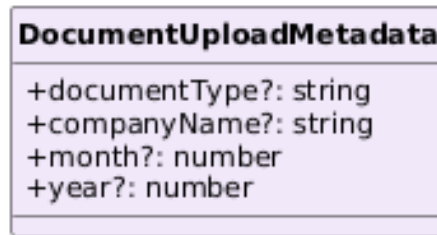


Figura 103: Tipo: DocumentUploadMetadata

6.3.2.5 DocumentUploadProgress

Avanzamento dell'upload e dell'elaborazione di un documento, con l'identificativo ricevuto non appena disponibile.



Figura 104: Tipo: DocumentUploadProgress

6.3.2.6 DocumentPage

Una pagina dello storico documenti, con il totale che le sta dietro: senza `total` la vista non può sapere quante pagine esistono. `CopilotPageViewModel` (Sezione 6.2) naviga esplicitamente fra le pagine tramite `goToPage()`.

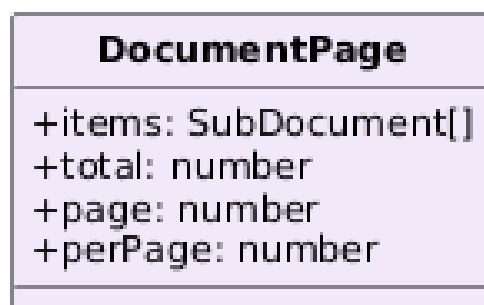


Figura 105: Tipo: DocumentPage

6.3.2.7 DocumentPreviewStatus

Stato dell'anteprima PDF di un sotto-documento, dal caricamento all'eventuale indisponibilità.

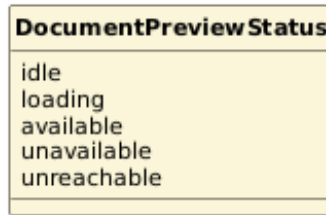


Figura 106: Tipo: DocumentPreviewStatus

6.3.2.8 DocumentFilterFormValue

Tipo derivato dal `FormGroup` dei filtri tramite `ReturnType`, così i due restano sincronizzati senza doverli dichiarare due volte.

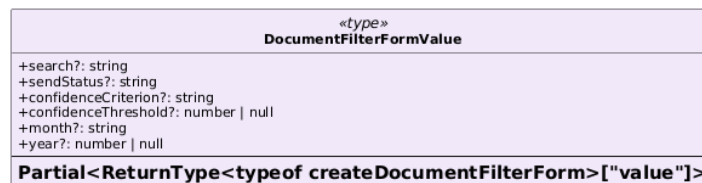


Figura 107: Tipo: DocumentFilterFormValue

6.3.2.9 createDocumentFilterForm e toDocumentFilters

Le due funzioni che costruiscono il `FormGroup` dei filtri e ne convertono il valore nei criteri inviati all'API.

Metodi

- `createDocumentFilterForm(): FormGroup`
- `toDocumentFilters(value: DocumentFilterFormValue): DocumentFilters`

6.4 Servizi di comunicazione con il backend

Ogni feature ha un servizio dedicato (`AssistantService`, `DocumentWorkflowService`) che incapsula le chiamate al client generato e aggiorna lo Store con la risposta autorevole. Nessun componente né `ViewModel` chiama direttamente il client HTTP: passa sempre dal servizio della propria feature, ricevuto come dipendenza dal costruttore del `ViewModel` (Sezione 6.2).

6.4.1 AssistantService

Il servizio più corposo del frontend, perché incapsula anche il consumo dello stream SSE lato client, simmetrico a quello che il backend espone (Sezione 3.2). `trackGeneration()` avvolge la chiamata HTTP iniziale e la connessione SSE risultante in un unico `Observable`: chi lo sottoscrive riceve una sequenza di stati di avanzamento (*queued* → testo → copertina → *completed/failed*) senza dover gestire direttamente gli eventi SSE.

La meccanica di basso livello (apertura dello stream, parsing dei frame, distinzione fra evento nominato **error** e caduta della connessione) vive in **SseClient** (più avanti in questa sezione); questo servizio resta responsabile solo dell'orchestrazione specifica di feature (interpretazione degli eventi *progress/text/cover/done*, aggiornamento dello Store).

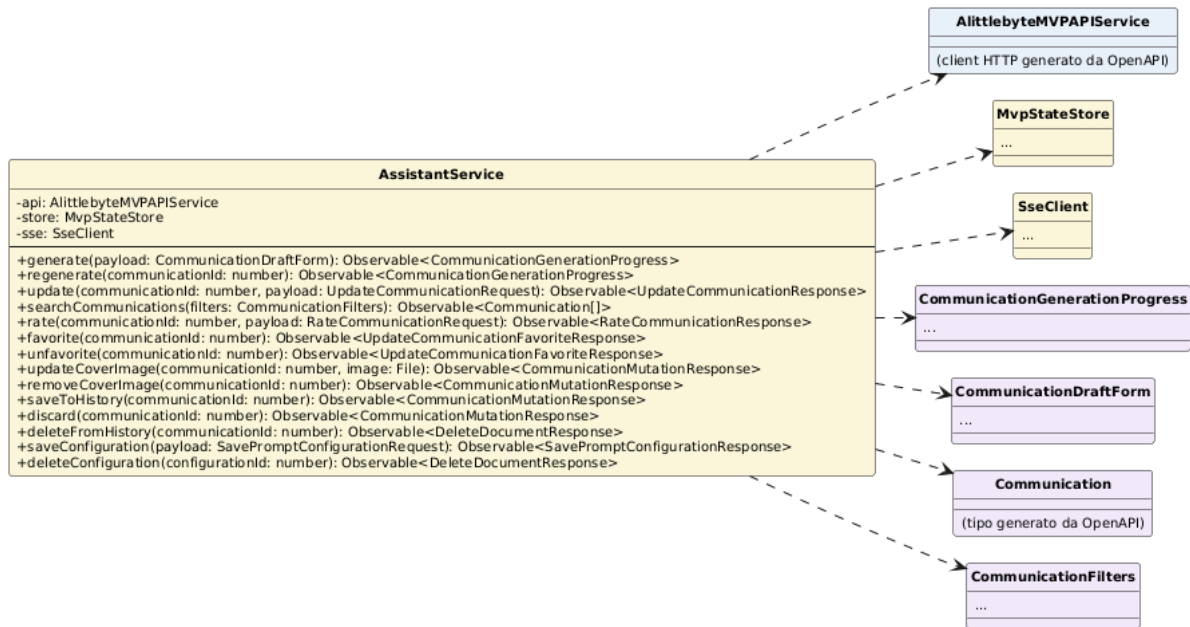


Figura 108: Classe: AssistantService

Attributi

- **api:** AlittlebyteMVPAPIService, **store:** MvpStateStore, **sse:** SseClient.

Metodi

- **generate(payload: CommunicationDraftForm):**
Observable<CommunicationGenerationProgress>
- **regenerate(communicationId: number):**
Observable<CommunicationGenerationProgress>
- **update(communicationId: number, payload: UpdateCommunicationRequest):**
Observable<UpdateCommunicationResponse>: usato da AssistantPageViewModel.saveDraft() per correggere titolo e corpo della bozza.
- **searchCommunications(filters: CommunicationFilters):**
Observable<Communication[]>
- **rate(communicationId: number, payload: RateCommunicationRequest):**
Observable<RateCommunicationResponse>
- **favorite(communicationId: number):**
Observable<UpdateCommunicationFavoriteResponse>

- `unfavorite(communicationId: number): Observable<UpdateCommunicationFavoriteResponse>`
- `updateCoverImage(communicationId: number, image: File): Observable<CommunicationMutationResponse>`
- `removeCoverImage(communicationId: number): Observable<CommunicationMutationResponse>`
- `saveToHistory(communicationId: number): Observable<CommunicationMutationResponse>`
- `discard(communicationId: number): Observable<CommunicationMutationResponse>`
- `deleteFromHistory(communicationId: number): Observable<DeleteDocumentResponse>`
- `saveConfiguration(payload: SavePromptConfigurationRequest): Observable<SavePromptConfigurationResponse>`
- `deleteConfiguration(configurationId: number): Observable<DeleteDocumentResponse>`

6.4.2 DocumentWorkflowService

Equivalente Co-Pilot di `AssistantService`, con lo stesso incapsulamento dello stream SSE lato client. `upload()` avvolge la chiamata HTTP di caricamento e la connessione SSE risultante in un unico `Observable`, esattamente come `trackGeneration()`: anche qui la meccanica di basso livello è delegata a `SseClient` (sottosezione successiva).

`previewStatus()` è invece un caso specifico di questo modulo, assente sul dominio Communications e indipendente da SSE: verifica il *content-type* della risposta prima di montare l'iframe di anteprima, perché l'endpoint può restituire il PDF (200), un 404 se il file non è ancora disponibile o un 503 JSON se lo storage non è raggiungibile: tre esiti che il componente chiamante deve distinguere per mostrare lo stato corretto.

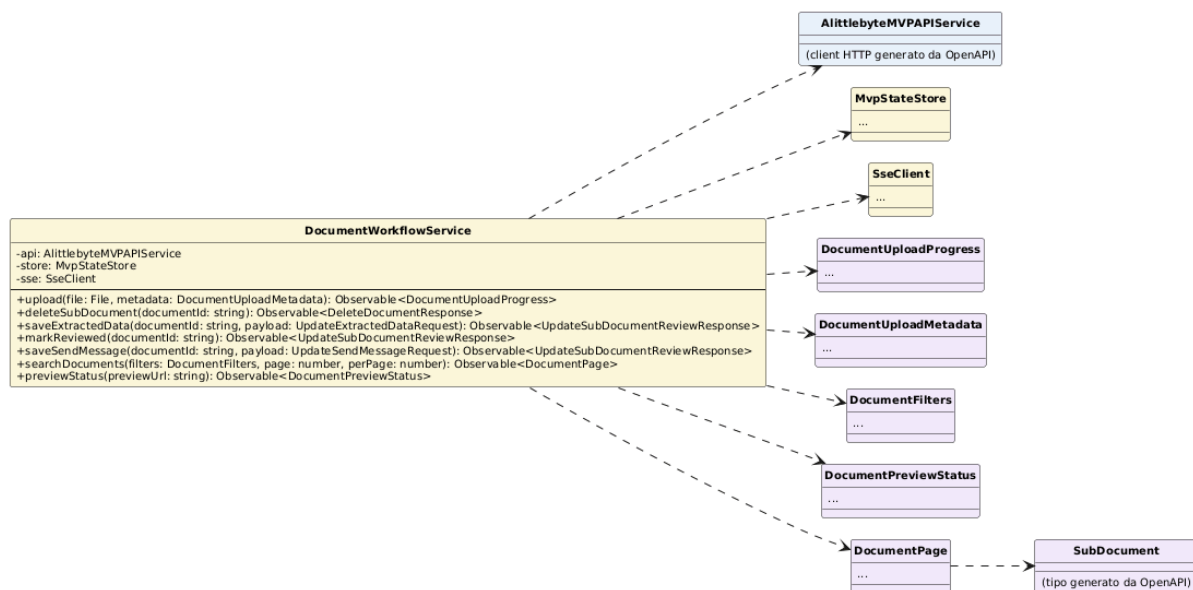


Figura 109: Classe: DocumentWorkflowService

Attributi

- `api`: `AlittlebyteMVPAPIService`, `store`: `MvpStateStore`, `sse`: `SseClient`.

Metodi

- `upload(file: File, metadata: DocumentUploadMetadata): Observable<DocumentUploadProgress>`
- `deleteSubDocument(documentId: string): Observable<DeleteDocumentResponse>`
- `saveExtractedData(documentId: string, payload: UpdateExtractedDataRequest): Observable<UpdateSubDocumentReviewResponse>`
- `markReviewed(documentId: string): Observable<UpdateSubDocumentReviewResponse>`
- `saveSendMessage(documentId: string, payload: UpdateSendMessageRequest): Observable<UpdateSubDocumentReviewResponse>`
- `searchDocuments(filters: DocumentFilters, page: number, perPage: number): Observable<DocumentPage>`: restituisce la pagina intera (`items` più `total`), non i soli elementi: senza il totale la vista non può sapere quante pagine esistono.
- `previewStatus(previewUrl: string): Observable<DocumentPreviewStatus>`

6.4.3 SseClient

Client SSE condiviso da entrambi i servizi di feature, iniettabile a livello applicazione (`providedIn: "root"`). A differenza di `EventSource`, è basato su `fetch`: può quindi inviare gli header di correlazione (`X-Request-ID`, `X-Correlation-ID`) sulla richiesta iniziale di apertura dello stream, cosa che `EventSource` non permette.

Distingue esplicitamente l'evento SSE nominato `error` (dati applicativi, es. un messaggio di errore) dalla caduta della connessione o da una risposta non riuscita: due condizioni che `EventSource` collassa in un unico `onerror`, costringendo il chiamante a indovinare quale sia successo.

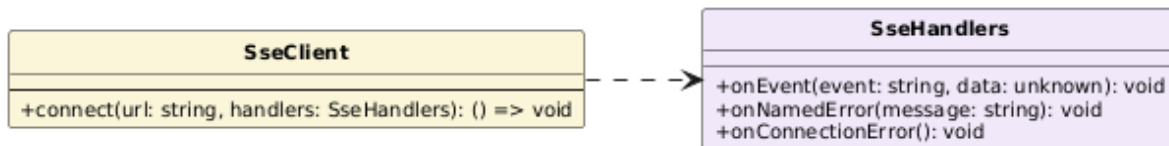


Figura 110: Classe: SseClient

Attributi

- Nessuno: nessuno stato d'istanza, ogni chiamata a `connect()` è indipendente.

Metodi

- `connect(url: string, handlers: SseHandlers): () => void`: apre lo stream e restituisce una funzione di chiusura (basata su `AbortController`), da invocare alla disiscrizione.

6.5 Interceptor HTTP e servizi trasversali

Tre `HttpInterceptorFn` registrati globalmente, eseguiti su ogni richiesta verso il backend, e tre servizi di supporto: due dichiarati `providedIn: "root"` e uno registrato esplicitamente come gestore degli errori dell'applicazione.

`apiBaseUrlInterceptor` premette `environment.apiUrl` alle sole richieste applicative (`/api`, `/health`, `/ready`) quando quella variabile è configurata; di default è vuota, quindi SPA e API condividono la stessa origine dietro il reverse proxy e l'interceptor è un no-op. Resta l'unico punto in cui puntare il frontend a un backend su un'origine diversa.

`correlationInterceptor` genera un identificativo di correlazione e lo propaga verso Laravel come header `X-Request-ID/X-Correlation-ID` su ogni richiesta che non ne porti già uno: il middleware `CorrelateRequests` del backend (Sezione 4.3.2) lo riusa se presente, così una richiesta originata dalla SPA si ritrova negli stessi log applicativi lato server. Solo header tecnici, nessun dato sensibile.

`errorLoggingInterceptor` traccia in modo strutturato ogni fallimento HTTP (status, metodo, endpoint privato della query string, correlation id) tramite `LoggerService`, poi rilancia l'errore invariato: la gestione verso l'utente resta a carico della feature che ha fatto la richiesta. Non registra mai il corpo della risposta, per evitare che un payload finisca nei log.

LoggerService è il logger applicativo del frontend, volutamente parco: accetta solo metadati tecnici (endpoint, stato HTTP, correlation id), mai token, header di autenticazione, PII o corpi di richiesta/risposta. In produzione filtra i messaggi di livello **info**, mantenendo solo **warn/error** per ridurre il rumore.

GlobalErrorHandler sostituisce l'**ErrorHandler** di default di Angular per gli errori non catturati altrove. Gli errori HTTP sono già tracciati dall'interceptor dedicato: per evitare un doppio log, questo gestore li riconosce e li ignora, registrando solo ciò che sfuggirebbe altrimenti (errori di rendering o di logica applicativa).

ThemeService gestisce il tema chiaro/scuro: preferenza persistita in **localStorage** con fallback su **prefers-color-scheme** al primo avvio, applicata come attributo **data-mvp-theme** su **<html>**, su cui agiscono i token CSS. Se **localStorage** non è disponibile (es. modalità privata) la preferenza semplicemente non persiste fra sessioni, senza errori visibili.



Figura 111: Classi: **apiBaseUrlInterceptor**, **correlationInterceptor**, **errorLoggingInterceptor**, **LoggerService**, **GlobalErrorHandler**, **ThemeService**

Attributi

- **Interceptor**: nessuno stato, sono funzioni pure di richiesta/risposta.
- **LoggerService**: nessuno stato osservabile dall'esterno.
- **GlobalErrorHandler**: **logger**: **LoggerService** (iniettato).
- **ThemeService**: **theme**: **Signal<"light" | "dark">** (sola lettura, sorgente del tema corrente).

Metodi

- **LoggerService**: **info(message, context)**, **warn(message, context)**, **error(message, context)**.
- **GlobalErrorHandler**: **handleError(error: unknown): void**, dall'interfaccia **ErrorHandler** di Angular.
- **ThemeService**: **toggle(): void**, alterna fra i due temi e persiste la scelta.

6.6 Serializzazione e deserializzazione dei dati

La serializzazione verso e dal backend è interamente generata: Orval crea, insieme alle interfacce dei modelli, anche le funzioni client che serializzano il corpo delle richieste e deserializzano le risposte JSON secondo i tipi del contratto OpenAPI. Non esiste quindi un adapter di serializzazione scritto a mano da documentare: la conformità fra client e contratto è verificata automaticamente in CI (Sezione 2.10, **openapi:check**), che rigenera il client e fallisce se il risultato differisce da quello committato.

6.7 Micro componenti UI

I componenti condivisi (`shared/components`) sono privi di logica di dominio: ricevono dati tramite segnali di input e riemettono eventi tramite `output()`, senza conoscere da quale feature vengono usati. Sono ventuno in tutto: `ButtonComponent`, `AlertComponent`, `ErrorStateComponent`, `EmptyStateComponent`, `LoadingStateComponent`, `StatusBadgeComponent`, `StatusDotComponent`, `AttentionNoteComponent`, `MetricCardComponent`, `MetricsPanelComponent`, `MetricShareComponent`, `MetricGaugeComponent`, `MetricStatusComponent`, `MetricBreakdownComponent`, `MetricStarsComponent`, `MetricCompositionComponent`, `StageProgressComponent`, `StarRating`, `SelectFieldComponent`, `TextAreaFieldComponent`, `FileDropzoneComponent`. La cartella `data-table` contiene solo un foglio di stile condiviso, non un componente: le tabelle restano markup HTML nativo nelle singole pagine, non un elemento riutilizzabile a sé.

`MetricsPanelComponent` è un dispatcher con più componenti di presentazione dedicati, ciascuno la risposta visiva a un tipo diverso di domanda che una metrica può porre (un andamento, una quota, una posizione su scala, una ripartizione...): cinque forme dedicate oltre a quella di default (Sezione successiva). `StarRating` vive in `shared/components` con due punti di utilizzo, entrambi nel modulo Assistant: la valutazione della bozza in anteprima e l'elenco delle bozze valutate di recente. La scheda `assistant.rating_average` del pannello metriche usa invece `MetricStarsComponent`.

6.7.1 ButtonComponent

Il più semplice dei componenti condivisi: non introduce un nuovo elemento HTML, ma si applica come attributo su un `<button>` nativo (`<button mvpButton variant="secondary">`). In questo modo il componente lascia al browser tipo, stato, disabilitato, attributi ARIA ed eventi nativi, occupandosi solo della variante di stile applicata all'host. `busy` segnala che il comando è in corso senza toccare l'etichetta, che resta ferma invece di cambiare a metà clic; `aria-busy` porta la stessa informazione alle tecnologie assistive.

ButtonComponent
+variant: «Input» "primary" "secondary" "icon"
+type: «Input» string
+busy: «Input» boolean

Figura 112: Classe: ButtonComponent

Attributi

- **variant:** `InputSignal<"primary" | "secondary" | "icon">`: variante di stile, di default *primary*.
- **type:** `InputSignal<string>`: tipo dell'attributo HTML `type`, di default *button*.
- **busy:** `InputSignal<boolean>`: default *false*; mostra uno spinner sull'host e imposta `aria-busy`, senza sostituire il contenuto proiettato.

Metodi

- Nessuno: il comportamento è interamente dichiarativo, espresso nei binding `host` del decoratore `@Component`.

6.7.2 AlertComponent, ErrorStateComponent

`AlertComponent` è il riquadro di base per un messaggio con titolo, usato ovunque serva segnalare qualcosa con enfasi (`role="alert"`, `aria-live="polite"`).

`ErrorStateComponent` non duplica quel markup: lo compone, fissando un titolo di default ("*Servizio non disponibile*") e passando il messaggio d'errore come contenuto proiettato: è il componente che compare in cima a ogni pagina quando lo stato di errore è valorizzato. Espone anche un pulsante di ripetizione opzionale ("*Riprova*"), renderizzato solo quando `canRetry()` è vero: alla pressione emette l'evento `retry`, lasciando al chiamante decidere cosa ritentare.

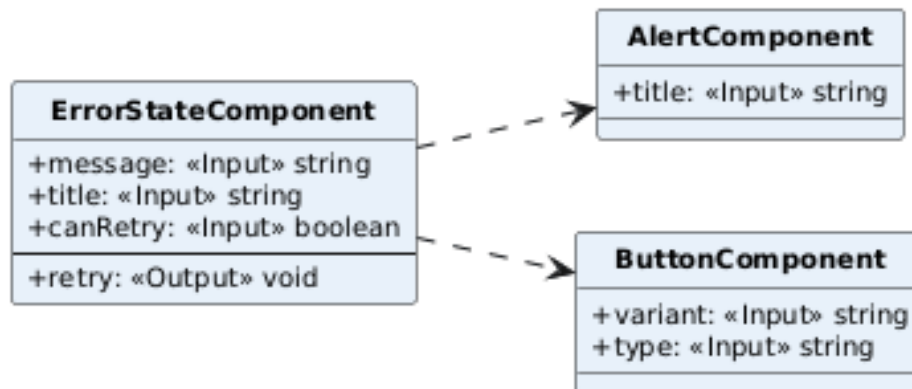


Figura 113: Classi: `AlertComponent`, `ErrorStateComponent`

Attributi

- `AlertComponent`: `title`: `InputSignal<string>` (obbligatorio).
- `ErrorStateComponent`: `message`: `InputSignal<string>` (obbligatorio), `title`: `InputSignal<string>` (default "*Servizio non disponibile*"), `canRetry`: `InputSignal<boolean>` (default *false*).

Metodi

- `ErrorStateComponent`: `retry`: `OutputEmitterRef<void>`: emesso al click sul pulsante "Riprova", visibile solo se `canRetry()` è vero.
- `AlertComponent`: nessuno.

6.7.3 EmptyStateComponent, LoadingStateComponent, StatusBadgeComponent, StatusDotComponent

Quattro componenti minimi di visualizzazione, senza alcuna logica: si limitano a proiettare contenuto o un'etichetta dentro un elemento con lo stile e la semantica ARIA corretti (`role="status"` per il caricamento, colori diversi per tono sull'etichetta di stato). `StatusDotComponent` è un indicatore binario (fatto/non fatto) da affiancare a `StatusBadgeComponent` senza confondersi con esso, perché rispondono a due domande diverse: l'etichetta dice in quale dei quattro stati di revisione si trova un sotto-documento, il pallino se è stato scaricato o no. Pieno contro vuoto non è il solo colore a portare l'informazione (SC 1.4.1): il pallino è comunque `aria-hidden`, perché ripete un'informazione che il testo proiettato accanto dice già.

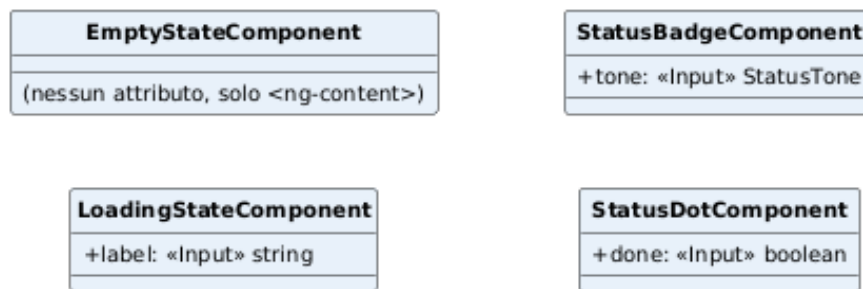


Figura 114: Classi: `EmptyStateComponent`, `LoadingStateComponent`, `StatusBadgeComponent`, `StatusDotComponent`

Attributi

- `EmptyStateComponent`: nessuno, solo `<ng-content>`.
- `LoadingStateComponent`: `label`: `InputSignal<string>` (default *"Caricamento in corso."*).
- `StatusBadgeComponent`: `tone`: `InputSignal<StatusTone>` (*info, success, warning, danger*, default *neutral*).
- `StatusDotComponent`: `done`: `InputSignal<boolean>` (default *false*).

Metodi

- Nessuno.

6.7.4 AttentionNoteComponent

Riga che segnala qualcosa su cui agire, con l'azione che la risolve: una scheda metrica dice *quanto* ("23 da verificare"), questo componente dice *cosa fare*, con un pulsante opzionale che porta alla coda interessata. Va montato solo quando c'è davvero qualcosa da fare (`OverviewPage` lo mostra solo se `vm.quarantined() > 0`, Sezione 6.9): mostrato sempre diventerebbe arredamento, e si smetterebbe di leggerlo.

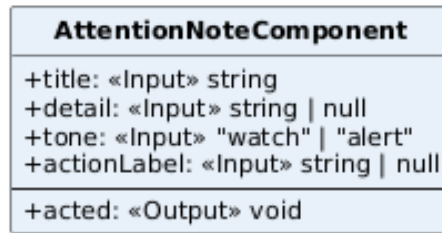


Figura 115: Classe: AttentionNoteComponent

Attributi

- **title:** `InputSignal<string>` (obbligatorio), **detail:** `InputSignal<string | null>` (default *null*).
- **tone:** `InputSignal<"watch" | "alert">` (default *watch*): cambia glifo e colore insieme, mai il solo colore.
- **actionLabel:** `InputSignal<string | null>` (default *null*): quando valorizzato monta il pulsante d'azione.

Metodi

- **acted:** `OutputEmitterRef<void>`: emesso al click sul pulsante d'azione.

6.7.5 MetricCardComponent, MetricsPanelComponent

MetricCardComponent mostra la forma di scheda di default (un conteggio e il proprio andamento a sette giorni). **MetricsPanelComponent** è un dispatcher: per ciascuna metrica ricevuta sceglie, in base alla mappa **presentation** passata dalla pagina (il tipo **MetricPresentation**, Sezione 6.2), quale fra sei componenti di presentazione montare (Sezione successiva): **MetricCardComponent** resta la scelta di default quando la mappa non specifica un **kind** diverso. Gestisce inoltre da sé gli stati della richiesta a monte: caricamento (schede segnaposto con **isLoading**), errore (messaggio dedicato, distinto da "nessuna metrica": in errore i dati esistono, è la lettura ad essere fallita) o elenco vuoto (**EmptyStateComponent**). Montato da **AssistantPage** e **CopilotPage**; **OverviewPage** non lo usa (Sezione 6.9).

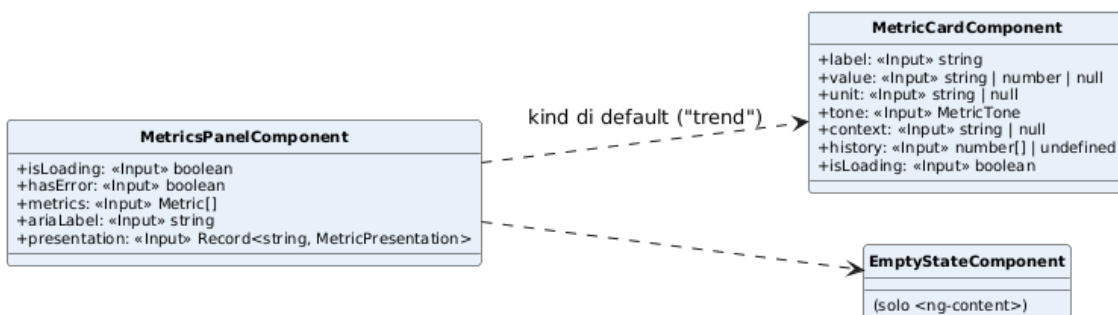


Figura 116: Classi: MetricCardComponent, MetricsPanelComponent

Attributi

- **MetricCardComponent**: `label`: `InputSignal<string>` (obbligatorio), `value`: `InputSignal<string | number | null>` (obbligatorio, `null` distinto da zero: uno zero durante il caricamento affermerebbe un dato che non c'è ancora), `unit`, `tone`, `context`, `history`: `InputSignal<readonly number[] | undefined>` (flusso degli ultimi sette giorni, disegnato a barre, non a linea, perché sono ingressi distinti, non un valore continuo), `isLoading`.
- **MetricsPanelComponent**: `isLoading`: `InputSignal<boolean>` (default *false*), `hasError`: `InputSignal<boolean>` (default *false*), `metrics`: `InputSignal<Metric[]>` (default `[]`), `ariaLabel`: `InputSignal<string>` (default *"Metriche"*), `presentation`: `InputSignal<Record<string, MetricPresentation>>` (default `{}`): forma, rilievo e ingombro nella griglia per ciascuna chiave di metrica, decisi dalla pagina che conosce il contesto, non dal pannello.

Metodi

- Nessuno.

6.7.6 Le forme di scheda del pannello metriche

Le cinque forme di presentazione oltre a quella di default, ciascuna la risposta a un tipo diverso di domanda che una metrica può porre: **MetricShareComponent** (quanta parte di un totale, con l'anello che colora anche il residuo perché è quello il numero su cui agire), **MetricGaugeComponent** (dove cade un valore dentro una scala nota, con la tacca della soglia oltre la quale il sistema decide da solo), **MetricStatusComponent** (c'è qualcosa da guardare? Il verdetto prima del numero, perché un conteggio di guasti vale quasi sempre zero e uno zero grande quanto un totale chiederebbe di fermarsi a capire), **MetricBreakdownComponent** (come si divide un insieme fra gli stati in cui si trova, un anello a più colori invece della quota a due) e **MetricStarsComponent** (una media su una scala a stelle, con l'ultima riempita per la frazione reale e non arrotondata). Tutte condividono lo stesso stato di caricamento (`isLoading`, schede segnaposto) e lo stesso involucro visivo (`shared/styles/metric-shell.css`), montate dal dispatcher **MetricsPanelComponent** in base al `kind` dichiarato nella mappa `presentation` della pagina, oppure direttamente da una pagina che ne abbia bisogno fuori dal pannello. `copilot.processing_seconds` è una singola media in secondi (`kind trend`, come un conteggio qualunque): il modulo Communications non ha una metrica equivalente, la durata della generazione non è oggi esposta come voce del contratto `/api/v1/state`. Nessuna delle due pagine usa **MetricGaugeComponent**, **MetricStatusComponent** o **MetricBreakdownComponent** tramite il pannello: restano forme valide del dispatcher, e **MetricGaugeComponent** è montato direttamente da **OverviewPage** (Sezione 6.9) per la qualità del Co-Pilot, insieme a **MetricStarsComponent** per quella dell'Assistant.

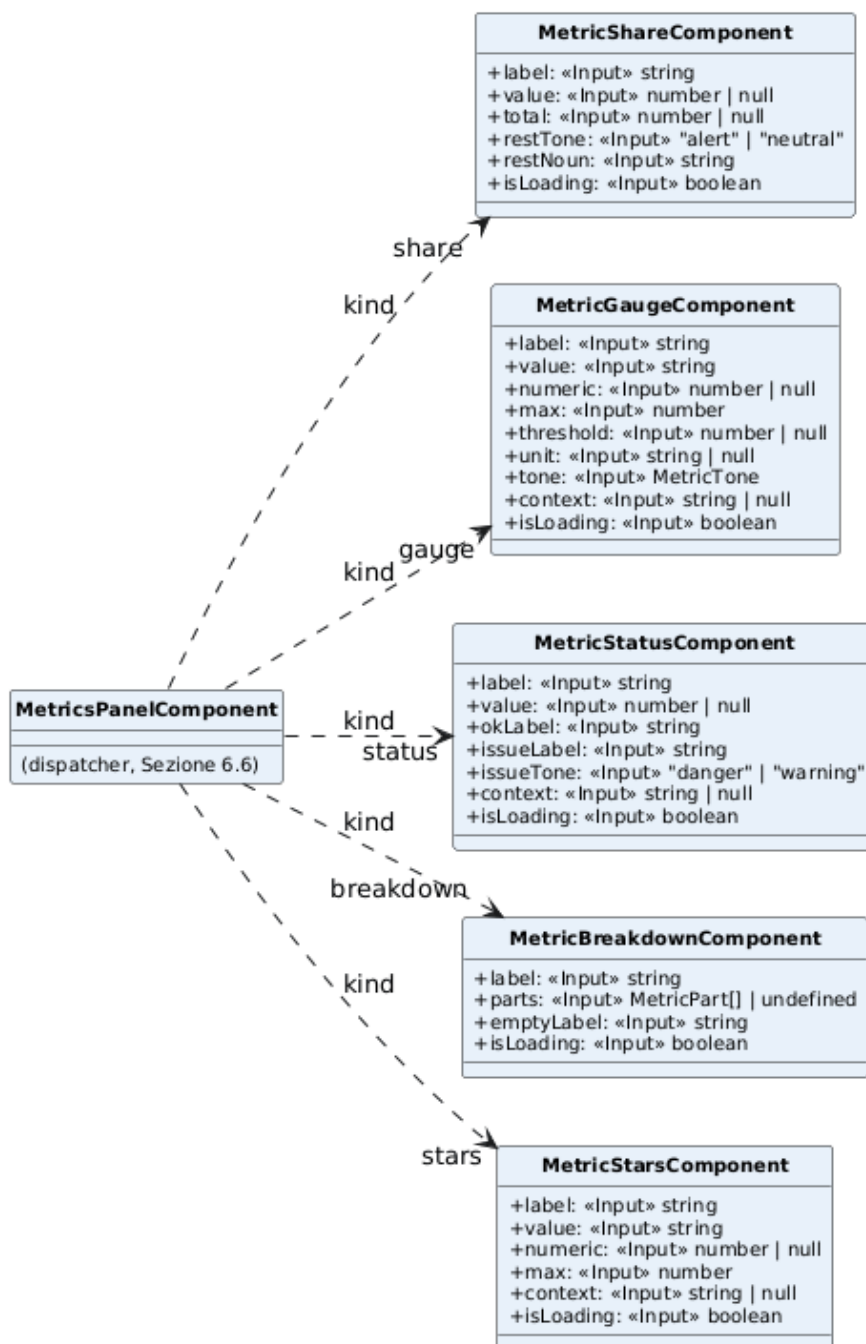


Figura 117: Classi: le cinque forme di scheda disacciate da MetricsPanelComponent

Attributi

- **MetricShareComponent:** `label`, `value`: `InputSignal<number | null>`, `total`: `InputSignal<number | null>` (obbligatori), `restTone`: `InputSignal<"alert" | "neutral">` (default *alert*: distingue un residuo che chiede un intervento da uno che è solo il complemento di una misura), `restNoun`, `isLoading`.
- **MetricGaugeComponent:** `label`, `value`: `InputSignal<string>` (obbligatori, già formattato), `numeric`: `InputSignal<number | null>` (posiziona il

riempimento), max (default 100), threshold: `InputSignal<number | null>`, unit, tone, context, isLoading.

- **MetricStatusComponent**: label, value: `InputSignal<number | null>` (obbligatori), okLabel (default *"Nessun caso"*), issueLabel (default *"da esaminare"*), issueTone: `InputSignal<"danger" | "warning">` (default *danger*: un guasto blocca, un degrado no), context, isLoading.
- **MetricBreakdownComponent**: label: `InputSignal<string>` (obbligatorio), parts: `InputSignal<readonly MetricPart[] | undefined>`, emptyLabel, isLoading.
- **MetricStarsComponent**: label, value: `InputSignal<string>` (obbligatori, già formattato), numeric: `InputSignal<number | null>` (riempie le stelle), max (default 5), context, isLoading.

Metodi

- Nessuno: ciascuna deriva internamente solo i segnali **computed** necessari al proprio disegno (l'arco dell'anello, il riempimento della scala, i segmenti della barra...), senza superficie pubblica oltre agli `input()`.

6.7.7 MetricCompositionComponent

Barra segmentata di una partizione completa, usata dove i conteggi affiancati nasconderebbero le proporzioni: la ripartizione delle bozze fra "in bozza" e "decise" in **AssistantPage** (`vm.draftComposition()`, Sezione 6.2) è la sola istanza attuale, montata fuori da **MetricsPanelComponent** perché non è una metrica di per sé, ma una vista sulla stessa metrica `assistant.total/assistant.drafts` già mostrata come scheda. Le fette usano una scala monocromatica derivata dal colore primario, non colori di stato: verde e rosso qui suggerirebbero un giudizio che una composizione non esprime.

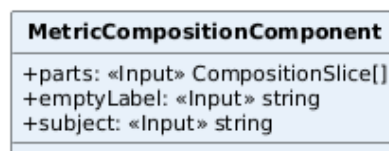


Figura 118: Classe: MetricCompositionComponent

Attributi

- **parts**: `InputSignal<readonly CompositionSlice[]>` (obbligatorio): coppie {label, value}.
- **emptyLabel**: `InputSignal<string>` (default *"Nessun elemento da ripartire."*), **subject**: `InputSignal<string>` (default *"elementi"*): nome della grandezza ripartita, usato nella descrizione accessibile.

Metodi

- Nessuno.

6.7.8 StageProgressComponent

Avanzamento per tappe, non a percentuale: le tappe (`ProgressStage`, `{id, label}`) sono gli stati reali che arrivano dallo stream SSE, quindi l'avanzamento è mostrato senza inventare una quantità: un documento non è mai "al 60%" di niente; un contatore del tempo trascorso scorre accanto, perché senza di esso un'attesa lunga è indistinguibile da un blocco. Condiviso fra le pipeline asincrone di entrambi i moduli tramite due adattatori di feature (`GenerationProgressComponent` per l'Assistant, quattro tappe; `UploadProgressComponent` per il Co-Pilot, cinque), che mappano le rispettive fasi di dominio sull'elenco di `ProgressStage` ricevuto.

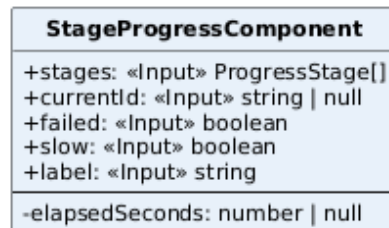


Figura 119: Classe: StageProgressComponent

Attributi

- `stages`: `InputSignal<readonly ProgressStage[]>` (obbligatorio), `currentId`: `InputSignal<string | null>` (default *null*, tappa raggiunta), `failed`, `slow` (annotazione sulla tappa corrente, non una tappa a sé: la pipeline resta nello stesso stato, solo da più tempo del previsto), `label` (default *"Avanzamento elaborazione"*).
- `elapsedSeconds`: `Signal<number | null>`: cronometro interno, azzerato quando l'avanzamento torna indietro o riparte dopo essersi fermato: altrimenti una corsa nuova erediterebbe il tempo di quella precedente.

Metodi

- Nessuno: la sola logica è l'`effect()` nel costruttore che aggiorna il cronometro, interno al componente.

6.7.9 StarRating

Valutazione a stelle come gruppo di radio ARIA (il pattern che l'APG indica per una scala a valori discreti): le frecce scorrono i punteggi, il gruppo ha un nome, e ogni valore ne annuncia uno proprio ("3 stelle su 5"). Il disegno è un SVG disegnato due volte (contorno e riempimento ritagliato), così non dipende dal glifo di sistema; il contorno resta visibile in entrambi gli stati, cosicché chi non distingue i colori legga comunque quante stelle sono piene (SC 1.4.1).

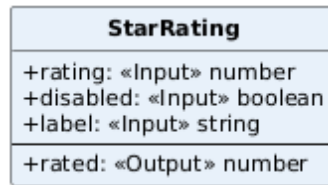


Figura 120: Classe: StarRating

Attributi

- **rating:** `InputSignal<number>` (default 0), **disabled:** `InputSignal<boolean>` (default *false*), **label:** `InputSignal<string>` (default *"Valutazione da 1 a 5 stelle"*).

Metodi

- **rated:** `OutputEmitterRef<number>`: emesso alla selezione di un punteggio; il punteggio mostrato resta guidato dal chiamante, il componente non lo applica da sé.

6.7.10 SelectFieldComponent, TextAreaFieldComponent

I due componenti di campo condivisi dai form reattivi della SPA, con la stessa struttura: un'etichetta associata via `<label for>`, un identificativo generato automaticamente dal testo dell'etichetta se non fornito esplicitamente, e un `FormControl` ricevuto come segnale di input invece che tramite `ControlValueAccessor`: una scelta più semplice, sufficiente perché nessuno dei due deve integrarsi con `ngModel`.

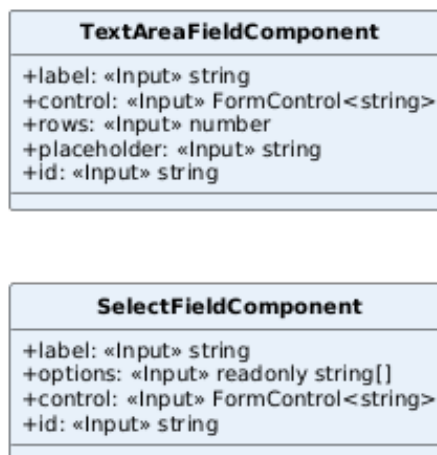


Figura 121: Classi: SelectFieldComponent, TextAreaFieldComponent

Attributi

- **SelectFieldComponent:** **label,** **options:** `InputSignal<readonly string[]>`, **control:** `InputSignal<FormControl<string>>` (tutti obbligatori), **id:** `InputSignal<string>` (opzionale).

- `TextAreaFieldComponent`: `label`, `control` (obbligatori), `rows`: `InputSignal<number>` (default 4), `placeholder`, `id` (opzionali).

Metodi

- Nessuno.

6.7.11 FileDropzoneComponent

Area di trascinalamento/selezione file per il caricamento documenti nel Co-Pilot, ristretta ai soli PDF (`accept="application/pdf"`). Ripulisce il valore dell'<input> subito dopo aver emesso il file selezionato, così lo stesso file può essere ricaricato una seconda volta senza che il browser lo consideri un valore invariato e sopprima l'evento `change`.

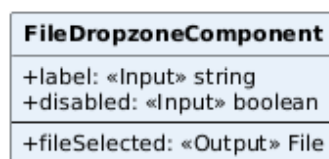


Figura 122: Classe: FileDropzoneComponent

Attributi

- `label`: `InputSignal<string>` (obbligatorio), `disabled`: `InputSignal<boolean>` (default *false*).

Metodi

- `fileSelected`: `OutputEmitterRef<File>`: evento emesso alla selezione di un file valido.

6.8 Componenti di feature

6.8.1 Componenti di feature dell'AI Assistant Generativo

A differenza dei componenti condivisi (Sezione 6.7), questi conoscono il dominio Communications: ricevono `input()` tipizzati sui modelli generati dal contratto OpenAPI (es. `Communication`, `PromptConfiguration`) e rimettono eventi verso `AssistantPage`, che li traduce in chiamate a `AssistantPageViewModel`. Nessuno di loro chiama direttamente `AssistantService` o lo Store: restano *dumb*, coerentemente con il pattern MVVM di Sezione 6.2.

`CommunicationGeneratorPanelComponent` è il form di generazione (prompt, tono, stile) e ospita anche il salvataggio di una configurazione di prompt riutilizzabile (UC-19): un `effect()` interno applica al form i valori di `prefill()` quando l'utente sceglie di riusare una configurazione salvata, e un secondo `effect()` richiude da solo il modulo del nome quando l'elenco delle configurazioni si allunga (salvataggio riuscito), distinguendo quel caso da un nuovo riferimento allo stesso array prodotto da una mutazione qualunque altrove nella pagina.

GeneratedCommunicationPreviewComponent è il più corposo del modulo: mostra la bozza generata, la sua copertina (con upload/rimozione), il testo in sola lettura o modificabile, l'esportazione e la valutazione a stelle. Tre **effect()** interni tengono sincronizzati, rispetto alla bozza ricevuta in ingresso, lo stato di conferma scarto, il form di modifica e il punteggio mostrato, ciascuno confrontando il contenuto rilevante (non il solo riferimento all'oggetto) per non perdere una modifica non salvata quando il backend rimpiazza l'intero stato condiviso per una mutazione che riguarda un'altra voce dello storico. La valutazione, una volta assegnata, disabilita stelle e commento (Sezione 3.2): il componente non permette di inviarne una seconda.

CommunicationHistoryListComponent e **PromptConfigurationListComponent** sono i due elenchi laterali (storico comunicazioni e configurazioni salvate): stessa forma, selezione/uso più eliminazione con conferma a due passi, che il componente dichiara con un evento (**confirmDeleteRequested**) invece di gestire da sé, lasciando lo stato del passo di conferma al ViewModel della pagina.

RecentFeedbackListComponent è in sola lettura (UC-27.3): mostra le ultime bozze valutate con voto e commento testuale, senza selezione né azioni, a differenza dello storico.

GenerationProgressComponent è un adattatore verso **StageProgressComponent** (Sezione 6.7): traduce le quattro fasi reali della pipeline di generazione (in coda, testo, copertina, completata), più gli stati **still_running/failed** che restano ancorati alla tappa "copertina", nella forma richiesta dal componente condiviso di avanzamento.

CommunicationGeneratorPanelComponent	GeneratedCommunicationPreviewComponent
+isGenerating: «Input» boolean +status: «Input» string +phase: «Input» CommunicationGenerationPhase "idle" null +promptConfigurations: «Input» PromptConfiguration[] +isSavingConfiguration: «Input» boolean +saveConfigurationError: «Input» string null +prefill: «Input» CommunicationDraftForm null +generate: «Output» CommunicationDraftForm +saveConfiguration: «Output» SavePromptConfigurationRequest	+draft: «Input» GeneratedDraft null +isUpdatingCover: «Input» boolean +isGenerating: «Input» boolean +isDiscarding: «Input» boolean +isSavingToHistory: «Input» boolean +isRating: «Input» boolean +rateError: «Input» string null +isSaving: «Input» boolean +saveError: «Input» string null +uploadCover: «Output» File +removeCover: «Output» void +regenerate: «Output» void +discard: «Output» void +saveToHistory: «Output» void +rate: «Output» RateDraftPayload +saveRequested: «Output» { communicationId, payload }

Figura 123: Classi: CommunicationGeneratorPanelComponent, GeneratedCommunicationPreviewComponent

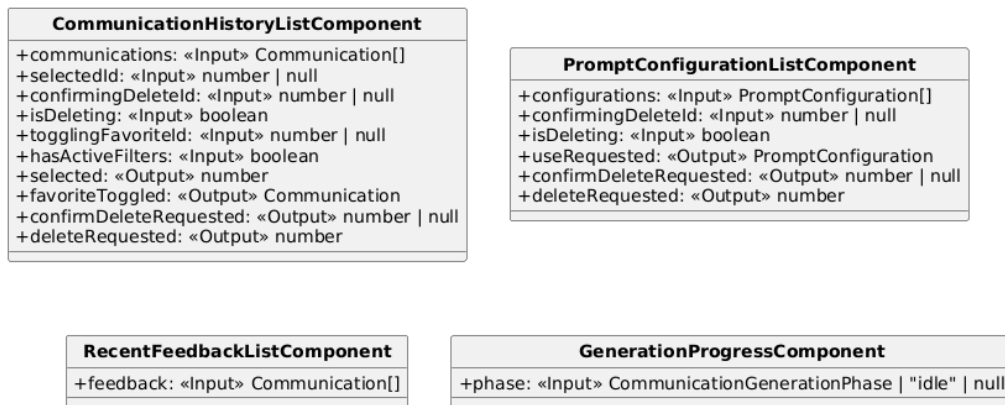


Figura 124: Classi: CommunicationHistoryListComponent, PromptConfigurationListComponent, RecentFeedbackListComponent, GenerationProgressComponent

Attributi

- **CommunicationGeneratorPanelComponent**: `isGenerating`, `status`, `phase` (obbligatori); `promptConfigurations` (default `[]`), `isSavingConfiguration` (default `false`), `saveConfigurationError` (default `null`), `prefill`: `InputSignal<CommunicationDraftForm | null>` (valori da precaricare nel form al riuso di una configurazione, UC-19).
- **GeneratedCommunicationPreviewComponent**: `draft`: `InputSignal<GeneratedDraft | null>` (default `null`), `isUpdatingCover`, `isGenerating`, `isDiscarding`, `isSavingToHistory`, `isRating`, `rateError`, `isSaving`, `saveError` (tutti opzionali, di default `false`/null).
- **CommunicationHistoryListComponent**: `communications` (default `[]`), `selectedId`, `confirmingDeleteId`, `isDeleting`, `togglingFavoriteId`, `hasActiveFilters` (cambia il messaggio di elenco vuoto fra "nessun risultato" e "niente da mostrare").
- **PromptConfigurationListComponent**: `configurations` (default `[]`), `confirmingDeleteId`, `isDeleting`.
- **RecentFeedbackListComponent**: `feedback`: `InputSignal<Communication[]>` (default `[]`).
- **GenerationProgressComponent**: `phase`: `InputSignal<CommunicationGenerationPhase | "idle" | null>` (obbligatorio).

Metodi

- **CommunicationGeneratorPanelComponent**: `generate`: `OutputEmitterRef<CommunicationDraftForm>`, `saveConfiguration`: `OutputEmitterRef<SavePromptConfigurationRequest>`.

- **GeneratedCommunicationPreviewComponent**: `uploadCover`: `OutputEmitterRef<File>`, `removeCover`, `regenerate`, `discard`, `saveToHistory`: `OutputEmitterRef<void>`, `rate`: `OutputEmitterRef<RateDraftPayload>`, `saveRequested`: `OutputEmitterRef<{communicationId, payload}>`.
- **CommunicationHistoryListComponent**: `selected`: `OutputEmitterRef<number>`, `favoriteToggled`: `OutputEmitterRef<Communication>`, `confirmDeleteRequested`: `OutputEmitterRef<number | null>`, `deleteRequested`: `OutputEmitterRef<number>`.
- **PromptConfigurationListComponent**: `useRequested`: `OutputEmitterRef<PromptConfiguration>`, `confirmDeleteRequested`, `deleteRequested`.
- **RecentFeedbackListComponent**: nessuno, sola lettura.
- **GenerationProgressComponent**: nessuno, solo segnali `computed` derivati da `phase()`.

6.8.2 Componenti di feature dell'AI Co-Pilot

Equivalenti diretti dei precedenti, applicati al dominio Documents: conoscono i tipi generati dal contratto (`SubDocument`) e rimettono eventi verso `CopilotPage`, senza chiamare direttamente `DocumentWorkflowService` o lo Store.

`DocumentListComponent` è lo storico documenti paginato: la tabella dedica una colonna per aspetto dell'analisi (tipologia, data di caricamento, confidenza, stato di revisione, stato di invio) invece che ai singoli campi del documento sorgente, che restano raggruppati nella prima cella: una prima versione con nove colonne alla pari lasciava che la confidenza, il dato più rilevante per decidere cosa aprire, occupasse solo il 6% della riga.

`SubDocumentListComponent` è il pannello di dettaglio più corposo del modulo: anteprima del documento (con gli stati raggiungibile/in caricamento/non raggiungibile/non disponibile), form di revisione dei campi estratti con indicatore di provenienza per campo (`FieldOriginComponent`), e form del messaggio precompilato di invio (RF95-OB/UC-58). Il modulo di revisione non è un `<form>` HTML: per la maggior parte del tempo è in sola lettura, e un form senza un comando di invio proprio sarebbe una promessa che l'elemento non mantiene; il salvataggio resta un comando esplicito. Un `effect()` risincronizza i due moduli (revisione e invio) solo quando il documento mostrato cambia davvero (id diverso), non ad ogni nuovo riferimento con lo stesso id prodotto da una mutazione qualunque altrove nella pagina: altrimenti una modifica non ancora salvata sparirebbe in silenzio. Un secondo `effect()` rimappa sui controlli del form gli errori di validazione per campo restituiti dal backend (UC-55/UC-69).

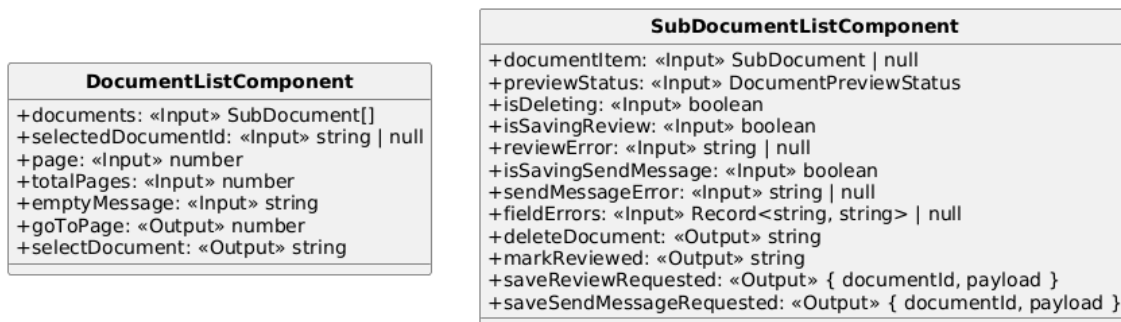


Figura 125: Classi: DocumentListComponent, SubDocumentListComponent

DocumentUploadPanelComponent raccoglie il file (via **FileDropzoneComponent**, Sezione 6.7) insieme a metadati manuali opzionali (tipologia, mese, anno, azienda) che restano autoritativi e non vengono sovrascritti dall'estrazione AI; il form dei metadati si disabilita durante il caricamento.

DocumentStatusTimelineComponent mostra lo stato di revisione corrente del sotto-documento come **StatusBadgeComponent**: nonostante il nome, non è una vera timeline con più tappe storiche, solo lo stato presente.

UploadProgressComponent è l'adattatore per la pipeline documentale, equivalente a **GenerationProgressComponent** ma con cinque tappe (caricamento, in coda, analisi OCR, estrazione campi, conclusa) invece di quattro.

FieldOriginComponent è il glifo di provenienza mostrato dentro ogni campo estratto: distingue un dato letto dall'AI con buona confidenza, uno sotto soglia da revisionare, uno corretto a mano dall'operatore e uno bloccato perché di sistema, con lo stesso colore già usato per lo stato di revisione (SC 1.4.1: l'informazione non passa dal solo colore, anche il glifo cambia) e il suggerimento che riporta la confidenza letta quando nota. La provenienza è per singolo campo, non dell'intero sotto-documento: un documento in revisione può avere un campo dubbio e uno nitido nella stessa scheda.

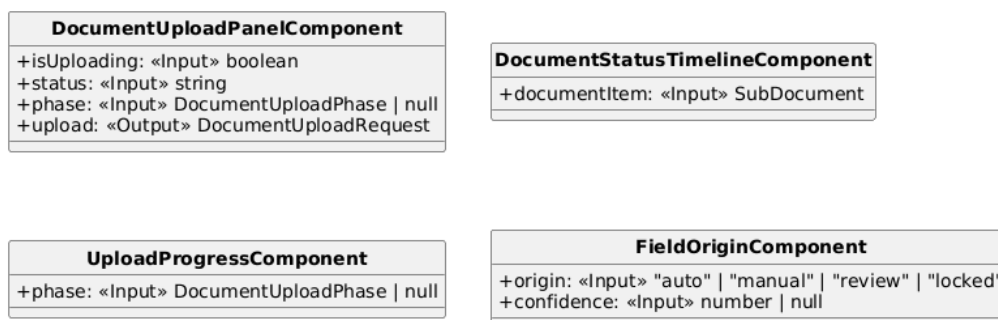


Figura 126: Classi: DocumentUploadPanelComponent, DocumentStatusTimelineComponent, UploadProgressComponent, FieldOriginComponent

Attributi

- **DocumentListComponent**: **documents** (default `[]`), **selectedDocumentId**, **page** (default 1), **totalPages** (default 1), **emptyMessage** (default *"I documenti caricati compariranno qui."*).

- **SubDocumentListComponent**: `documentItem`: `InputSignal<SubDocument | null>` (default `null`), `previewStatus`, `isDeleting`, `isSavingReview` (obbligatori), `reviewError`, `isSavingSendMessage`, `sendMessageError`, `fieldErrors`: `InputSignal<Record<string, string> | null>` (errori di validazione per campo restituiti dal backend).
- **DocumentUploadPanelComponent**: `isUploading`, `status`, `phase` (tutti obbligatori).
- **DocumentStatusTimelineComponent**: `documentItem`: `InputSignal<SubDocument>` (obbligatorio).
- **UploadProgressComponent**: `phase`: `InputSignal<DocumentUploadPhase | null>` (obbligatorio).
- **FieldOriginComponent**: `origin`: `InputSignal<"auto" | "manual" | "review" | "locked">` (obbligatorio), `confidence`: `InputSignal<number | null>` (default `null`, leggibilità del testo da cui viene il campo, quando nota).

Metodi

- **DocumentListComponent**: `goToPage`: `OutputEmitterRef<number>`, `selectDocument`: `OutputEmitterRef<string>`.
- **SubDocumentListComponent**: `deleteDocument`: `OutputEmitterRef<string>`, `markReviewed`, `saveReviewRequested`: `OutputEmitterRef<{documentId, payload}>`, `saveSendMessageRequested`: `OutputEmitterRef<{documentId, payload}>`.
- **DocumentUploadPanelComponent**: `upload`: `OutputEmitterRef<DocumentUploadRequest>` (file più metadati manuali).
- **DocumentStatusTimelineComponent**: nessuno.
- **UploadProgressComponent**: nessuno, solo segnali `computed` derivati da `phase()`.
- **FieldOriginComponent**: nessuno; espone solo l'etichetta accessibile derivata da `origin()` e `confidence()`.

6.8.3 SectionComponent

Contenitore "card" condiviso da entrambi i moduli e dalle pagine (Sezione 6.9): l'host stesso è la sezione stilizzata, così `id` per l'ancoraggio (usato dallo scorrimento verso una sezione target, Sezione 6.2) e classi modificatore applicate dal chiamante restano nel suo scope sotto la view encapsulation di Angular. L'intestazione (titolo più azioni proiettate in `[actions]`) compare solo quando è presente un titolo.

SectionComponent
+title: «Input» string undefined
+id: «Input» string undefined

Figura 127: Classe: SectionComponent

Attributi

- **title:** `InputSignal<string | undefined>`: quando assente, l'intestazione non è renderizzata.
- **id:** `InputSignal<string | undefined>`: applicato come attributo id sull'host, per l'ancoraggio da `scrollToElement`.

Metodi

- Nessuno.

6.9 Macro componenti UI (pagine)

Le pagine sono i componenti radice di ciascuna feature, montati dal router: **OverviewPage** (cruscotto con le metriche di entrambi i moduli), **AssistantPage** (AI Assistant Generativo) e **CopilotPage** (AI Co-Pilot). Nel pattern MVVM descritto in Sezione 6.2 sono la View: si procurano con `inject()` le dipendenze Angular (Store, servizio di feature) e costruiscono il proprio ViewModel passandoglielo nel costruttore, poi compongono i componenti di feature e quelli condivisi legando il template ai segnali e ai metodi esposti dal ViewModel. Sulla View restano solo gli aspetti che richiedono un injection context Angular che il ViewModel non ha (i **FormGroup** reattivi dei filtri, le sottoscrizioni che ne inoltrano le variazioni al ViewModel e gli `effect()` che reagiscono ai suoi segnali), oltre a poche azioni locali prive di logica di dominio (`resetFilters()`, su **AssistantPage** anche `resetMetricsFilters()` per il filtro separato della sezione Metriche, e su **CopilotPage** anche lo scorrimento verso il dettaglio in `selectDocument()`); nessuna chiamata HTTP e nessuna regola di business vive nella pagina.

6.9.1 AssistantPage

Compone il pannello di generazione (**CommunicationGeneratorPanelComponent**) e quello di anteprima (**GeneratedCommunicationPreviewComponent**), entrambi componenti di feature, insieme all'elenco delle configurazioni di prompt salvate (**PromptConfigurationListComponent**) e allo storico comunicazioni (**CommunicationHistoryListComponent**), al proprio pannello di metriche (**MetricsPanelComponent**, Sezione 6.7, più **MetricCompositionComponent** per la ripartizione delle bozze e **RecentFeedbackListComponent**, componente di feature, per gli ultimi feedback testuali, UC-27.3) e a componenti condivisi per stato di errore e stato vuoto, legando ogni input e ogni evento del template a un segnale o un metodo di `vm` (**AssistantPageViewModel**, Sezione 6.2), senza eccezioni: anche il banner d'errore principale chiama `(retry)="vm.reloadState()",` non lo Store direttamente (Sezione 6.2).

Nel costruttore istanzia il ViewModel con `new AssistantPageViewModel(inject(AssistantService), this.store):` non riceve una funzione di scorrimento, perché **AssistantPageViewModel** non la chiede al costruttore (Sezione 6.2).

Collega un **FormGroup** di filtri e un `effect()` che si limita a leggere lo storico condiviso e i filtri attivi del ViewModel, poi chiama `vm.reload():` la chiamata

di ricerca vera e propria vive nel ViewModel, non qui. Un secondo `effect()` legge `vm.metricsFilters()` e chiama `vm.refreshFilteredMetrics()` per la sezione Metriche (RF38-OB..RF41-OB), filtro separato da quello dello storico e collegato a un `FormGroup` proprio (`metricsFilterForm`). Un terzo `effect()` legge `vm.pendingScrollTarget()` e, quando non è nullo, chiama `scrollToElement` (l'unica operazione DOM richiesta dal ViewModel) poi azzerà il segnale: è la View a eseguire lo scorrimento richiesto. Registra infine `inject(DestroyRef).onDestroy(() => vm.destroy())`, così la sottoscrizione di lettura in corso viene annullata alla distruzione del componente senza che il ViewModel sappia nulla del ciclo di vita di Angular.

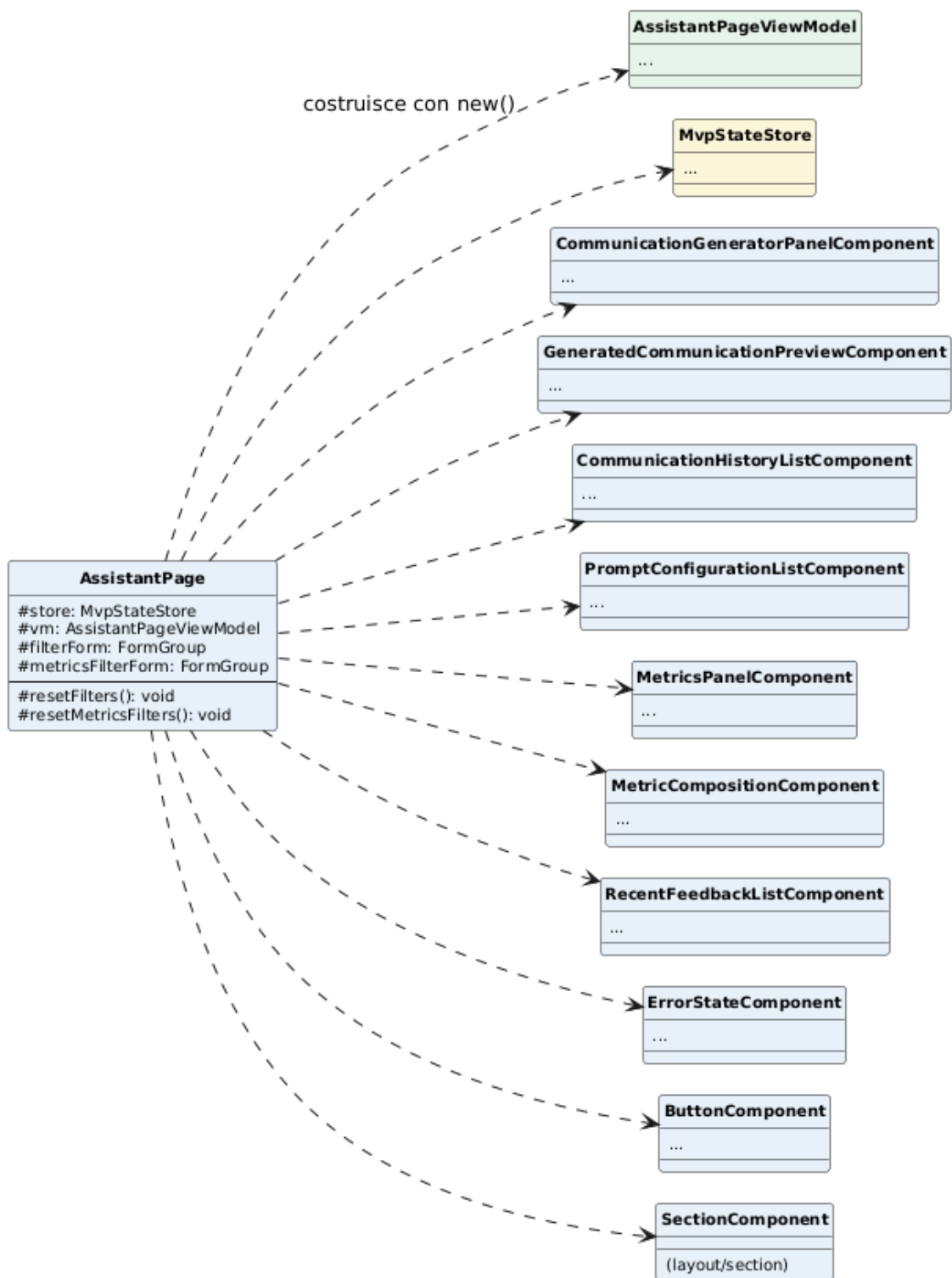


Figura 128: Classe: AssistantPage

Attributi

- **store:** MvpStateStore: dipendenza Angular procurata con `inject()`, campo `protected`: passata al costruttore del ViewModel e letta come segnale sorgente

nell'`effect()` di ricerca; il template non la referencia mai (Sezione 6.2).

- **vm:** `AssistantPageViewModel`: istanziato nel costruttore, punto a cui si lega il resto del template.
- **filterForm:** `FormGroup`: form reattivo dei filtri dello storico, resta sulla View perché `ReactiveFormsModule` richiede un `injection context`.
- **metricsFilterForm:** `FormGroup`: form reattivo dei filtri della sezione Metriche (tono, stile, intervallo di date), separato da **filterForm**: non condivide stato con lo storico.

Metodi

- **resetFilters():** `void`: azzera il form dei filtri dello storico.
- **resetMetricsFilters():** `void`: azzera il form dei filtri della sezione Metriche.

6.9.2 CopilotPage

Stessa struttura di `AssistantPage`, applicata al Co-Pilot: compone il pannello di upload (`DocumentUploadPanelComponent`), l'elenco documenti (`DocumentListComponent`, a cui passa `[page]="vm.currentPage()"` e `[totalPages]="vm.totalPages()"`, inoltrando (`goToPage`) a `vm.goToPage($event)`), il pannello di revisione del sotto-documento selezionato (`SubDocumentListComponent`, a cui passa anche `[previewStatus]="vm.previewStatus()"`) e il proprio pannello di metriche (`MetricsPanelComponent`, con `[presentation]="vm.metricsPresentation()"`), leggendo dal template solo `vm.*`: anche qui il banner d'errore principale chiama (`retry`)="`vm.reloadState()`", non lo Store direttamente (Sezione 6.2). Istanza il `ViewModel` con `new CopilotPageViewModel(inject(DocumentWorkflowService), store)`, senza la dipendenza di scorrimento, che questo `ViewModel` non utilizza (Sezione 6.2).

Collega allo stesso modo un `FormGroup` di filtri (`createDocumentFilterForm()`) e un `effect()` che rilegge lo storico documentale chiamando `vm.reload()` a ogni cambio di filtro o di stato condiviso. Un secondo `effect()` legge `vm.selectedDocument()` e chiama `vm.loadPreviewStatus(document?.previewUrl ?? null)`: la fetch della raggiungibilità dell'anteprima segue la selezione, non un elenco a parte. Registra inoltre `inject(DestroyRef).onDestroy(() => vm.destroy())`.

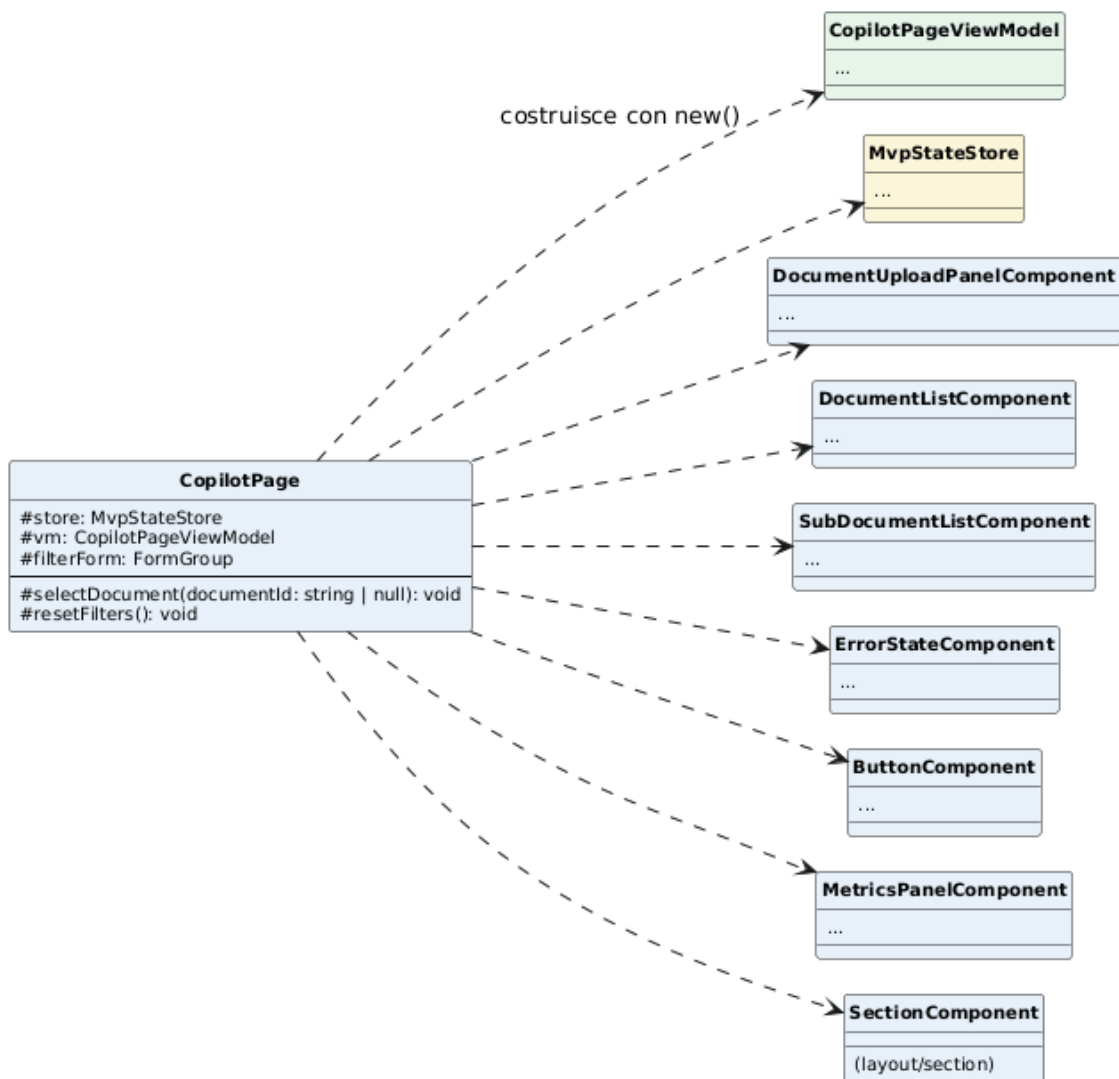


Figura 129: Classe: CopilotPage

Attributi

- **store:** `MvpStateStore`: dipendenza Angular procurata con `inject()`, campo `protected`: passata al costruttore del ViewModel e letta come segnale sorgente nel primo `effect()` (quello di ricerca); il secondo legge solo `vm.selectedDocument()`. Il template non la referencia mai, stesso ruolo che ha su `AssistantPage`.
- **vm:** `CopilotPageViewModel`: istanziato nel costruttore.
- **filterForm:** `FormGroup`: form reattivo dei filtri dello storico documenti.

Metodi

- **`selectDocument(documentId: string | null): void`**: apre il dettaglio del sotto-documento selezionato e vi scorre la pagina (verso `copilot-document-detail`), solo se `documentId` non è nullo. Lo scorrimento resta nella View e non nel ViewModel perché è lei l'unica responsabile della resa visiva.

- `resetFilters(): void`.

6.9.3 OverviewPage

La pagina più semplice nella logica di collegamento (nessun `FormGroup`, perché la Overview non ha uno storico filtrabile proprio): mostra le priorità operative (`AttentionNoteComponent`, montato solo se `vm.quarantined() > 0`, più una griglia di `MetricCardComponent` per `vm.priorities()`), gli indicatori di qualità per modulo (`MetricStarsComponent` per `vm.assistantQuality()`, `MetricGaugeComponent` per `vm.copilotQuality()`, Sezione 6.7) e le attività recenti già presenti nello Store, in una tabella con `StatusBadgeComponent/EmptyStateComponent`. Non monta `MetricsPanelComponent`: mostra direttamente le sue schede mirate, non un pannello generico.

Istanza il ViewModel con `new OverviewPageViewModel(this.store, inject(Router))`: non riceve una funzione di scorrimento nel costruttore, per lo stesso motivo di `AssistantPage`. I pulsanti di richiamo all'azione nella sezione hero e i due link "Vedi tutte le metriche" delegano interamente a `vm.navigate()`, che naviga e poi chiede lo scorrimento verso la sezione target sulla pagina di destinazione; un `effect()` nel costruttore legge `vm.pendingScrollTarget()` e lo esegue, stesso schema di `AssistantPage/CopilotPage`.

Qui la View non legge lo Store in nessun punto del template, banner d'errore compreso (`((retry)="vm.reload()")`): il ViewModel sintetizza priorità e qualità (Sezione 6.2), e il campo `store` sulla View resta solo per costruire il ViewModel, esattamente come su `AssistantPage` e `CopilotPage`.

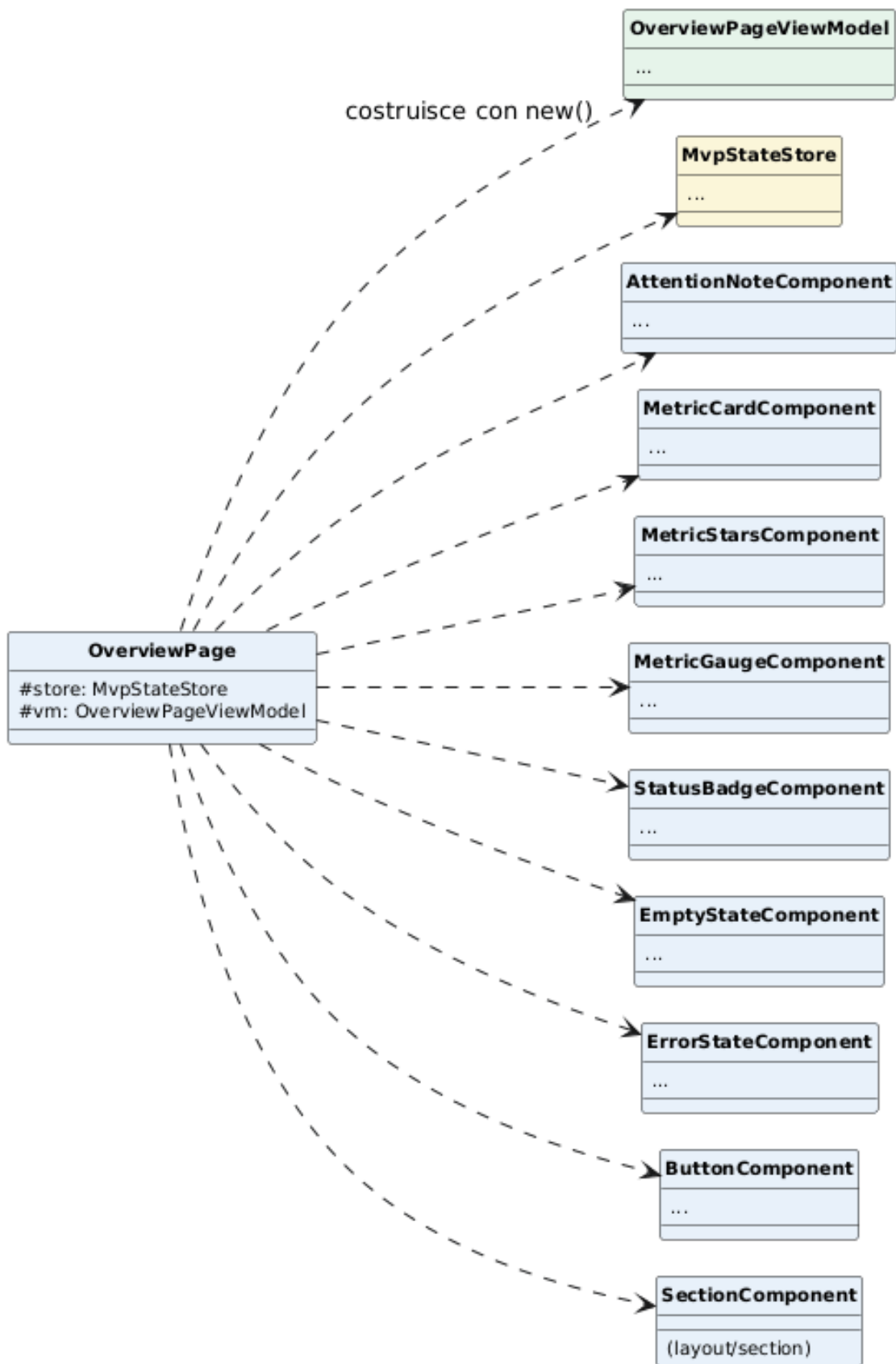


Figura 130: Classe: OverviewPage

Attributi

- **store:** `MvpStateStore`: dipendenza Angular procurata con `inject()`, campo `protected`: passata al costruttore del `ViewModel`; il template non la referencia mai.
- **vm:** `OverviewPageViewModel`: istanziato nel costruttore insieme a `inject(Router)`.

Metodi

- Nessuno: a differenza delle altre due pagine, non ha bisogno di cablare filtri propri; l'unico `effect()` è logica di collegamento nel costruttore, non un metodo della classe.

7 Design Pattern

I design pattern individuati nel sistema si dividono in due categorie: quelli applicati esplicitamente nella logica applicativa, con un adapter, una porta o un collaboratore reale dietro il nome (non citati per completare l'elenco), e i pattern nativi del framework, adottati così come sono senza reimplementarli. Per ciascun pattern applicato esplicitamente viene indicato dove si trova nel codice, quale problema risolve in quel punto specifico e cosa succederebbe senza. Due pattern valutati e scartati (Proxy, Abstract Factory) sono documentati alla fine della Sezione 7.1, per lo stesso motivo: un pattern citato solo per apparire nell'elenco non è meno fuorviante di uno assente.

Pattern	Categoria	Dove applicato
Adapter	Strutturale	I repository Eloquent, gli adapter Bedrock/Texttract e <code>SfnWorkflowEngineAdapter</code> di entrambi i domini (Sezione 5)
Strategy	Comportamentale	<code>WorkflowTaskHandler</code> (porta primaria), selezionato da <code>WorkflowTaskRegistry::for(\$taskType)</code>
Facade	Strutturale	I casi d'uso applicativi (<code>UploadDocumentService</code> , <code>GenerateCommunicationService</code> , ...), un solo metodo pubblico per coordinare più porte secondarie
Factory Method	Creazionale	<code>WorkflowTaskRegistry::for(string \$taskType): WorkflowTaskHandler</code>
Singleton	Creazionale	Binding dei client AWS e degli adapter nel service provider, bindati all'interfaccia di porta
Observer	Comportamentale	Porte dispatcher di eventi dei due domini → dispatcher Laravel, 13 + 20 listener
Command	Comportamentale	Ogni caso d'uso applicativo, una classe per una porta primaria

Tabella 15: Design pattern individuati nel backend

7.1 Design Pattern Backend

7.1.1 Adapter

Intent: convertire l'interfaccia di una classe in un'altra interfaccia attesa dal client, permettendo la collaborazione fra classi altrimenti incompatibili per via delle librerie o dei framework esterni che usano.

È il pattern costitutivo dell'intera architettura esagonale (Sezione 4.3): isola sistematicamente il dominio applicativo dalla forma specifica delle tecnologie esterne (ORM, SDK cloud) che altrimenti imporrebbero la propria interfaccia direttamente ai casi d'uso. Ogni porta secondaria ha un adapter che la implementa sopra una tecnologia concreta, mai il contrario.

Esempio nel progetto. `EloquentDocumentRepository` ed `EloquentCommunicationRepository` implementano le porte di persistenza sopra Eloquent, `BedrockDocumentAiAdapter`, `BedrockCommunicationAiAdapter` e `TextractOcrAdapter` sopra gli SDK AWS, `SfnWorkflowEngineAdapter` sopra Step Functions (Sezione 5). Un caso d'uso parla sempre con `DocumentRepository`, mai con `SubDocument::query()`. Senza questo pattern il dominio dipenderebbe direttamente da Eloquent e dagli SDK AWS, e nessun caso d'uso sarebbe testabile senza avviare Laravel e senza credenziali AWS/LocalStack: è questa la testabilità del dominio senza framework descritta in Sezione 4.7, dimostrata dalla suite dedicata `tests/DomainUnit` che istanzia i casi d'uso con implementazioni in memoria delle porte secondarie.

```
1 // app/Mvp/Documents/Adapters/Outbound/Ai/BedrockDocumentAiAdapter.php
2 class BedrockDocumentAiAdapter implements DocumentAiGatewayPort
3 {
4     public function __construct(private readonly BedrockService
5         $bedrock) {}
6
7     public function splitDocument(string $ocrText, int $pageCount,
8         string $boundaryNonce): array
9     {
10         return $this->bedrock->splitDocument($ocrText, $pageCount,
11             $boundaryNonce);
12     }
13
14     public function extractFields(string $ocrText): array
15     {
16         return $this->bedrock->extractFields($ocrText);
17     }
18
19     public function pageBoundaryMarker(int $page, string $nonce):
20         string
21     {
22         return BedrockService::pageBoundaryMarker($page, $nonce);
23     }
24
25     public function formatUserError(\Throwable $e, string
26         $defaultMessage): string
27     {
28         return BedrockService::formatUserError($e, $defaultMessage);
29     }
30 }
```


Listing 1: BedrockDocumentAiAdapter: adapter secondario sopra BedrockService

7.1.2 Strategy

Intent: definire una famiglia di algoritmi, incapsularli singolarmente e renderli intercambiabili indipendentemente dal client che li usa.

Risolve il problema di un componente infrastrutturale condiviso da più domini che deve restare agnostico rispetto al passo di business specifico di ciascuno, delegando solo quel passo a un'implementazione selezionata a runtime invece di conoscerla in anticipo.

Esempio nel progetto. `WorkflowTaskHandler` è la porta primaria implementata da `DocumentWorkflowTaskHandler` e `CommunicationWorkflowTaskHandler` (Sezione 5): `WorkflowTaskRunner`, il collaboratore condiviso che riceve un task da Step Functions/SQS, resta completamente agnostico rispetto al dominio (deduplicazione, presa in carico e audit sono identici per entrambi) e delega il solo passo di business all'handler selezionato da `WorkflowTaskRegistry` in base al tipo di task. Senza questo pattern, `WorkflowTaskRunner` avrebbe bisogno di un `match/if` sul dominio del task: l'infrastruttura di workflow condivisa dovrebbe conoscere i dettagli interni di Documents e Communications, lo stesso tipo di accoppiamento che la Dependency Rule (Sezione 4.7) vieta fra i due domini, anche se lo script di CI che la verifica copre solo i livelli Domain e Application, non l'infrastruttura di workflow condivisa.

```

1  // app/Mvp/Workflow/Services/WorkflowTaskRunner.php
2  class WorkflowTaskRunner
3  {
4      public function __construct(
5          private readonly WorkflowTaskRegistry $registry,
6          private readonly AuditLogger $audit,
7      ) {}
8
9      public function handle(array $message): array
10     {
11         $taskToken = (string) ($message['taskToken'] ?? $message['
12             task_token'] ?? '');
13         $taskType = (string) ($message['taskType'] ?? $message['
14             task_type'] ?? '');
15
16         if ($taskToken === '' || $taskType === '') {
17             throw new \InvalidArgumentException('Messaggio workflow non
18                 valido: taskToken e taskType sono obbligatori.');
```

```

25      // stato "running", gestione dei casi già completato/fallito:
26      // logica agnostica dal dominio, identica per ogni taskType.
27
28      // Delega del solo passo di business alla Strategy selezionata.
29      $result = $handler->execute($taskType, $subject, $message);
30
31      // [...] aggiornamento del task, audit, costruzione della
32      //      risposta.
33  }

```

Listing 2: WorkflowTaskRunner: selezione della Strategy concreta e delega del passo di business

7.1.3 Facade

Intent: fornire un'interfaccia unificata a un insieme di interfacce di un sottosistema, semplificando l'uso del sottosistema per il client.

Evita che l'adapter primario (un Controller HTTP o un handler di workflow) debba conoscere e orchestrare direttamente i singoli collaboratori necessari a soddisfare una richiesta applicativa, riducendo quella conoscenza a un solo metodo pubblico.

Esempio nel progetto. Ogni caso d'uso applicativo (Sezione 5) è una Facade sulle proprie porte secondarie: `ProcessDocumentService` coordina repository, storage, gateway AI, dispatcher di eventi, heartbeat e generatore di identificativi dietro un solo metodo pubblico (`process()`); l'adapter primario che lo invoca, in questo caso `DocumentWorkflowTaskHandler`, non deve sapere quanti collaboratori servono per soddisfare la richiesta. Senza questo pattern sarebbe l'adapter primario a orchestrare direttamente i dieci collaboratori del costruttore di `ProcessDocumentService`, la situazione esplicitamente evitata dalla separazione fra adapter primario e caso d'uso applicativo su cui si fonda tutta la Sezione 5.

```

1  // app/Mvp/Documents/Application/UseCases/ProcessDocumentService.php
2  class ProcessDocumentService implements ProcessDocumentUseCase
3  {
4      public function __construct(
5          private readonly DocumentRepository $documents,
6          private readonly DocumentStoragePort $storage,
7          private readonly DocumentAiGatewayPort $ai,
8          private readonly DocumentEventDispatcherPort $events,
9          private readonly WorkflowHeartbeatPort $heartbeat,
10         private readonly LoggerInterface $logger,
11         private readonly UniqueIdGeneratorPort $ids,
12         private readonly OcrRangeReader $ocrRange,
13         private readonly ExtractSubDocumentFieldsUseCase
14             $fieldsExtractor,
15         private readonly ClockInterface $clock,
16         private readonly string $tempDirectory,
17     ) {}
18
19     public function process(int $documentId): array
20     {

```

```

20     $document = $this->documents->findOriginalDocument($documentId)
21     ;
22     if ($document->processingStatus() === ProcessingStatus::
23         Completed) {
24         return ['skipped' => true];
25     }
26     $absoluteSource = null;
27
28     try {
29         $this->documents->updateOriginalDocument($documentId,
30             OriginalDocumentChanges::none()
31             ->withProcessingStatus(ProcessingStatus::Processing)
32             ->withErrorMessage(null));
33         $this->events->dispatch(new DocumentProcessingStarted(
34             $documentId, $document->tenantId));
35
36         $absoluteSource = $this->copyStorageFileToTemporaryPath(
37             $document->filePath);
38         $pdf = new Fpdi;
39         $pageCount = max(1, $pdf->setSourceFile($absoluteSource));
40
41         $this->heartbeat->beat(force: true);
42
43         $segments = $this->normalizeSegments($this->splitDocument(
44             $document, $pageCount), $pageCount);
45
46         $oldSplitPaths = $this->documents->
47             deleteExistingSubDocuments($documentId);
48         $this->deleteStoragePaths($oldSplitPaths);
49
50         foreach ($segments as $segment) {
51             $this->heartbeat->beat();
52             $splitPath = $this->extractPages($absoluteSource,
53                 $documentId, $segment['employee_name'], $segment['
54                     start_page'], $segment['end_page']);
55
56             try {
57                 $subDocumentId = $this->documents->
58                     createSubDocument(new NewSubDocument(
59                         originalDocumentId: $documentId,
60                         filePath: $splitPath,
61                         startPage: $segment['start_page'],
62                         endPage: $segment['end_page'],
63                     ));
64             } catch (\Throwable $e) {
65                 $this->deleteStoragePaths([$splitPath]);
66
67                 throw $e;
68             }
69
70             $this->fieldsExtractor->extractAndSaveFieldsWithContext
71                 ($subDocumentId, $document);
72         }
73     }
74 }

```

```

65         $document->complete();
66         $this->documents->saveOriginalDocument($document);
67         $this->events->dispatch(new DocumentProcessingCompleted(
68             $documentId, $document->tenantId));
69     } catch (\Throwable $e) {
70         $this->handleProcessingFailure($documentId, $e);
71     } finally {
72         if ($absoluteSource !== null) {
73             @unlink($absoluteSource);
74         }
75     }
76     return ['skipped' => false];
77 }
78
79 // [...] handleProcessingFailure(), normalizeSegments(),
80 // splitDocument(),
81 // deleteStoragePaths(), extractPages(),
82 // copyStorageFileToTemporaryPath(),
83 // temporaryPath(): dettagli privati di manipolazione PDF e storage
84 // esposti al chiamante di process().
85 }

```

Listing 3: ProcessDocumentService: Facade su repository, storage, gateway AI, eventi ed heartbeat dietro process()

7.1.4 Factory Method

Intent: definire un'interfaccia per creare un oggetto, lasciando che sia un metodo dedicato a decidere quale classe concreta istanziare, invece del chiamante.

Centralizza in un solo punto la selezione dell'implementazione corretta a runtime, così che il chiamante non debba conoscere in anticipo l'insieme delle classi concrete disponibili né duplicare la logica di scelta a ogni punto di invocazione.

Esempio nel progetto. `WorkflowTaskRegistry::for(string $taskType): WorkflowTaskHandler` seleziona l'handler corretto per tipo di task, senza che `WorkflowTaskRunner` (il chiamante) conosca le classi concrete `DocumentWorkflowTaskHandler/CommunicationWorkflowTaskHandler`. Senza questo pattern, quella selezione andrebbe duplicata con logica propria a ogni punto di invocazione del runner.

Nota di onestà: non è Factory Method in senso GoF stretto, che richiede una gerarchia di sottoclassi dove ciascuna decide per override quale prodotto concreto creare. `WorkflowTaskRegistry` è una singola classe con una mappa interna e un lookup a runtime, non una gerarchia, più vicino a un Registry (nel senso di Fowler) che a un Factory Method: stesso intento pratico (il chiamante non conosce le classi concrete), meccanismo diverso.

```

1 // app/Mvp/Workflow/Services/WorkflowTaskRegistry.php
2 class WorkflowTaskRegistry
3 {
4     /** @var array<string, WorkflowTaskHandler> */
5     private array $handlers = [];

```

```

6
7     public function register(WorkflowTaskHandler $handler): void
8     {
9         foreach ($handler->taskTypes() as $taskType) {
10             $this->handlers[$taskType] = $handler;
11         }
12     }
13
14     public function for(string $taskType): WorkflowTaskHandler
15     {
16         return $this->handlers[$taskType]
17             ?? throw new \InvalidArgumentException("Task workflow non
18                 supportato: {$taskType}");
19     }
20
21     /**
22      * @return array<int, string>
23      */
24     public function taskTypes(): array
25     {
26         return array_keys($this->handlers);
27     }

```

Listing 4: WorkflowTaskRegistry: registrazione e lookup dell'handler per tipo di task

7.1.5 Singleton

Intent: garantire che una classe abbia una sola istanza condivisa e fornire un punto di accesso globale ad essa.

Evita la costruzione ripetuta di collaboratori costosi da istanziare (un client HTTP verso un servizio esterno, tipicamente) e centralizza in un solo punto la scelta di quale implementazione concreta soddisfa una data porta.

Esempio nel progetto. I client AWS (`SfnClient`, il client Bedrock, il client Textract) e i rispettivi adapter sono bindati come singleton nel service provider (costruzione costosa evitata, connessioni riutilizzate), ma bindati all'interfaccia di porta, non alla classe concreta: il binding diventa così anche il punto unico in cui si decide quale adapter soddisfa quale porta. Senza questa scelta, cambiare adapter richiederebbe cercare e modificare ogni type-hint concreto sparso nella codebase.

Nota di onestà: non è un Singleton in senso GoF stretto, che richiede che sia la classe stessa a impedire una seconda istanza (costruttore privato, `getInstance()` statico). Qui è il container a garantire un'unica istanza condivisa per tutta la durata del processo che la risolve: una request HTTP per l'API, l'intero ciclo di vita del processo per i Worker, che girano in loop continuo: un vincolo di lifecycle esterno alla classe, non una proprietà che la classe impone su se stessa: `SfnWorkflowEngineAdapter` e gli altri adapter restano classi con un costruttore pubblico ordinario, e nulla impedirebbe a un altro punto del codice di chiamare comunque `new` e ottenere una seconda istanza.

```

1 // app/Providers/AppServiceProvider.php (register())
2 $this->app->singleton(BedrockRuntimeClient::class, function () {

```

```

3     return $this->bedrockClient((string) config('services.bedrock.
        region'));
4 });
5
6 $this->app->singleton(SfnClient::class, function () {
7     $config = [
8         'version' => 'latest',
9         'region' => config('services.workflow.region'),
10    ];
11
12    if (filled(config('services.workflow.endpoint'))) {
13        $config['endpoint'] = config('services.workflow.endpoint');
14    }
15
16    return new SfnClient($config);
17 });
18
19 // [...] SqsClient, TextractClient, WorkflowTaskHeartbeat: stesso
schema,
20 // client costoso o servizio condiviso, un solo binding.
21
22 // --- Dominio Documents: porta -> adapter ---
23 $this->app->singleton(OcrGatewayPort::class, TextractOcrAdapter::class)
24 ;
25 $this->app->singleton(DocumentAiGatewayPort::class,
    BedrockDocumentAiAdapter::class);
26 $this->app->singleton(DocumentStoragePort::class,
    FlysystemDocumentStorageAdapter::class);
27 $this->app->singleton(DocumentRepository::class,
    EloquentDocumentRepository::class);
28 $this->app->singleton(SendMessageRendererPort::class,
    DompdfSendMessageRenderer::class);
29 $this->app->singleton(DocumentEventDispatcherPort::class,
    LaravelDocumentEventDispatcher::class);
30
31 // Workflow: porta condivisa fra Documents e Communications.
32 $this->app->singleton(WorkflowEnginePort::class,
    SfnWorkflowEngineAdapter::class);

```

Listing 5: AppServiceProvider: binding singleton dei client AWS e delle porte secondarie

7.1.6 Observer

Intent: definire una dipendenza uno-a-molti fra oggetti, così che quando un oggetto cambia stato tutti i suoi dipendenti vengano notificati automaticamente, senza che il soggetto li conosca singolarmente.

Disaccoppia l'esecuzione di una regola di business dalla sua conseguenza collaterale (l'audit) che diventa una reazione indipendente a un evento invece di una chiamata esplicita ripetuta in ogni caso d'uso che ne ha bisogno.

Esempio nel progetto. Applicato simmetricamente a entrambi i domini, con porte separate perché gli eventi sono specifici di dominio: Documents dispatcha 14 eventi (DocumentEventDispatcherPort → LaravelDocumentEventDispatcher, 13 listener), Communications 21 (CommunicationEventDispatcherPort →

LaravelCommunicationEventDispatcher, 20 listener; si veda la Sezione 5 per l'elenco delle classi che li dispatchano). In ciascun dominio un evento di completamento pipeline (DocumentWorkflowCompleted, CommunicationWorkflowCompleted) non ha un listener registrato: lo stato che segnala è già persistito direttamente dal caso d'uso che lo dispatcha, non da una reazione all'evento. Ogni listener dipende da un solo collaboratore, AuditLogger (Sezione 5.3).

Senza Observer, ogni nuova reazione a un evento di dominio richiederebbe toccare ogni caso d'uso che genera quell'evento, invece di aggiungere un listener.

```
1 // app/Providers/AppServiceProvider.php (boot())
2 Event::listen(DocumentUploadAccepted::class,
3     RecordDocumentUploadAccepted::class);
4 Event::listen(DocumentProcessingStarted::class,
5     RecordDocumentProcessingStarted::class);
6 Event::listen(DocumentProcessingCompleted::class,
7     RecordDocumentProcessingCompleted::class);
8 Event::listen(DocumentProcessingFailed::class,
9     RecordDocumentProcessingFailed::class);
10 // [...] altri 9 listener per Documents, 20 per Communications: stesso
11 // schema, un Event::listen per coppia evento/listener.
12
13 // app/Mvp/Documents/Application/Listeners/
14 // RecordDocumentProcessingStarted.php
15 class RecordDocumentProcessingStarted
16 {
17     public function __construct(private readonly AuditLogger $audit) {}
18
19     public function handle(DocumentProcessingStarted $event): void
20     {
21         $this->audit->record(
22             'mvp-document-processing-started',
23             resourceType: 'original_document',
24             resourceId: (string) $event->documentId,
25             metadata: ['status' => ProcessingStatus::Processing->value],
26             tenantId: $event->tenantId,
27         );
28     }
29 }
```

Listing 6: Registrazione del listener in AppServiceProvider e reazione in RecordDocumentProcessingStarted

Il caso d'uso che pubblica l'evento (ProcessDocumentService::process(), si veda il Listing precedente) non conosce RecordDocumentProcessingStarted: chiama `$this->events->dispatch(new DocumentProcessingStarted(...))` sulla propria porta secondaria, e il dispatcher Laravel invoca tutti i listener registrati per quell'evento.

7.1.7 Command

Intent: incapsulare una richiesta come oggetto a sé stante, permettendo di parametrizzare i client con richieste diverse e di trattare ogni richiesta come una singola unità testabile in isolamento.

Trasforma ogni operazione applicativa in un oggetto con una singola responsabilità, testabile passando implementazioni fittizie delle sue dipendenze senza dover avviare l'intero framework che lo circonda.

Esempio nel progetto. Ogni caso d'uso applicativo (Sezione 5) è una classe dedicata a una singola porta primaria, la stessa forma già usata da `WorkflowTaskHandler::execute()`: un caso d'uso, una responsabilità, testabile passando implementazioni fittizie delle porte secondarie senza avviare Laravel.

```
1 // app/Mvp/Documents/Application/UseCases/DeleteDocumentService.php
2 class DeleteDocumentService implements DeleteDocumentUseCase
3 {
4     public function __construct(
5         private readonly DocumentRepository $documents,
6         private readonly DocumentStoragePort $storage,
7         private readonly DocumentEventDispatcherPort $events,
8         private readonly LoggerInterface $logger,
9     ) {}
10
11     public function delete(int $subDocumentId, Actor $actor): void
12     {
13         $subDocument = $this->documents->findSubDocument($subDocumentId
14             );
15         $originalDocumentId = $subDocument->originalDocumentId;
16         $original = $this->documents->findOriginalDocument(
17             $originalDocumentId);
18
19         if ($original->tenantId !== $actor->tenantId) {
20             throw new DocumentNotAuthorizedException;
21         }
22
23         $this->documents->deleteSubDocument($subDocumentId);
24         $this->deleteStoragePath($subDocument->filePath);
25
26         $this->events->dispatch(new SubDocumentDeleted($subDocumentId,
27             $originalDocumentId, $actor));
28
29         if (! $this->documents->
30             originalDocumentHasRemainingSubDocuments(
31                 $originalDocumentId)) {
32             $this->documents->deleteOriginalDocumentWithWorkflowTasks(
33                 $originalDocumentId);
34             $this->deleteStoragePath($original->filePath);
35         }
36     }
37
38     private function deleteStoragePath(string $path): void
39     {
40         try {
41             $this->storage->delete($path);
42         } catch (\Throwable $e) {
43             $this->logger->warning('DeleteDocumentService: storage
44                 cleanup failed', ['path' => $path, 'message' => $e->
45                     getMessage()]);
46         }
47     }
48 }
```


39
40

```
}  
}
```

Listing 7: DeleteDocumentService: caso d'uso applicativo con una singola porta primaria (delete())

Nota di onestà: l'idea "un caso d'uso = una classe con un solo metodo pubblico" non regge alla lettera per le porte che raggruppano più transizioni sullo stesso aggregato: `CommunicationDraftUseCase` ne ha cinque (`favorite/unfavorite/update/save/discard`), `StartCommunicationWorkflowUseCase` e `UpdateCommunicationCoverUseCase` ne hanno due ciascuna. È una scelta esplicita (raggruppare transizioni correlate sullo stesso aggregato invece di una classe per transizione), non un Command puro: si applica alla maggior parte dei casi d'uso, non a tutti: le porte che raggruppano più transizioni sono una decina, distribuite su entrambi i domini.

7.1.8 Pattern nativi del framework

Laravel contribuisce due pattern propri, adottati così come sono anziché reimplementati: Eloquent (variante di *Active Record*, POEAA) per il mapping oggetti-tabelle, usato però solo dentro gli adapter di persistenza, mai nel dominio: è proprio ciò che l'Adapter (Sezione 7.1.1) isola. Il Service Container di Laravel fornisce la dependency injection su cui si appoggiano sia il binding a Singleton (Sezione 7.1.5) sia la risoluzione automatica dei costruttori dei casi d'uso. Da non confondere con l'omonimo pattern Facade di Laravel (`Log::`, `Storage::`, `Auth::`): un accesso statico ai binding del container, un'accezione diversa dal GoF Facade applicato ai casi d'uso in Sezione 7.1.3): quelle Facade restano confinate agli adapter secondari (es. `Storage::` in `FlysystemDocumentStorageAdapter`, `Log::` in `TextractOcrAdapter`): mai usate nel Dominio di Documents e Communications, dove lo stesso script di CI che verifica la Dependency Rule (Sezione 4.7) blocca esplicitamente ogni riferimento a `Illuminate\*`. Nell'Applicazione lo script è più permissivo e non vieta `Illuminate\*` in generale (solo `Aws\*`, i model Eloquent e il namespace dell'altro dominio): quelle tre Facade restano comunque assenti anche lì, ma per disciplina di progetto, non per un vincolo meccanico.

Due pattern sono stati valutati e scartati esplicitamente, non per assenza di un candidato ma perché inapplicabili al problema reale: **Proxy** (caching o circuit-breaking davanti ai gateway Bedrock/Textextract) contraddirebbe una scelta esplicita del progetto: nessun fallimento di un servizio AI deve essere mascherato da un comportamento sostitutivo automatico. **Abstract Factory** non ha un candidato reale: non esiste nel progetto una famiglia di adapter correlati da costruire coerentemente insieme (gli adapter per Local-Stack e per AWS reale non sono famiglie diverse, sono la stessa classe con endpoint e credenziali diversi), e l'unico bisogno di selezione a runtime è già coperto dal Factory Method di `WorkflowTaskRegistry`.

7.2 Design Pattern Frontend

7.2.1 Facade

Intent: fornire un'interfaccia unificata a un insieme di interfacce di un sottosistema, semplificando l'uso del sottosistema per il client: lo stesso intento del Facade backend (Sezione 7.1.3), applicato al lato client.

Evita che un ViewModel debba orchestrare direttamente più collaboratori del livello dati (client HTTP, canale SSE, stato condiviso) per compiere una singola azione, riducendo quell'orchestrazione a un solo metodo.

Esempio nel progetto. `AssistantService` e `DocumentWorkflowService` (Sezione 6.4) sono Facade sul client HTTP generato, su `SseClient` e sullo Store condiviso: un ViewModel chiama un solo metodo (`generate()`, `upload()`...) senza sapere che dietro c'è una richiesta HTTP, l'apertura di uno stream SSE e un aggiornamento dello stato applicativo. Senza questo pattern, ogni ViewModel dovrebbe orchestrare direttamente client API, `SseClient` e `MvpStateStore` per ciascuna azione, duplicando la stessa sequenza di passi in più punti.

```
1 // apps/frontend/src/app/features/assistant/data/assistant.service.ts
2 generate(payload: CommunicationDraftForm): Observable<
3   CommunicationGenerationProgress> {
4   return this.trackGeneration(this.api.generateMvpCommunication(payload));
5 }
6 private trackGeneration(
7   start$: Observable<StartCommunicationGenerationResponse>
8 ): Observable<CommunicationGenerationProgress> {
9   return new Observable<CommunicationGenerationProgress>((observer) =>
10     {
11       let closeStream: (() => void) | null = null;
12
13       const subscription = start$.subscribe({
14         next: (response) => {
15           observer.next({
16             status: response.message,
17             phase: "queued",
18             communicationId: response.communicationId
19           });
20
21           closeStream = this.sse.connect(response.streamUrl, {
22             onEvent: (event, data) => {
23               // [...] rilancia sull'observer un evento
24               // CommunicationGenerationProgress
25               // per ciascun evento SSE ("progress", "text", "cover", "
26               // still_running"),
27               // e su "done" aggiorna lo Store con lo stato autorevole
28               // del backend:
29               if (event === "done") {
30                 const payload = data as { communication?: Communication;
31                   state?: MvpState };
32
33                 if (payload.state) {
```

```

29         this.store.setState(payload.state);
30     }
31
32     observer.next({
33         status: "Bozza generata correttamente.",
34         phase: "completed",
35         communicationId: response.communicationId,
36         communication: payload.communication
37     });
38     closeStream?();
39     observer.complete();
40 }
41 },
42 onNamedError: (message) => {
43     observer.next({
44         status: message || "Generazione non disponibile.
45         Controlla lo stato della bozza.",
46         phase: "failed",
47         communicationId: response.communicationId
48     });
49     closeStream?();
50     this.store.reload();
51     observer.complete();
52 },
53 onConnectionError: () => {
54     closeStream?();
55     this.store.reload();
56     observer.complete();
57 }
58 });
59 error: (error: unknown) => observer.error(error)
60 });
61
62 return () => {
63     subscription.unsubscribe();
64     closeStream?();
65 };
66 });
67 }

```

Listing 8: AssistantService: Facade su client HTTP generato, SseClient e MvpStateStore dietro generate()

Il chiamante (`AssistantPageViewModel::generate()`, si veda il Listing successivo) vede un solo `Observable<CommunicationGenerationProgress>`: la richiesta HTTP iniziale, la connessione SSE e l'aggiornamento dello Store restano interamente dietro questo metodo.

7.2.2 Presentation Model

Intent (Fowler, POEAA): rappresentare lo stato e il comportamento della view in una classe indipendente dai widget della piattaforma, così che la view sia una proiezione sottile di quello stato invece di contenerlo.

A differenza del Facade, non è un pattern GoF ma architetturale (POEAA, come Repository o Active Record). Separa la logica applicativa di una pagina dal framework che la ospita, rendendola verificabile senza avviare l'infrastruttura di rendering: al prezzo di un file in più per ogni pagina a cui viene applicato.

Esempio nel progetto. Descritto per intero in Sezione 6.2: ogni pagina principale (AssistantPage, CopilotPage, OverviewPage) ha un *ViewModel gemello (classe TypeScript pura, costruita con `new`, senza `inject()` né riferimenti al DOM) che il Component costruisce e a cui lega l'intero template.

```
1 // apps/frontend/src/app/features/assistant/assistant-page.view-model.  
2   ts  
3 export class AssistantPageViewModel {  
4   readonly phase: WritableSignal<CommunicationGenerationPhase | "idle">  
5     = signal("idle");  
6   readonly isGenerating: Signal<boolean> = computed(  
7     () => this.phase() === "queued" || this.phase() === "generating-  
8       text" || this.phase() === "generating-cover"  
9   );  
10   readonly status = signal("In attesa di istruzioni.");  
11   readonly selectedDraftId: WritableSignal<number | null> = signal(null  
12     );  
13   readonly latestDraft: WritableSignal<GeneratedDraft | null> = signal(  
14     null);  
15  
16   // [...] altri trenta signal/computed per storico, configurazioni,  
17   // filtri, presentazione delle metriche, stati di caricamento delle  
18   // singole azioni.  
19  
20   readonly error: Signal<string | null> = computed(() => this.store.  
21     error() ?? this.store.filteredAssistantError());  
22  
23   constructor(  
24     private readonly assistant: AssistantService,  
25     private readonly store: MvpStateStore  
26   ) {}  
27  
28   generate(payload: CommunicationDraftForm): void {  
29     this.phase.set("queued");  
30     this.status.set("Generazione in corso.");  
31     this.selectedDraftId.set(null);  
32     this.latestDraft.set(null);  
33     this.rateError.set(null);  
34  
35     this.assistant.generate(payload).subscribe({  
36       next: (progress) => this.handleProgress(progress),  
37       error: (error: unknown) => {  
38         this.phase.set("failed");  
39         this.status.set(getApiErrorMessage(error, "Generazione non  
40           disponibile."));  
41       }  
42     });  
43   }  
44  
45   // [...] regenerate(), rateDraft(), uploadCover(), saveDraft(),
```

```

39     discard(),
    // saveToHistory(), toggleFavorite(), ...: stessa forma per ogni
    azione,
40     // un metodo che aggiorna i signal e chiama AssistantService (Facade)
    .
41 }

```

Listing 9: AssistantPageViewModel: classe pura costruita con `new()`, niente `inject()` né riferimenti al DOM

Le dipendenze (`AssistantService`, `MvpStateStore`) arrivano dal costruttore, non da `inject()`: la classe è istanziabile con `new AssistantPageViewModel(...)` in un test senza `TestBed`, il requisito che ha guidato la scelta di questo pattern al posto del `@Component` stesso come `ViewModel` (si veda oltre).

Il minimo richiesto sarebbe il `@Component` stesso come `ViewModel` (MVVM in senso lasco): scartato come unica implementazione perché non regge a un test costruito con `new`. Senza questo pattern, ogni test sulla logica di una pagina richiederebbe `TestBed/Injector.create()`: il costo che l'intera Sezione 6.2 spiega di voler evitare, e solo dove il beneficio lo giustifica (le tre pagine principali, non ogni componente della SPA).

8 Database

Il sistema non ha una tabella `users`: l'identità (tenant, attore) è risolta da configurazione o da header fidati a monte della richiesta, a seconda della modalità (Sezione 3), non da una tabella anagrafica locale: coerentemente, ogni tabella di dominio porta un `tenant_id` testuale invece di una chiave esterna verso un'entità utente. Le sette tabelle di dominio descritte in questa sezione escludono le tabelle infrastrutturali di Laravel (`sessions`, `cache`, `jobs`, `password_reset_tokens`), prive di rilevanza per il modello dei dati applicativo.

8.1 Schema ER

Le relazioni con vincolo di integrità referenziale dichiarato sono solo due: lo `split` produce i sotto-documenti (`original_documents` 1:N `sub_documents`, *on delete cascade*) e l'estrazione produce i dati strutturati (`sub_documents` 1:1 `extracted_data`, *on delete cascade*). Le altre relazioni visibili nel diagramma sono polimorfiche e non vincolate a livello di database: `workflow_tasks` si riferisce a un `original_documents` o a una `communications` tramite la coppia `subject_type/subject_id` (Sezione 8.4), mentre `audit_events` si riferisce a un `original_documents`, una `communications`, un `sub_documents` o una `prompt_configurations` tramite `resource_type/resource_id` (i quattro soli valori usati da chi scrive l'audit trail, Sezione 7.1.6): nessuna tabella di dominio è quindi realmente isolata dal resto. Coerentemente con l'assenza di una tabella utenti (Sezione 8), l'appartenenza al tenant è registrata come colonna `tenant_id` sulle sole tabelle radice (`communications`, `original_documents`, `prompt_configurations` e `audit_events`), mai come foreign key: `sub_documents`, `extracted_data` e `workflow_tasks` non hanno una colonna propria e ricadono nel tenant della radice a cui appartengono, verificato a livello applicativo da ogni caso d'uso (Sezione 4.7).

8.2 Schema Relazionale

A differenza dello Schema ER (Sezione 8.1), qui compaiono i tipi di colonna effettivi e le foreign key dichiarate esplicitamente: le due sole relazioni con vincolo di integrità referenziale reale (`sub_documents.original_document_id` e `extracted_data.sub_document_id`, entrambe *on delete cascade*) sono frecce continue, mentre le quattro relazioni polimorfiche restano tratteggiate, senza vincolo dichiarato nel database. Il vincolo di unicità su `prompt_configurations (tenant_id, name)` è riportato come nota nella tabella stessa, non come relazione.

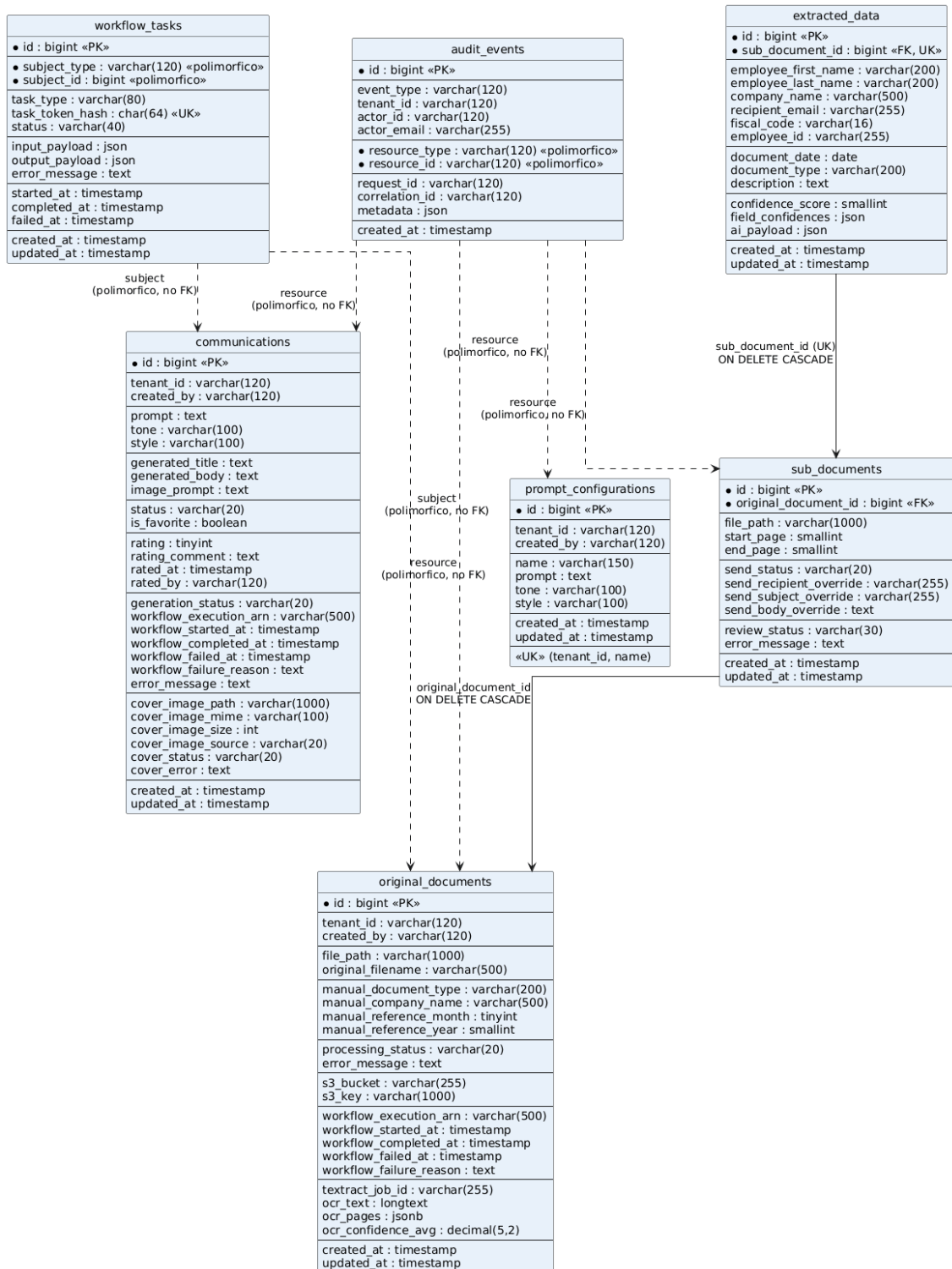


Figura 132: Schema Relazionale

8.3 Schema ERD (Entity-Relationship Diagram)

Stesse tabelle, stesse colonne e stessi vincoli della Sezione 8.2, qui ridisegnati con la cardinalità indicata direttamente sulle linee di collegamento invece che a parole: un pallino

vuoto indica cardinalità opzionale (0), una barra indica cardinalità obbligatoria (1), il ramo aperto indica “molti”.

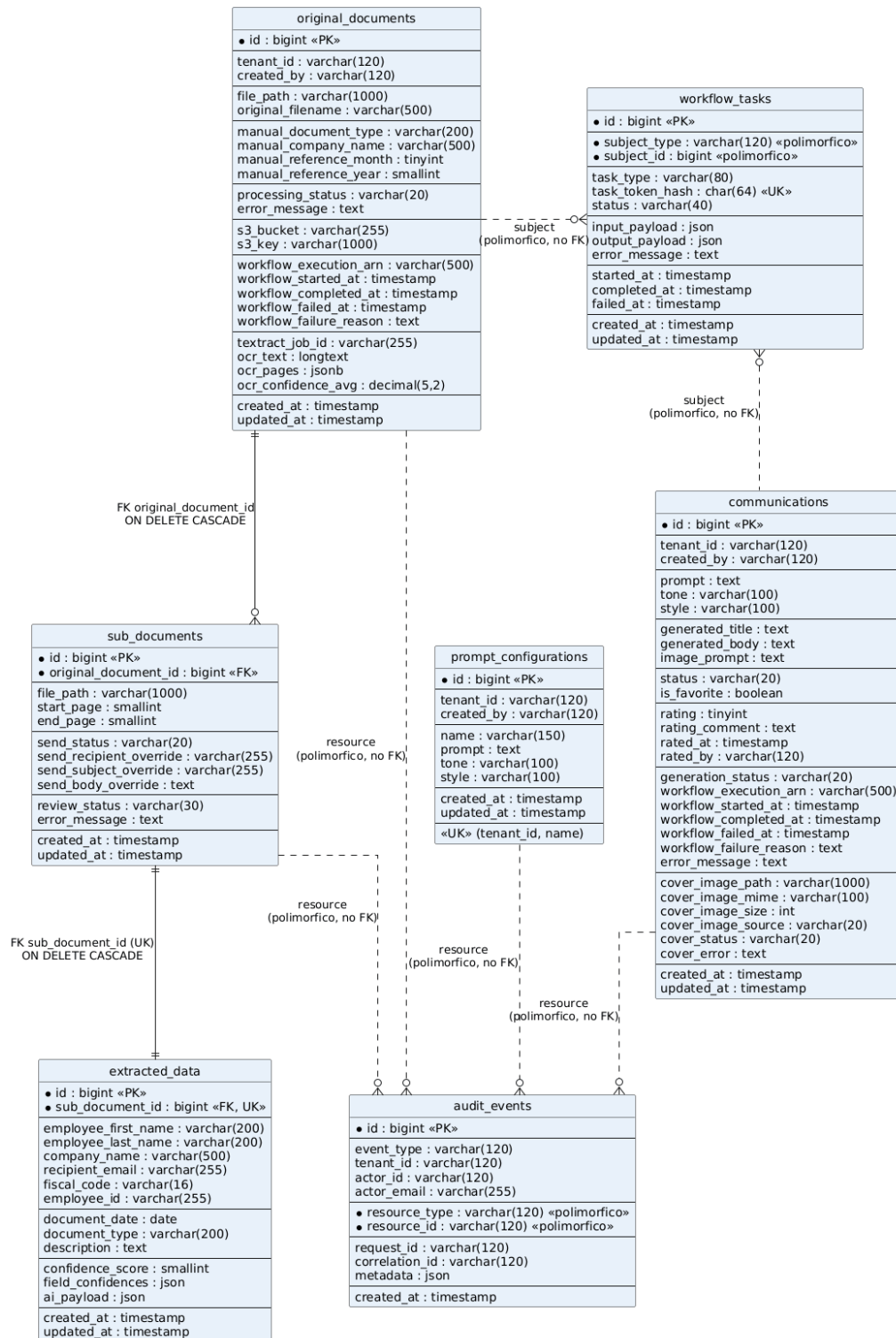


Figura 133: Schema ERD

8.4 Descrizione tabelle

8.4.1 communications

Aggregato radice del modulo AI Assistant Generativo: l'intero ciclo di vita di una bozza di comunicazione, dal prompt iniziale fino allo stato finale, vive su questa unica riga. Accumula sia lo stato editoriale deciso dall'operatore (`status`, `is_favorite`, `rating`) sia lo stato tecnico della pipeline asincrona di generazione (`generation_status`, `cover_status`, i riferimenti all'esecuzione Step Functions), perché l'entità di dominio `Communication` (Sezione 5.2) le governa entrambe nello stesso posto invece di separarle su tabelle diverse.

communications
<u>id</u>
tenant_id
created_by
prompt
tone
style
generated_title
generated_body
image_prompt
status
is_favorite
rating
rating_comment
rated_at
rated_by
generation_status
workflow_execution_arn
workflow_started_at
workflow_completed_at
workflow_failed_at
workflow_failure_reason
error_message
cover_image_path
cover_image_mime
cover_image_size
cover_image_source
cover_status
cover_error
created_at
updated_at

Figura 134: Tabella communications

Colonne

- `tenant_id`, `created_by`: scoping multi-tenant e attore, risolti da header fidati (Sezione 3), non da una foreign key verso una tabella utenti.

- `prompt`, `tone`, `style`: input della generazione.
- `generated_title`, `generated_body`, `image_prompt`: output testuale e prompt visivo prodotto per la copertina.
- `status` (*draft/approved/discarded*), `is_favorite`.
- `rating` (1-5, nullable), `rating_comment`, `rated_at`, `rated_by`.
- `generation_status` (*pending/processing/completed/failed*), `workflow_execution_arn`, `workflow_started_at/completed_at/failed_at`, `workflow_failure_reason`, `error_message`.
- `cover_image_path`, `cover_image_mime`, `cover_image_size`, `cover_image_source`, `cover_status` (*pending/processing/ready/failed/removed*), `cover_error`.

8.4.2 original_documents

Aggregato radice del modulo AI Co-Pilot: il file caricato dall'utente, il testo e le pagine riconosciute dall'OCR, e lo stato del workflow Step Functions che lo elabora fino allo split in sotto-documenti. I metadati `manual_*` (tipo documento, azienda, mese/anno di riferimento), se impostati in fase di upload, prevalgono sempre su quelli estratti successivamente dall'AI (Sezione 5.1, `UploadDocumentService`).

original_documents
<u>id</u>
tenant_id
created_by
file_path
original_filename
manual_document_type
manual_company_name
manual_reference_month
manual_reference_year
processing_status
error_message
s3_bucket
s3_key
workflow_execution_arn
workflow_started_at
workflow_completed_at
workflow_failed_at
workflow_failure_reason
textract_job_id
ocr_text
ocr_pages
ocr_confidence_avg
created_at
updated_at

Figura 135: Tabella original_documents

Colonne

- tenant_id, created_by.
- file_path, original_filename.
- manual_document_type, manual_company_name, manual_reference_month, manual_reference_year: metadati dichiarati in upload, mai sovrascritti dall'AI.
- processing_status (*pending/processing/completed/failed*), error_message.
- s3_bucket, s3_key: posizione reale su S3, distinta da file_path quando Textract reale è abilitato (Sezione 5.1, StartDocumentWorkflowService).
- workflow_execution_arn, workflow_started_at/completed_at/failed_at, workflow_failure_reason.
- textract_job_id, ocr_text, ocr_pages (jsonb: testo per pagina con relativa confidenza, usato per assegnare gli intervalli di pagina ai sotto-documenti), ocr_confidence_avg.

8.4.3 sub_documents

Un segmento del documento originale prodotto dallo split AI, associato a un singolo destinatario individuato. Governa la propria revisione (`review_status`) e il proprio stato di invio (`send_status`), oltre alle eventuali sovrascritture manuali del messaggio precompilato che l'operatore invia fuori piattaforma (Sezione 5.1, `SendMessageService`).

sub_documents	
id	
original_document_id	«FK»
file_path	
start_page	
end_page	
send_status	
send_recipient_override	
send_subject_override	
send_body_override	
review_status	
error_message	
created_at	
updated_at	

Figura 136: Tabella sub_documents

Colonne

- `original_document_id`: FK verso `original_documents`, *on delete cascade*.
- `file_path`, `start_page`, `end_page`.
- `send_status` (*pending/sent*), `send_recipient_override`, `send_subject_override`, `send_body_override`.
- `review_status` (*needs_review/auto_validated/quarantined/manually_validated*).
- `error_message`.

8.4.4 extracted_data

I campi strutturati che l'AI estrae per un singolo sotto-documento (nominativo, azienda, data, tipo documento), in relazione 1-1 con `sub_documents`. `ai_payload` conserva lo snapshot integrale e non tipizzato dell'output del modello: la revisione manuale che corregge le colonne tipizzate non cancella così il dato originale prodotto dall'AI.

extracted_data	
id	
sub_document_id	«FK, UK»
employee_first_name	
employee_last_name	
company_name	
recipient_email	
fiscal_code	
employee_id	
document_date	
document_type	
description	
confidence_score	
field_confidences	
ai_payload	
created_at	
updated_at	

Figura 137: Tabella extracted_data

Colonne

- **sub_document_id**: FK univoca verso `sub_documents`, *on delete cascade*.
- **employee_first_name**, **employee_last_name**, **company_name**, **recipient_email**, **fiscal_code**, **employee_id**.
- **document_date**, **document_type**, **description**.
- **confidence_score** (0–100, nullable): combina la confidenza media OCR con la completezza dei campi chiave trovati (Sezione 5.1, `ExtractSubDocumentFieldsService`).
- **field_confidences** (json, nullable): confidenza scomposta per singolo campo chiave, non solo il valore aggregato di **confidence_score**. Introdotta perché la confidenza restituita da Textract è quasi sempre alta di per sé: il segnale utile in pratica non era il punteggio OCR complessivo, ma la presenza o assenza dei singoli campi chiave, che riduceva l'esito a poche combinazioni discrete; scomporre la confidenza per campo rende quel segnale esplicito invece di appiattirlo in un unico numero.
- **ai_payload** (json).

8.4.5 workflow_tasks

Tabella unica dei task di callback dai worker asincroni verso Step Functions/SQS, condivisa da entrambi i domini invece di duplicata per modulo: la presa in carico atomica e la deduplicazione sul token vivono in un solo posto, e il dominio del task è identificato dalla coppia polimorfica `subject_type/subject_id` invece di una foreign key dedicata.

workflow_tasks
id subject_type «polimorfico» subject_id «polimorfico»
task_type task_token_hash status input_payload output_payload error_message started_at completed_at failed_at created_at updated_at

Figura 138: Tabella workflow_tasks

Colonne

- subject_type, subject_id: riferimento polimorfico a OriginalDocument o Communication, senza vincolo di integrità referenziale dichiarato.
- task_type, task_token_hash (univoca).
- status, input_payload, output_payload, error_message.
- started_at, completed_at, failed_at.

8.4.6 prompt_configurations

Combinazioni di prompt, tono e stile salvate dall'utente per essere riutilizzate come preset del form di generazione, indipendenti da qualunque generazione effettiva (Sezione 5.2, PromptConfigurationService). Il nome è univoco per tenant, protetto da un vincolo di unicità a livello di database oltre che dalla logica applicativa.

prompt_configurations
id
tenant_id created_by name prompt tone style created_at updated_at
«UK» (tenant_id, name)

Figura 139: Tabella prompt_configurations

Colonne

- `tenant_id`, `created_by`.
- `name`, `prompt`, `tone`, `style`.

8.4.7 `audit_events`

Log di audit trasversale a entrambi i domini, popolato in prevalenza dai listener degli eventi di dominio (Sezione 7.1.6, Observer) invece che da chiamate sparse nei singoli casi d'uso; restano due sole scritture dirette, dalla validazione dell'upload e dall'esecutore dei task di workflow. Ogni riga registra chi ha compiuto un'azione, su quale risorsa, correlata alla richiesta HTTP che l'ha generata.

audit_events	
<u>id</u>	
event_type	
tenant_id	
actor_id	
actor_email	
resource_type	«polimorfico»
resource_id	«polimorfico»
request_id	
correlation_id	
metadata	
created_at	

Figura 140: Tabella `audit_events`

Colonne

- `event_type`.
- `tenant_id`, `actor_id`, `actor_email`.
- `resource_type`, `resource_id`: colonne testuali, non una foreign key: identificano la risorsa coinvolta senza un vincolo relazionale.
- `request_id`, `correlation_id`: propagati dagli stessi header di correlazione usati dallo stream SSE (Sezione 6.4).
- `metadata` (json).

9 Requisiti Funzionali Soddisfatti

Questo capitolo riporta lo stato di implementazione di ciascun requisito funzionale identificato nell'Analisi dei Requisiti, verificato direttamente sul codice sorgente del sistema alla data di redazione di questo documento. I requisiti funzionali si dividono in obbligatori e

opzionali a seconda della loro importanza per il progetto: ne sono stati identificati 109, di cui 94 obbligatori e 15 opzionali, derivati dall'analisi dei casi d'uso (Analisi dei Requisiti, Sezione 4.1). Tutti i requisiti obbligatori risultano soddisfatti.

9.1 Completamento requisiti

- **Requisiti Obbligatori:** 94/94 requisiti funzionali soddisfatti.
- **Requisiti Opzionali:** 11/15 requisiti funzionali soddisfatti.

I quattro requisiti opzionali non ancora soddisfatti alla data di redazione:

- **RF4-OP:** nessun parametro di lunghezza della bozza né nella richiesta di generazione (`GenerateCommunicationCommand`) né nel form di generazione.
- **RF16-OP:** nessun metodo di ripristino del contenuto generato dall'AI dopo una modifica manuale; `CommunicationDraftService::update()` (Sezione 5.2) sovrascrive titolo e corpo senza conservare una copia del contenuto originale.
- **RF20-OP:** nessuna funzionalità di inoltra via e-mail per una comunicazione generata; nel modulo Co-Pilot il messaggio precompilato (`SendMessageService`, Sezione 5.1) non viene inviato via e-mail dalla piattaforma, ma scaricato dall'operatore e recapitato fuori dalla piattaforma, con lo stato marcato come inviato al download.
- **RF107-OP:** il campo e-mail del destinatario, se assente, è mostrato come casella vuota anziché con un indicatore esplicito di dato non disponibile, a differenza degli altri campi dello stesso pannello (nome, azienda, data, tipologia, descrizione, confidenza), che lo hanno.

9.2 Tabella di tracciamento dei requisiti

Codice	Descrizione	Stato	Priorità
<i>AI Assistant Generativo</i>			
RF1-OB	Il sistema deve permettere al Redattore di inserire il prompt testuale per la generazione del contenuto.	Implementato	Obbligatorio
RF2-OB	Il sistema deve permettere al Redattore di selezionare il tono comunicativo della bozza da generare.	Implementato	Obbligatorio
RF3-OB	Il sistema deve permettere al Redattore di selezionare lo stile editoriale della bozza da generare.	Implementato	Obbligatorio
RF4-OP	Il sistema deve permettere al Redattore di selezionare la lunghezza desiderata per la bozza da generare.	Non implementato	Opzionale
RF5-OB	Il sistema deve avviare la generazione del contenuto tramite AI a partire dal prompt e dai parametri configurati.	Implementato	Obbligatorio
RF6-OB	Il sistema deve visualizzare un'anteprima della bozza generata.	Implementato	Obbligatorio

Codice	Descrizione	Stato	Priorità
RF7-OB	Il sistema deve visualizzare il titolo della bozza generata nell'anteprima.	Implementato	Obbligatorio
RF8-OB	Il sistema deve visualizzare il testo della bozza generata nell'anteprima.	Implementato	Obbligatorio
RF9-OB	Il sistema deve visualizzare l'immagine di copertina della bozza generata nell'anteprima.	Implementato	Obbligatorio
RF10-OB	Il sistema deve permettere al Redattore di modificare manualmente il titolo della bozza.	Implementato	Obbligatorio
RF11-OB	Il sistema deve permettere al Redattore di modificare manualmente il testo della bozza.	Implementato	Obbligatorio
RF12-OB	Il sistema deve permettere al Redattore di sostituire l'immagine di copertina della bozza con un file caricato localmente.	Implementato	Obbligatorio
RF13-OB	Il sistema deve permettere al Redattore di eliminare l'immagine di copertina della bozza.	Implementato	Obbligatorio
RF14-OB	Il sistema deve permettere al Redattore di richiedere la rigenerazione della bozza mantenendo i parametri di configurazione.	Implementato	Obbligatorio
RF15-OB	Il sistema deve permettere al Redattore di eliminare la bozza generata corrente.	Implementato	Obbligatorio
RF16-OP	Il sistema deve permettere al Redattore di annullare le modifiche manuali apportate alla bozza, ripristinando il contenuto originariamente generato.	Non implementato	Opzionale
RF17-OB	Il sistema deve permettere il salvataggio della bozza nello storico.	Implementato	Obbligatorio
RF18-OP	Il sistema deve permettere al Redattore di visualizzare un'anteprima del documento prima dell'esportazione definitiva.	Implementato	Opzionale
RF19-OB	Il sistema deve permettere la pubblicazione del documento generato e la sua esportazione in formato PDF.	Implementato	Obbligatorio
RF20-OP	Il sistema deve permettere al Redattore di inoltrare il documento finalizzato tramite posta elettronica.	Non implementato	Opzionale
RF21-OB	Il sistema deve fornire al Redattore la libreria personale con lo storico delle generazioni passate e i prompt salvati.	Implementato	Obbligatorio
RF22-OB	Il sistema deve permettere la ricerca e il filtraggio dello storico delle generazioni e dei prompt salvati.	Implementato	Obbligatorio
RF23-OB	Il sistema deve permettere al Redattore di filtrare lo storico delle generazioni tramite ricerca per parola chiave.	Implementato	Obbligatorio
RF24-OB	Il sistema deve permettere al Redattore di filtrare lo storico delle generazioni per tono comunicativo.	Implementato	Obbligatorio
RF25-OB	Il sistema deve permettere al Redattore di filtrare lo storico delle generazioni per stile editoriale.	Implementato	Obbligatorio
RF26-OB	Il sistema deve permettere al Redattore di filtrare lo storico delle generazioni per data.	Implementato	Obbligatorio

Codice	Descrizione	Stato	Priorità
RF27-OB	Il sistema deve permettere al Redattore di salvare la configurazione corrente del prompt nella libreria personale, assegnandole un nome identificativo.	Implementato	Obbligatorio
RF28-OB	Il sistema deve permettere il caricamento e il riutilizzo di una configurazione prompt salvata.	Implementato	Obbligatorio
RF29-OB	Il sistema deve permettere al Redattore di contrassegnare una generazione come preferita nella libreria personale.	Implementato	Obbligatorio
RF30-OB	Il sistema deve permettere al Redattore di rimuovere una generazione dai preferiti nella libreria personale.	Implementato	Obbligatorio
RF31-OP	Il sistema deve permettere l'eliminazione di elementi dallo storico e dalla libreria dei prompt.	Implementato	Opzionale
RF32-OB	Il sistema deve permettere al Redattore di assegnare un rating numerico alla bozza generata.	Implementato	Obbligatorio
RF33-OB	Il sistema deve permettere al Redattore di aggiungere un commento testuale qualitativo alla valutazione della bozza.	Implementato	Obbligatorio
RF34-OB	Il sistema deve fornire all'Analista dati una dashboard con metriche dell'AI Assistant, con possibilità di esportare un report riepilogativo.	Implementato	Obbligatorio
RF35-OB	Il sistema deve visualizzare nella dashboard le statistiche di utilizzo del modulo AI Assistant.	Implementato	Obbligatorio
RF36-OB	Il sistema deve visualizzare nella dashboard il rating medio delle bozze generate.	Implementato	Obbligatorio
RF37-OB	Il sistema deve permettere all'Analista dati di consultare i feedback testuali ricevuti dagli utenti.	Implementato	Obbligatorio
RF38-OB	Il sistema deve permettere all'Analista dati di applicare filtri dinamici ai dati visualizzati nella dashboard analitica.	Implementato	Obbligatorio
RF39-OB	Il sistema deve permettere all'Analista dati di filtrare i dati della dashboard per intervallo di date.	Implementato	Obbligatorio
RF40-OB	Il sistema deve permettere all'Analista dati di filtrare i dati della dashboard per tono comunicativo.	Implementato	Obbligatorio
RF41-OB	Il sistema deve permettere all'Analista dati di filtrare i dati della dashboard per stile editoriale.	Implementato	Obbligatorio
<i>AI Co-Pilot per Consulenti del Lavoro</i>			
RF42-OB	Il sistema deve permettere al Consulente del lavoro di caricare un singolo documento per l'analisi AI automatica.	Implementato	Obbligatorio
RF43-OB	Il sistema deve permettere al Consulente del lavoro di caricare documenti con classificazione e metadati inseriti manualmente.	Implementato	Obbligatorio
RF44-OB	Il sistema deve permettere la selezione della tipologia del documento durante il caricamento manuale.	Implementato	Obbligatorio
RF45-OB	Il sistema deve permettere l'inserimento del mese di riferimento del documento durante il caricamento manuale.	Implementato	Obbligatorio

Codice	Descrizione	Stato	Priorità
RF46-OB	Il sistema deve permettere l'inserimento dell'anno di riferimento del documento durante il caricamento manuale.	Implementato	Obbligatorio
RF47-OB	Il sistema deve permettere l'inserimento dell'azienda di riferimento del documento durante il caricamento manuale.	Implementato	Obbligatorio
RF48-OB	Il sistema deve permettere al Consulente del lavoro di visualizzare lo storico dei documenti analizzati.	Implementato	Obbligatorio
RF49-OB	Il sistema deve permettere il filtraggio dello storico documenti tramite criteri multipli.	Implementato	Obbligatorio
RF50-OB	Il sistema deve permettere il filtraggio dello storico documenti tramite barra di ricerca testuale.	Implementato	Obbligatorio
RF51-OB	Il sistema deve permettere il filtraggio dello storico documenti per stato di scaricamento.	Implementato	Obbligatorio
RF52-OB	Il sistema deve permettere il filtraggio dello storico documenti per livello di confidenza dell'analisi AI.	Implementato	Obbligatorio
RF53-OB	Il sistema deve permettere il filtraggio dello storico documenti per mese e anno.	Implementato	Obbligatorio
RF54-OB	Il sistema deve permettere al Consulente del lavoro di visualizzare i dettagli di un singolo documento dallo storico.	Implementato	Obbligatorio
RF55-OB	Il sistema deve visualizzare la porzione di documento relativa al dipendente (versione splittata).	Implementato	Obbligatorio
RF56-OB	Il sistema deve visualizzare il documento originale completo (non splittato).	Implementato	Obbligatorio
RF57-OB	Il sistema deve visualizzare il nome e cognome del dipendente associato al documento.	Implementato	Obbligatorio
RF58-OB	Il sistema deve visualizzare l'azienda associata al documento.	Implementato	Obbligatorio
RF59-OB	Il sistema deve visualizzare il nome del file del documento.	Implementato	Obbligatorio
RF60-OB	Il sistema deve visualizzare la data del documento.	Implementato	Obbligatorio
RF61-OB	Il sistema deve visualizzare il numero di pagine del documento.	Implementato	Obbligatorio
RF62-OB	Il sistema deve visualizzare la tipologia del documento.	Implementato	Obbligatorio
RF63-OB	Il sistema deve visualizzare la breve descrizione del documento.	Implementato	Obbligatorio
RF64-OB	Il sistema deve visualizzare il livello di confidenza in percentuale del documento.	Implementato	Obbligatorio
RF65-OP	Il sistema deve visualizzare lo stato del documento, comprensivo di stato di revisione e stato di scaricamento.	Implementato	Opzionale
RF66-OP	Il sistema deve visualizzare l'e-mail del destinatario del documento.	Implementato	Opzionale
RF67-OP	Il sistema deve visualizzare il codice fiscale del destinatario.	Implementato	Opzionale
RF68-OP	Il sistema deve visualizzare la matricola del dipendente.	Implementato	Opzionale

Codice	Descrizione	Stato	Priorità
RF69-OP	Il sistema deve visualizzare la data e ora di caricamento del documento.	Implementato	Opzionale
RF70-OB	Il sistema deve permettere al Consulente del lavoro di modificare il nome e cognome del dipendente associato al documento.	Implementato	Obbligatorio
RF71-OB	Il sistema deve permettere la modifica dell'azienda associata al documento.	Implementato	Obbligatorio
RF72-OB	Il sistema deve permettere la modifica della data del documento.	Implementato	Obbligatorio
RF73-OB	Il sistema deve permettere la modifica della tipologia del documento.	Implementato	Obbligatorio
RF74-OB	Il sistema deve permettere la modifica della breve descrizione del documento.	Implementato	Obbligatorio
RF75-OP	Il sistema deve permettere la modifica dell'indirizzo e-mail del destinatario.	Implementato	Opzionale
RF76-OP	Il sistema deve permettere la modifica del codice fiscale del destinatario.	Implementato	Opzionale
RF77-OP	Il sistema deve permettere la modifica della matricola del dipendente.	Implementato	Opzionale
RF78-OB	Il sistema deve permettere al Consulente del lavoro di annullare le modifiche dei dati del singolo documento.	Implementato	Obbligatorio
RF79-OB	Il sistema deve permettere al Consulente del lavoro di eliminare il singolo documento.	Implementato	Obbligatorio
RF80-OB	Il sistema deve generare automaticamente il contenuto del messaggio precompilato per il documento da spedire.	Implementato	Obbligatorio
RF81-OB	Il sistema deve visualizzare il destinatario del messaggio precompilato generato automaticamente.	Implementato	Obbligatorio
RF82-OB	Il sistema deve visualizzare l'oggetto del messaggio precompilato generato automaticamente.	Implementato	Obbligatorio
RF83-OB	Il sistema deve visualizzare il testo del messaggio precompilato generato automaticamente.	Implementato	Obbligatorio
RF84-OB	Il sistema deve permettere la conferma e lo scaricamento del messaggio precompilato.	Implementato	Obbligatorio
RF85-OB	Il sistema deve permettere al Consulente del lavoro di modificare il destinatario del messaggio precompilato.	Implementato	Obbligatorio
RF86-OB	Il sistema deve permettere la modifica dell'oggetto del messaggio precompilato.	Implementato	Obbligatorio
RF87-OB	Il sistema deve permettere la modifica del testo del messaggio precompilato.	Implementato	Obbligatorio
RF88-OB	Il sistema deve fornire all'Analista dati una dashboard dedicata con metriche di monitoraggio del modulo AI Co-Pilot.	Implementato	Obbligatorio
RF89-OB	Il sistema deve visualizzare nella dashboard del Co-Pilot il numero totale di documenti analizzati.	Implementato	Obbligatorio
RF90-OB	Il sistema deve visualizzare nella dashboard del Co-Pilot la percentuale di classificazioni corrette.	Implementato	Obbligatorio

Codice	Descrizione	Stato	Priorità
RF91-OB	Il sistema deve visualizzare nella dashboard del Co-Pilot i documenti sotto soglia di confidenza.	Implementato	Obbligatorio
RF92-OB	Il sistema deve visualizzare nella dashboard del Co-Pilot la percentuale di destinatari riconosciuti automaticamente.	Implementato	Obbligatorio
RF93-OB	Il sistema deve visualizzare nella dashboard del Co-Pilot il tempo medio di analisi dei documenti.	Implementato	Obbligatorio

Gestione Errori

RF94-OB	Il sistema deve notificare il Redattore in caso di formato file non valido durante la sostituzione dell'immagine di copertina.	Implementato	Obbligatorio
RF95-OB	Il sistema deve notificare l'utente in caso di indirizzo e-mail non valido durante l'inoltro del documento.	Implementato	Obbligatorio
RF96-OB	Il sistema deve notificare il Redattore quando la ricerca nello storico delle generazioni non produce risultati.	Implementato	Obbligatorio
RF97-OB	Il sistema deve notificare il Redattore quando il commento testuale supera il limite di caratteri consentito.	Implementato	Obbligatorio
RF98-OB	Il sistema deve notificare l'Analista dati quando i dati analitici non sono disponibili per i criteri selezionati.	Implementato	Obbligatorio
RF99-OB	Il sistema deve rilevare i formati file non supportati durante l'upload e notificare il Consulente del lavoro.	Implementato	Obbligatorio
RF100-OB	Il sistema deve notificare l'utente quando nessun documento corrisponde ai criteri di visualizzazione dello storico.	Implementato	Obbligatorio
RF101-OB	Il sistema deve gestire il caso in cui il nome e cognome del dipendente non siano disponibili, visualizzando un opportuno indicatore.	Implementato	Obbligatorio
RF102-OB	Il sistema deve gestire il caso in cui l'azienda non sia disponibile, visualizzando un opportuno indicatore.	Implementato	Obbligatorio
RF103-OB	Il sistema deve gestire il caso in cui la data del documento non sia disponibile, visualizzando un opportuno indicatore.	Implementato	Obbligatorio
RF104-OB	Il sistema deve gestire il caso in cui la tipologia del documento non sia disponibile, visualizzando un opportuno indicatore.	Implementato	Obbligatorio
RF105-OB	Il sistema deve gestire il caso in cui la breve descrizione non sia disponibile, visualizzando un opportuno indicatore.	Implementato	Obbligatorio
RF106-OB	Il sistema deve gestire il caso in cui il livello di confidenza non sia disponibile, visualizzando un opportuno indicatore.	Implementato	Obbligatorio
RF107-OP	Il sistema deve gestire il caso in cui l'e-mail del destinatario non sia disponibile, visualizzando un opportuno indicatore.	Non implementato	Opzionale
RF108-OP	Il sistema deve notificare il Consulente del lavoro in caso di codice fiscale non valido durante la modifica.	Implementato	Opzionale

Codice	Descrizione	Stato	Priorità
RF109-OB	Il sistema deve notificare il Redattore quando il testo del prompt è mancante o insufficiente per avviare la generazione.	Implementato	Obbligatorio

Tabella 16: Tabella di tracciamento dei requisiti funzionali